# CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

*Data Structures and Algorithms*

**Hiểu Bản Chất • Nắm Vững Ứng Dụng • Thành Thạo Kỹ Năng**

Dành cho sinh viên Khoa học Máy tính

Ngôn ngữ minh họa: C++

2026

# MỤC LỤC

1. Lời nói đầu . . . . . 12
2. Giới thiệu về Cấu trúc Dữ liệu và Giải thuật . . . . . 13
2.1. Tổng quan chương . . . . . 13
2.1.1. Nội dung chính của chương . . . . . 13
2.1.2. Kiến thức nền cần biết trước . . . . . 13
2.2. Giải thích từng khái niệm . . . . . 14
2.2.1. Data (Dữ liệu) . . . . . 14
2.2.2. Data Type (Kiểu dữ liệu) . . . . . 15
2.2.3. Data Structure (Cấu trúc dữ liệu) . . . . . 16
2.2.4. Abstract Data Type (Kiểu dữ liệu trừu tượng - ADT) . . . . . 18
2.2.5. Algorithm (Giải thuật) . . . . . 20
2.2.6. Pseudocode (Mã giả) . . . . . 22
2.3. Bản chất trong máy tính . . . . . 24
2.3.1. Cách dữ liệu được lưu trong memory . . . . . 24
2.3.2. CPU xử lý dữ liệu . . . . . 25
2.4. Lịch sử / Nguồn gốc . . . . . 26
2.4.1. Nguồn gốc khái niệm Data Structure . . . . . 26
2.4.2. Tại sao có ADT? . . . . . 26
2.5. Phân tích thuật toán . . . . . 27
2.6. Minh họa từng bước . . . . . 28
2.7. Code minh họa bằng C++ . . . . . 29
2.7.1. Struct (Cấu trúc) . . . . . 29
2.7.2. Class với Constructor . . . . . 29
2.7.3. Pointers (Con trỏ) . . . . . 30
2.7.4. Arrays (Mảng) . . . . . 31
2.7.5. Pointer to Struct . . . . . 31
2.7.6. Pointer to Class . . . . . 32
2.8. Những lỗi phổ biến khi code . . . . . 34
2.8.1. Lỗi với Pointers . . . . . 34
2.8.2. Lỗi với Arrays . . . . . 35
2.8.3. Lỗi với Struct/Class . . . . . 35
2.9. Khi nào nên dùng / không nên dùng . . . . . 36
2.9.1. Struct vs Class . . . . . 36

2.9.2. Array vs Pointer . . . . . 36
2.10. Ứng dụng thực tế . . . . . 38
2.10.1. Data Structures trong Hệ điều hành . . . . . 38
2.10.2. Data Structures trong Database . . . . . 38
2.10.3. Data Structures trong Compiler . . . . . 38
2.10.4. Data Structures trong Web . . . . . 38
2.10.5. Data Structures trong Game . . . . . 39
2.11. Tóm tắt chương . . . . . 40
3. Độ phức tạp giải thuật (Algorithm Complexity) . . . . . 41
3.1. Tổng quan chương . . . . . 41
3.1.1. Nội dung chính của chương . . . . . 41
3.1.2. Kiến thức nền cần biết trước . . . . . 41
3.2. Giải thích từng khái niệm . . . . . 42
3.2.1. Algorithm Efficiency (Hiệu quả giải thuật) . . . . . 42
3.2.2. Big-O Notation . . . . . 43
3.2.3. Các loại Complexity Classes . . . . . 44
3.2.4. Phân tích các loại vòng lặp . . . . . 46
3.2.5. Best, Worst, Average Case . . . . . 49
3.3. Bản chất trong máy tính . . . . . 50
3.3.1. Tại sao Big-O quan trọng? . . . . . 50
3.3.2. Cache và Memory Access . . . . . 50
3.4. Lịch sử / Nguồn gốc . . . . . 52
3.4.1. Sự ra đời của Big-O Notation . . . . . 52
3.4.2. Tại sao cần phân tích độ phức tạp? . . . . . 52
3.5. Phân tích thuật toán . . . . . 53
3.5.1. Binary Search (Tìm kiếm nhị phân) . . . . . 53
3.5.2. Sequential Search (Tìm kiếm tuần tự) . . . . . 54
3.5.3. Quick Sort (Sắp xếp nhanh) . . . . . 54
3.6. Minh họa từng bước . . . . . 56
3.6.1. Binary Search - Tìm số 21 . . . . . 56
3.6.2. Sequential Search - Tìm số 7 . . . . . 57
3.7. Code minh họa bằng C++ . . . . . 58
3.7.1. Binary Search . . . . . 58
3.7.2. Sequential Search . . . . . 60
3.7.3. Code phân tích các loại vòng lặp . . . . . 61

3.7.4. Đo thời gian thực tế . . . . . 63
3.8. Những lỗi phổ biến khi code . . . . . 64
3.9. Khi nào nên dùng / không nên dùng . . . . . 66
3.9.1. Binary Search vs Sequential Search . . . . . 66
3.9.2. So sánh Complexity Classes . . . . . 66
3.10. Ứng dụng thực tế . . . . . 67
3.10.1. Hệ điều hành (Operating System) . . . . . 67
3.10.2. Database . . . . . 67
3.10.3. Web Applications . . . . . 67
3.10.4. Game Development . . . . . 67
3.10.5. Machine Learning / AI . . . . . 68
3.11. Tóm tắt chương . . . . . 69
4. Đệ quy (Recursion) . . . . . 71
4.1. Tổng quan chương . . . . . 71
4.1.1. Nội dung chính của chương . . . . . 71
4.1.2. Kiến thức nền cần biết trước . . . . . 71
4.2. Giải thích từng khái niệm . . . . . 72
4.2.1. Recursion (Đệ quy) . . . . . 72
4.2.2. Thành phần cơ bản của Đệ quy . . . . . 74
4.2.3. Properties of Recursion (Tính chất của Đệ quy) . . . . . 75
4.3. Bản chất trong máy tính . . . . . 76
4.3.1. Call Stack trong Đệ quy . . . . . 76
4.3.2. Đệ quy vs Iteration - Hiệu suất . . . . . 76
4.4. Lịch sử / Nguồn gốc . . . . . 78
4.4.1. Lịch sử của Đệ quy . . . . . 78
4.4.2. Towers of Hanoi - Câu chuyện . . . . . 78
4.5. Phân tích thuật toán . . . . . 79
4.5.1. Factorial (Giai thừa) . . . . . 79
4.5.2. Fibonacci Numbers . . . . . 80
4.5.3. Towers of Hanoi (Tháp Hà Nội) . . . . . 81
4.5.4. GCD - Euclidean Algorithm (Ước số chung lớn nhất) . . . . . 83
4.6. Minh họa từng bước . . . . . 84
4.6.1. Factorial(3) - Chi tiết . . . . . 84
4.6.2. Towers of Hanoi với 3 đĩa . . . . . 86
4.7. Code minh họa bằng C++ . . . . . 87

4.7.1. Factorial - Đệ quy vs Iteration ..... 87
4.7.2. Fibonacci - Đệ quy (Naive) ..... 88
4.7.3. Fibonacci - Tối ưu với Memoization ..... 89
4.7.4. Towers of Hanoi ..... 90
4.7.5. GCD (Euclidean Algorithm) ..... 92
4.7.6. Print Linked List in Reverse (Đệ quy) ..... 93
4.8. Những lỗi phổ biến khi code ..... 95
4.9. Khi nào nên dùng / không nên dùng ..... 97
4.9.1. Khi nào dùng Đệ quy? ..... 97
4.9.2. Khi nào KHÔNG nên dùng Đệ quy? ..... 97
4.9.3. So sánh Recursion vs Iteration ..... 98
4.10. Ứng dụng thực tế ..... 99
4.11. Tóm tắt chương ..... 102
5. Danh sách liên kết (Linked Lists) ..... 104
5.1. Tổng quan chương ..... 104
5.1.1. Nội dung chính của chương ..... 104
5.1.2. Kiến thức nền cần biết trước ..... 104
5.2. Giải thích từng khái niệm ..... 105
5.2.1. Linear List (Danh sách tuyến tính) ..... 105
5.2.2. List ADT (Abstract Data Type) ..... 107
5.2.3. Array-based List (Danh sách dựa trên mảng) ..... 108
5.2.4. Singly Linked List (Danh sách liên kết đơn) ..... 110
5.3. Bản chất trong máy tính ..... 112
5.3.1. Array-based List trong Memory ..... 112
5.3.2. Linked List trong Memory ..... 112
5.4. Lịch sử / Nguồn gốc ..... 114
5.4.1. Lịch sử của List ..... 114
5.5. Phân tích thuật toán ..... 115
5.5.1. Array-based List Operations ..... 115
5.5.2. Singly Linked List Operations ..... 115
5.6. Minh họa từng bước ..... 118
5.6.1. Array List - Insert at position 2 ..... 118
5.6.2. Linked List - Insert at head ..... 118
5.6.3. Linked List - Delete first node ..... 118
5.7. Code minh họa bằng C++ ..... 120

5.7.1. Dynamic Array List - Complete Implementation ..... 120
5.7.2. Singly Linked List - Complete Implementation ..... 124
5.8. Những lỗi phổ biến khi code ..... 129
5.9. Khi nào nên dùng / không nên dùng ..... 131
5.9.1. ArrayList vs LinkedList ..... 131
5.9.2. Khi nào dùng ArrayList? ..... 131
5.9.3. Khi nào dùng LinkedList? ..... 131
5.10. Ứng dụng thực tế ..... 133
5.11. Tóm tắt chương ..... 137
6. Stack và Queue ..... 139
6.1. Tổng quan chương ..... 139
6.1.1. Nội dung chính ..... 139
6.1.2. Kiến thức nền ..... 139
6.1.3. Linear List Concepts ..... 139
6.2. Basic Operations of Stacks ..... 140
6.2.1. Định nghĩa Stack ..... 140
6.2.2. Các thao tác cơ bản ..... 140
6.2.3. Tại sao cần Stack? ..... 141
6.3. Implementation of Stacks ..... 142
6.3.1. Linked-list Implementation ..... 142
6.3.2. Create an Empty Linked Stack ..... 143
6.3.3. Push Data into a Linked Stack ..... 144
6.3.4. Pop Linked Stack ..... 145
6.3.5. Stack Top ..... 146
6.3.6. Destroy Stack ..... 146
6.3.7. isEmpty và isFull Linked Stack ..... 148
6.3.8. Print Stack ..... 148
6.3.9. Using Linked Stack ..... 150
6.4. Array-based Stack Implementation ..... 151
6.4.1. Nguyên lý ..... 151
6.4.2. C++ Implementation ..... 151
6.4.3. Using Array-based Stack ..... 153
6.4.4. So sánh Array vs Linked Stack ..... 153
6.5. Applications of Stack ..... 154
6.6. Basic Operations of Queues ..... 156

6.6.1. Định nghĩa Queue ..... 156
6.6.2. Các thao tác cơ bản ..... 156
6.6.3. Tại sao cần Queue? ..... 156
6.7. Implementation of Queue ..... 157
6.7.1. Linked-list Implementation ..... 157
6.7.2. Create Queue ..... 159
6.7.3. Enqueue ..... 159
6.7.4. Dequeue ..... 161
6.7.5. Queue Front and Rear ..... 162
6.7.6. Destroy Queue ..... 163
6.7.7. Queue Empty ..... 163
6.7.8. Print Queue ..... 165
6.7.9. Using Linked Queue ..... 165
6.8. Array-based Queue Implementation ..... 166
6.8.1. C++ Implementation ..... 166
6.8.2. Using Array-based Queue ..... 168
6.8.3. So sánh Array vs Linked Queue ..... 168
6.9. Applications of Queue ..... 169
6.10. Lỗi phổ biến ..... 170
6.11. Khi nào dùng Stack và Queue ..... 171
6.12. Tóm tắt ..... 172
7. Cây (Tree) ..... 173
7.1. Tổng quan chương ..... 173
7.1.1. Nội dung chính của chương ..... 173
7.1.2. Kiến thức nền cần biết trước ..... 173
7.2. Giải thích từng khái niệm ..... 174
7.2.1. Tree (Cây) ..... 174
7.2.2. Binary Tree (Cây nhị phân) ..... 176
7.2.3. Tree Traversals (Duyệt cây) ..... 178
7.2.4. Binary Search Tree (BST) ..... 180
7.3. Bản chất trong máy tính ..... 181
7.3.1. Memory Layout của Binary Tree ..... 181
7.3.2. Recursion trong Tree Operations ..... 183
7.4. Lịch sử / Nguồn gốc ..... 184
7.4.1. Lịch sử của Tree Structure ..... 184

7.4.2. Tại sao Trees ra đời? . . . 184
7.5. Phân tích thuật toán . . . 185
7.5.1. BST Search . . . 185
7.5.2. BST Insert . . . 185
7.5.3. BST Delete . . . 186
7.5.4. Tree Traversals . . . 186
7.6. Minh họa từng bước . . . 187
7.6.1. BST Insert . . . 187
7.6.2. BST Delete node with 2 children . . . 187
7.6.3. Traversals . . . 188
7.7. Code minh họa bằng C++ . . . 190
7.7.1. Binary Tree Node Structure . . . 190
7.7.2. Binary Search Tree - Complete Implementation . . . 190
7.8. Những lỗi phổ biến khi code . . . 196
7.9. Khi nào nên dùng / không nên dùng . . . 198
7.9.1. Khi nào dùng Binary Search Tree? . . . 198
7.9.2. Khi nào KHÔNG nên dùng BST? . . . 198
7.9.3. So sánh với các cấu trúc khác . . . 198
7.10. Ứng dụng thực tế . . . 200
7.11. Tóm tắt chương . . . 203
8. Cây AVL (AVL Tree) . . . 205
8.1. Tổng quan chương . . . 205
8.1.1. Nội dung chính . . . 205
8.1.2. Kiến thức nền . . . 205
8.2. Giải thích từng khái niệm . . . 206
8.2.1. AVL Tree . . . 206
8.2.2. Rotations (Xoay cây) . . . 208
8.2.3. 4 Cases Unbalanced . . . 210
8.3. Bản chất trong máy tính . . . 211
8.3.1. AVL Node Structure . . . 211
8.3.2. Rotation Operations . . . 211
8.4. Lịch sử / Nguồn gốc . . . 212
8.5. Phân tích thuật toán . . . 213
8.5.1. AVL Insert . . . 213
8.5.2. AVL Delete . . . 214

8.5.3. Rotations ..... 214
8.6. Minh họa từng bước ..... 215
8.6.1. AVL Insert với Rotation ..... 215
8.6.2. Left-Right Case ..... 215
8.7. Code minh họa C++ ..... 217
8.8. Lỗi phổ biến ..... 223
8.9. Khi nào dùng AVL vs BST vs Red-Black ..... 224
8.10. Ứng dụng thực tế ..... 225
8.11. Tóm tắt ..... 226
9. Heap ..... 227
9.1. Tổng quan chương ..... 227
9.1.1. Nội dung chính ..... 227
9.1.2. Kiến thức nền ..... 227
9.2. Giải thích từng khái niệm ..... 228
9.2.1. Heap ..... 228
9.2.2. Heap Array Representation ..... 230
9.2.3. ReheapUp và ReheapDown ..... 231
9.3. Bản chất trong máy tính ..... 233
9.3.1. Memory Layout ..... 233
9.3.2. Cache Performance ..... 233
9.4. Lịch sử / Nguồn gốc ..... 234
9.5. Phân tích thuật toán ..... 235
9.5.1. Insert into Heap ..... 235
9.5.2. Delete Max (Root) ..... 235
9.5.3. Build Heap ..... 235
9.5.4. Heap Sort ..... 236
9.6. Minh họa từng bước ..... 237
9.6.1. Build Heap từ array ..... 237
9.6.2. Heap Sort ..... 238
9.7. Code minh họa C++ ..... 239
9.8. Lỗi phổ biến ..... 244
9.9. Khi nào dùng / không dùng ..... 246
9.9.1. Khi nào dùng Heap? ..... 246
9.9.2. Heap vs BST vs Array ..... 246
9.10. Ứng dụng thực tế ..... 247

9.11. Tóm tắt . . . . . 250
10. Bảng băm (Hash Table) . . . . . 252
10.1. Tổng quan chương . . . . . 252
10.1.1. Nội dung chính . . . . . 252
10.1.2. Kiến thức nền . . . . . 252
10.2. Giải thích từng khái niệm . . . . . 253
10.2.1. Hash Table và Hash Function . . . . . 253
10.2.2. Hash Functions . . . . . 255
10.2.3. Collision (Đụng độ) . . . . . 257
10.2.4. Open Addressing . . . . . 258
10.2.5. Chaining (Separate Chaining) . . . . . 258
10.2.6. Load Factor . . . . . 260
10.3. Bản chất trong máy tính . . . . . 261
10.3.1. Memory Layout . . . . . 261
10.3.2. Cache Performance . . . . . 261
10.4. Lịch sử / Nguồn gốc . . . . . 262
10.5. Phân tích thuật toán . . . . . 263
10.5.1. Hash Function . . . . . 263
10.5.2. Search/Insert/Delete . . . . . 263
10.6. Minh họa từng bước . . . . . 264
10.6.1. Chaining Example . . . . . 264
10.6.2. Linear Probing Example . . . . . 265
10.7. Code minh họa C++ . . . . . 266
10.7.1. Hash Table with Chaining . . . . . 266
10.7.2. Hash Table with Linear Probing . . . . . 268
10.8. Lỗi phổ biến . . . . . 271
10.9. Khi nào dùng / không dùng . . . . . 272
10.9.1. Khi nào dùng Hash Table? . . . . . 272
10.9.2. Khi nào KHÔNG dùng Hash Table? . . . . . 272
10.10. Ứng dụng thực tế . . . . . 273
10.11. Tóm tắt . . . . . 275
11. Thuật toán Sắp xếp (Sorting Algorithms) . . . . . 277
11.1. Tổng quan chương . . . . . 277
11.1.1. Nội dung chính . . . . . 277
11.1.2. Kiến thức nền . . . . . 277

11.2. Giải thích từng khái niệm ..... 278
11.2.1. Sorting (Sắp xếp) ..... 278
11.2.2. Bubble Sort (Sắp xếp nổi bọt) ..... 280
11.2.3. Selection Sort ..... 280
11.2.4. Insertion Sort ..... 280
11.2.5. Merge Sort ..... 281
11.2.6. Quick Sort ..... 281
11.2.7. Heap Sort ..... 282
11.3. Bản chất trong máy tính ..... 283
11.3.1. Tại sao Quick Sort nhanh trong thực tế? ..... 283
11.3.2. Memory Access Patterns ..... 283
11.4. Lịch sử / Nguồn gốc ..... 284
11.5. Phân tích thuật toán ..... 285
11.5.1. Bảng tổng hợp ..... 285
11.6. Minh họa từng bước ..... 286
11.6.1. Quick Sort với Partition ..... 286
11.7. Code minh họa C++ ..... 287
11.7.1. Bubble Sort ..... 287
11.7.2. Selection Sort ..... 287
11.7.3. Insertion Sort ..... 287
11.7.4. Merge Sort ..... 288
11.7.5. Quick Sort ..... 289
11.7.6. Demo tất cả algorithms ..... 290
11.8. Lỗi phổ biến ..... 292
11.9. Khi nào dùng / không dùng ..... 294
11.9.1. Chọn thuật toán theo tình huống ..... 294
11.9.2. Hybrid Approaches (Thực tế) ..... 294
11.10. Ứng dụng thực tế ..... 295
11.11. Tóm tắt ..... 296
12. Đồ thị (Graph) ..... 297
12.1. Tổng quan chương ..... 297
12.1.1. Nội dung chính ..... 297
12.1.2. Kiến thức nền ..... 297
12.2. Giải thích từng khái niệm ..... 298
12.2.1. Graph (Đồ thị) ..... 298

12.2.2. Graph Representations . . . . . 300
12.2.3. Depth-First Search (DFS) . . . . . 302
12.2.4. Breadth-First Search (BFS) . . . . . 302
12.2.5. Shortest Path - Dijkstra's Algorithm . . . . . 304
12.3. Bản chất trong máy tính . . . . . 305
12.3.1. Memory Layout . . . . . 305
12.3.2. DFS vs BFS - Stack vs Queue . . . . . 305
12.4. Lịch sử / Nguồn gốc . . . . . 306
12.5. Phân tích thuật toán . . . . . 307
12.6. Minh họa từng bước . . . . . 308
12.6.1. Dijkstra's Algorithm . . . . . 308
12.7. Code minh họa C++ . . . . . 309
12.7.1. Graph Representation . . . . . 309
12.7.2. Weighted Graph với Dijkstra . . . . . 311
12.8. Lỗi phổ biến . . . . . 314
12.9. Khi nào dùng / không dùng . . . . . 316
12.9.1. Matrix vs List . . . . . 316
12.9.2. DFS vs BFS . . . . . 316
12.10. Ứng dụng thực tế . . . . . 317
12.11. Tóm tắt . . . . . 320

# 1. Lời nói đầu

Cấu trúc dữ liệu và Giải thuật (Data Structures and Algorithms - DSA) là nền tảng của khoa học máy tính. Đây không chỉ là môn học mà còn là kỹ năng cốt lõi mà mọi lập trình viên chuyên nghiệp cần nắm vững.

Cuốn sách này được viết với mục tiêu giúp bạn **hiểu bản chất**, không chỉ ghi nhớ máy móc. Mỗi chương được thiết kế để trả lời 3 câu hỏi cơ bản:

- **WHAT** (Nó là gì?) - Định nghĩa và khái niệm
- **WHY** (Tại sao cần nó?) - Lý do ra đời và giá trị thực tiễn
- **HOW** (Nó hoạt động như thế nào?) - Cơ chế hoạt động trong máy tính

Cuốn sách này dành cho sinh viên khoa học máy tính, với code minh họa bằng C++ - ngôn ngữ phổ biến trong giảng dạy DSA. Tuy nhiên, các khái niệm và nguyên lý có thể áp dụng cho bất kỳ ngôn ngữ lập trình nào.

Chúc bạn học tập hiệu quả!

# 2. Giới thiệu về Cấu trúc Dữ liệu và Giải thuật

## 2.1. Tổng quan chương

### 2.1.1. Nội dung chính của chương

Chương này là nền tảng của toàn bộ môn học Cấu trúc Dữ liệu và Giải thuật. Chúng ta sẽ tìm hiểu:

1. **Các khái niệm cơ bản:**
   - Data (Dữ liệu)
   - Data Type (Kiểu dữ liệu)
   - Data Structure (Cấu trúc dữ liệu)
   - Abstract Data Type (Kiểu dữ liệu trừu tượng)
   - Algorithm (Giải thuật)
   - Pseudocode (Mã giả)
2. **Ôn tập C++ cơ bản:**
   - Structures (Cấu trúc)
   - Classes (Lớp)
   - Pointers (Con trỏ)
   - Arrays (Mảng)
   - Pointers to structures and classes

### 2.1.2. Kiến thức nền cần biết trước

- Lập trình C++ cơ bản
- Hiểu biết về biến, hàm, vòng lặp, điều kiện
- Khái niệm về bộ nhớ máy tính (memory)

## 2.2. Giải thích từng khái niệm

### 2.2.1. Data (Dữ liệu)

**WHAT - Dữ liệu là gì?**

**Dữ liệu** (Data) là thông tin đã được chuyển đổi sang dạng thuận tiện hơn để tính toán và phân tích.

Dữ liệu có thể là:

- **Qualitative data** (Dữ liệu định tính): Thông tin mô tả (ví dụ: tên, màu sắc, địa chỉ)
- **Quantitative data** (Dữ liệu định lượng): Thông tin số học (ví dụ: tuổi, chiều cao, điểm số)
- **Discrete data** (Dữ liệu rời rạc): Chỉ nhận các giá trị nhất định (ví dụ: số nguyên)
- **Continuous data** (Dữ liệu liên tục): Có thể nhận bất kỳ giá trị nào trong một khoảng (ví dụ: số thực)

**WHY**

Trong lập trình, dữ liệu là nguyên liệu thô mà chương trình xử lý. Mọi chương trình đều làm việc với dữ liệu:

- Nhận dữ liệu đầu vào (input)
- Xử lý dữ liệu (processing)
- Trả về kết quả (output)

Ví dụ: Một ứng dụng quản lý sinh viên cần lưu trữ và xử lý dữ liệu như tên, MSSV, điểm số.

**HOW**

Trong máy tính, mọi dữ liệu đều được biểu diễn dưới dạng **bit** (0 hoặc 1):

- Số nguyên: Mã hóa nhị phân (binary encoding)
- Số thực: Định dạng IEEE 754 (floating-point)
- Ký tự: Mã ASCII hoặc Unicode
- Chuỗi: Dãy ký tự liên tiếp trong bộ nhớ

Ví dụ: Số nguyên 5 được lưu dưới dạng `00000101` (8 bit).

### 2.2.2. Data Type (Kiểu dữ liệu)

**WHAT - Kiểu dữ liệu là gì?**

**Kiểu dữ liệu** là một lớp các đối tượng dữ liệu có cùng tính chất.

Một kiểu dữ liệu bao gồm:

1. **Tập hợp các giá trị** (set of values)
2. **Tập hợp các phép toán** trên các giá trị đó (set of operations)

Ví dụ về các kiểu dữ liệu cơ bản trong C++:

| Kiểu | Tập giá trị | Phép toán |
|---|---|---|
| `int` | $-\infty, \ldots, -2, -1, 0, 1, 2, \ldots, \infty$ | `+`, `-`, `*`, `/`, `%`, `++`, `--` |
| `float` / `double` | $-\infty, \ldots, 0.0, \ldots, \infty$ | `+`, `-`, `*`, `/` |
| `char` | `'\\0'`, `'A'`, `'B'`, ..., `'a'`, `'b'`, ... | `<`, `>`, `==`, `!=` |
| `bool` | `true`, `false` | `&&`, `\|\|`, `!` |

**WHY**

Kiểu dữ liệu giúp:

1. **Xác định cách lưu trữ** dữ liệu trong bộ nhớ
2. **Đảm bảo tính an toàn** khi thao tác với dữ liệu
3. **Tối ưu hóa** việc sử dụng bộ nhớ
4. **Ngăn chặn lỗi** (type safety): Không thể cộng một số với một chuỗi

Ví dụ: Khi khai báo `int age = 25;`, compiler biết:

- Cấp phát 4 bytes cho biến `age`
- Chỉ cho phép các phép toán số học trên `age`
- Không cho phép gán chuỗi vào `age`

**HOW**

**Trong bộ nhớ:**

- `int` (4 bytes = 32 bits): Lưu giá trị nguyên dạng two's complement
- `float` (4 bytes): Lưu theo chuẩn IEEE 754 (1 bit dấu + 8 bits exponent + 23 bits mantissa)
- `char` (1 byte): Lưu mã ASCII
- `bool` (1 byte): `true` = 1, `false` = 0

**CPU xử lý:**

- Phép toán số nguyên: Sử dụng ALU (Arithmetic Logic Unit)
- Phép toán số thực: Sử dụng FPU (Floating Point Unit)

### 2.2.3. Data Structure (Cấu trúc dữ liệu)

**WHAT - Cấu trúc dữ liệu là gì?**

**Cấu trúc dữ liệu** là sự kết hợp của nhiều phần tử, trong đó mỗi phần tử có thể là:

1. Một kiểu dữ liệu cơ bản, hoặc
2. Một cấu trúc dữ liệu khác

Kèm theo đó là một tập hợp các **mối quan hệ** (relationships) hoặc **cấu trúc** (structure) giữ các phần tử lại với nhau.

Ví dụ đơn giản: **Array** (Mảng)

```
int fibonacci[8] = {1, 2, 3, 5, 8, 13, 21, 34};
```

- **Các phần tử:** 8 số nguyên
- **Cấu trúc:** Các phần tử được sắp xếp theo thứ tự, liên tiếp trong bộ nhớ
- **Mối quan hệ:** Phần tử thứ i có chỉ số i (index), liền kề với phần tử i-1 và i+1

**WHY**

Tại sao cần cấu trúc dữ liệu?

1. **Tổ chức dữ liệu hiệu quả:** Giúp lưu trữ và truy xuất dữ liệu nhanh chóng
2. **Giải quyết các bài toán thực tế:**
   - Danh sách sinh viên → Array hoặc Linked List
   - Lịch sử trình duyệt → Stack
   - Hàng đợi in ấn → Queue
   - Cây thư mục → Tree
   - Bản đồ đường đi → Graph
3. **Tối ưu hóa hiệu suất:** Cấu trúc dữ liệu phù hợp giúp thuật toán chạy nhanh hơn

**Ví dụ thực tế:**

- Tìm kiếm trong mảng chưa sắp xếp: O(n)
- Tìm kiếm trong mảng đã sắp xếp (binary search): O(log n)
- Tìm kiếm trong hash table: O(1)

**HOW**

**Trong bộ nhớ:**

Mảng `int arr[5]` được lưu như sau:

```
Memory Address    Value
0x1000           arr[0]
0x1004           arr[1]
```

```
0x1008          arr[2]
0x100C          arr[3]
0x1010          arr[4]
```

- Các phần tử nằm liên tiếp nhau
- Khoảng cách giữa 2 phần tử = kích thước kiểu dữ liệu (4 bytes cho `int`)
- Truy xuất `arr[i]` = `*(arr + i)` = địa chỉ đầu + i × sizeof(int)

**CPU xử lý:**

- Tính địa chỉ: `address = base_address + index * element_size`
- Truy xuất giá trị tại địa chỉ đó (1 phép toán, rất nhanh → O(1))

### 2.2.4. Abstract Data Type (Kiểu dữ liệu trừu tượng - ADT)

**WHAT - ADT là gì?**

**Abstract Data Type (ADT)** là một kiểu dữ liệu được định nghĩa bởi:

1. **Dữ liệu** (Data) - những gì nó lưu trữ
2. **Các phép toán** (Operations) - những gì có thể làm với nó
3. **Tính đóng gói** (Encapsulation) - ẩn chi tiết triển khai

ADT chỉ mô tả **WHAT** (làm gì), không mô tả **HOW** (làm thế nào).

**Khái niệm Abstraction (Trừu tượng hóa):**

- Người dùng biết ADT **có thể làm gì**
- **Cách thức hoạt động bên trong** được ẩn đi

**Ví dụ: List ADT**

**Interface (Giao diện):**

- **Data:** Một chuỗi các phần tử có cùng kiểu
- **Operations:**
  - `insert(element, position)` - Chèn phần tử
  - `remove(position)` - Xóa phần tử
  - `get(position)` - Lấy phần tử
  - `size()` - Số lượng phần tử

**Implementation (Triển khai):**

- Cách 1: Dùng **Array** (mảng tĩnh)
- Cách 2: Dùng **Linked List** (danh sách liên kết)

Người dùng chỉ cần biết **có thể insert, remove, get**, không cần biết bên trong dùng array hay linked list.

**WHY**

Tại sao cần ADT?

1. **Tách biệt giao diện và triển khai:**
   - Người dùng không cần biết chi tiết bên trong
   - Có thể thay đổi cách triển khai mà không ảnh hưởng code sử dụng
2. **Tái sử dụng code:**
   - Một ADT có thể có nhiều cách triển khai
   - Chọn cách triển khai phù hợp với bài toán cụ thể
3. **Dễ bảo trì:**
   - Thay đổi bên trong không làm hỏng code bên ngoài
   - Tối ưu hóa dễ dàng hơn

**Ví dụ thực tế:**

- `std::vector` trong C++ STL là một List ADT
- Bạn dùng `push_back()`, `pop_back()` mà không cần biết bên trong quản lý bộ nhớ thế nào

**HOW**

**Cách triển khai ADT trong C++:**

Sử dụng `class` với:

- `private`: Dữ liệu và chi tiết triển khai (ẩn đi)
- `public`: Giao diện (phép toán)

```
class Stack {
private:
    int data[100];  // Chi tiết triển khai (ẩn)
    int top;

public:
    void push(int x);  // Giao diện (public)
    int pop();
    bool isEmpty();
};
```

Người dùng chỉ thấy `push()`, `pop()`, `isEmpty()`, không thấy mảng `data[]` bên trong.

### 2.2.5. Algorithm (Giải thuật)

**WHAT - Giải thuật là gì?**

**Giải thuật** (Algorithm) là một chuỗi các bước logic để giải quyết một vấn đề.

Theo Niklaus Wirth (nhà khoa học máy tính nổi tiếng):

**Program = Data Structures + Algorithms**

Chương trình = Cấu trúc dữ liệu + Giải thuật

**Đặc điểm của một giải thuật tốt:**

1. **Input:** Có đầu vào rõ ràng
2. **Output:** Có đầu ra xác định
3. **Definiteness:** Mỗi bước được định nghĩa rõ ràng, không mơ hồ
4. **Finiteness:** Kết thúc sau một số hữu hạn bước
5. **Effectiveness:** Mỗi bước có thể thực hiện được

**WHY**

Tại sao cần giải thuật?

1. **Giải quyết vấn đề một cách có hệ thống**
2. **Tối ưu hóa hiệu suất:** Giải thuật tốt giúp chương trình chạy nhanh hơn
3. **Có thể tái sử dụng:** Giải thuật độc lập với ngôn ngữ lập trình

**Ví dụ:**

- Sắp xếp 1 triệu số:
  - Bubble Sort: 10 phút
  - Quick Sort: 1 giây
- Tìm kiếm trong 1 tỷ phần tử:
  - Linear Search: vài phút
  - Binary Search: < 1 giây

**HOW**

**Giải thuật được biểu diễn bằng:**

1. **Ngôn ngữ tự nhiên:** Mô tả bằng lời
2. **Flowchart:** Sơ đồ khối
3. **Pseudocode:** Mã giả (giữa ngôn ngữ tự nhiên và code)
4. **Programming language:** Code thực tế

**Cách CPU thực thi:**

- Fetch (Lấy lệnh từ bộ nhớ)
- Decode (Giải mã lệnh)
- Execute (Thực thi lệnh)

- Lặp lại cho lệnh tiếp theo

### 2.2.6. Pseudocode (Mã giả)

**WHAT - Pseudocode là gì?**

**Pseudocode** là cách biểu diễn giải thuật bằng ngôn ngữ gần với tiếng Anh, kết hợp với cấu trúc lập trình.

Pseudocode = English + Code

- Cú pháp thoải mái, dễ đọc
- Sử dụng cấu trúc điều khiển cơ bản (sequential, conditional, iterative)

**Cấu trúc một Pseudocode:**

1. **Algorithm Header:**
   - Tên giải thuật
   - Tham số và kiểu dữ liệu
   - Mục đích (Purpose)
   - Điều kiện tiên quyết (Precondition)
   - Điều kiện sau (Postcondition)
   - Giá trị trả về (Return)
2. **Algorithm Body:**
   - Các câu lệnh
   - Đánh số thập phân để thể hiện mức độ
   - Các biến quan trọng
   - Chú thích giải thích

**Ví dụ về Pseudocode:**

```
Algorithm average
Pre: nothing
Post: the average of the input numbers is printed

1  i = 0
2  sum = 0
3  while all numbers not read do
4      i = i + 1
5      read number
6      sum = sum + number
7  end
8  average = sum / i
9  print average
10 End average
```

**WHY**

Tại sao dùng Pseudocode?

1. **Độc lập với ngôn ngữ:** Không phụ thuộc vào C++, Java, Python…
2. **Dễ hiểu:** Gần với tư duy con người
3. **Tập trung vào logic:** Không lo về cú pháp chi tiết
4. **Dễ chuyển đổi:** Có thể code bằng bất kỳ ngôn ngữ nào
5. **Giao tiếp hiệu quả:** Team dễ thảo luận về giải thuật

## 2.3. Bản chất trong máy tính

### 2.3.1. Cách dữ liệu được lưu trong memory

**Layout bộ nhớ của một chương trình C++:**

```
+------------------+  <- Địa chỉ cao (High Address)
|      Stack       |  Biến local, tham số hàm (tự động quản lý)
|        ↓         |  Phát triển xuống dưới
+------------------+
|        ↑         |
|      Heap        |  Bộ nhớ động (new/delete)
|                  |  Phát triển lên trên
+------------------+
|   BSS Segment    |  Biến global chưa khởi tạo
+------------------+
|   Data Segment   |  Biến global đã khởi tạo
+------------------+
|   Text Segment   |  Code (instructions)
+------------------+  <- Địa chỉ thấp (Low Address)
```

**Ví dụ cụ thể:**

```cpp
int globalVar = 100;        // Data Segment

int main() {
    int localVar = 50;      // Stack
    int* ptr = new int(25); // ptr trên Stack, *ptr trên Heap

    return 0;
}
```

**Memory layout:**

```
Stack:
  [localVar = 50]  <- 0x7FFF1234
  [ptr = 0x1A2B]   <- 0x7FFF1230

Heap:
  [25]             <- 0x00001A2B (địa chỉ ptr trỏ tới)

Data Segment:
  [globalVar = 100] <- 0x00400000
```

### 2.3.2. CPU xử lý dữ liệu

**Các bước CPU thực thi một lệnh:**

1. **Fetch:** Lấy instruction từ RAM vào CPU
2. **Decode:** Giải mã instruction (ADD, MOV, JMP…)
3. **Execute:** Thực hiện phép toán
4. **Write-back:** Ghi kết quả vào register hoặc memory

**Ví dụ:** `int c = a + b;`

```
1. LOAD R1, [address_of_a]    // Đọc a từ RAM vào register R1
2. LOAD R2, [address_of_b]    // Đọc b từ RAM vào register R2
3. ADD R3, R1, R2             // R3 = R1 + R2
4. STORE [address_of_c], R3   // Ghi R3 vào địa chỉ của c
```

**Tốc độ truy xuất:**

- Register: < 1 ns (nhanh nhất)
- L1 Cache: 1 ns
- L2 Cache: 3-10 ns
- RAM: 100 ns
- SSD: 100,000 ns
- HDD: 10,000,000 ns

→ Cấu trúc dữ liệu tốt giúp tận dụng cache, giảm truy xuất RAM.

## 2.4. Lịch sử / Nguồn gốc

### 2.4.1. Nguồn gốc khái niệm Data Structure

**Thời kỳ đầu (1940s-1950s):**

- Máy tính đầu tiên: ENIAC (1945)
- Lập trình bằng machine code (0 và 1)
- Chưa có khái niệm cấu trúc dữ liệu

**Sự ra đời của Array (1950s):**

- Array là cấu trúc dữ liệu đầu tiên
- Sinh ra từ nhu cầu lưu trữ nhiều giá trị cùng kiểu

**Thời kỳ phát triển (1960s-1970s):**

- **Linked List** (1955): Tony Hoare
- **Tree** (1960s): Cây nhị phân, AVL tree
- **Hash Table** (1953): Hans Peter Luhn
- **Stack & Queue**: Xuất phát từ nhu cầu quản lý bộ nhớ và CPU scheduling

**Ngôn ngữ lập trình ảnh hưởng:**

- **FORTRAN** (1957): Đưa array vào ngôn ngữ lập trình
- **LISP** (1958): Linked list
- **Algol** (1960): Block structure
- **C** (1972): Pointers, structures
- **C++** (1983): Classes, templates, STL

### 2.4.2. Tại sao có ADT?

**Vấn đề trước khi có ADT:**

- Code phụ thuộc vào chi tiết triển khai
- Khó bảo trì và mở rộng
- Không thể tái sử dụng

**Giải pháp: Information Hiding (1972):**

- David Parnas đề xuất ý tưởng ẩn chi tiết triển khai
- Dẫn đến khái niệm ADT và OOP

**Ngày nay:**

- Hầu hết ngôn ngữ đều hỗ trợ ADT
- C++ STL (Standard Template Library) cung cấp các ADT sẵn: `vector`, `list`, `stack`, `queue`, `map`, `set`...

## 2.5. Phân tích thuật toán

Chương này chủ yếu về khái niệm, chưa có thuật toán cụ thể để phân tích. Chúng ta sẽ phân tích các thuật toán cụ thể trong các chương sau.

Tuy nhiên, có thể lấy một ví dụ đơn giản:

**Ví dụ: Thuật toán tính trung bình cộng**

```
Algorithm average
Pre: nothing
Post: the average of the input numbers is printed

1  i = 0
2  sum = 0
3  while all numbers not read do
4      i = i + 1
5      read number
6      sum = sum + number
7  end
8  average = sum / i
9  print average
10 End average
```

**Phân tích:**

- **Input:** n số
- **Output:** Trung bình cộng
- **Time Complexity:** O(n) - phải đọc qua n số
- **Space Complexity:** O(1) - chỉ dùng 3 biến (i, sum, average)

## 2.6. Minh họa từng bước

**Ví dụ: Tính trung bình của 5, 10, 15, 20**

```
Bước 1: i = 0, sum = 0
Bước 2: Đọc 5
        i = 1, sum = 0 + 5 = 5
Bước 3: Đọc 10
        i = 2, sum = 5 + 10 = 15
Bước 4: Đọc 15
        i = 3, sum = 15 + 15 = 30
Bước 5: Đọc 20
        i = 4, sum = 30 + 20 = 50
Bước 6: average = 50 / 4 = 12.5
Bước 7: In ra 12.5
```

## 2.7. Code minh họa bằng C++

### 2.7.1. Struct (Cấu trúc)

```
#include <iostream>
#include <string>
using namespace std;

// Định nghĩa struct
struct Car {
    string brand;
    int year;
};

int main() {
    // Khai báo biến kiểu Car
    Car myCar;
    myCar.brand = "Audi";
    myCar.year = 2011;

    // In thông tin
    cout << "My favorite car is: " << endl;
    cout << myCar.brand << " (" << myCar.year << ")" << endl;

    return 0;
}
```

**Output:**

```
My favorite car is:
Audi (2011)
```

### 2.7.2. Class với Constructor

```
#include <iostream>
using namespace std;

class Rectangle {
    int width, height;  // Private

public:
    // Constructor
    Rectangle(int w, int h) {
        width = w;
```

```
        height = h;
    }

    // Method
    int area() {
        return width * height;
    }
};

int main() {
    Rectangle rectA(3, 4);
    Rectangle rectB(5, 6);

    cout << "rectA area: " << rectA.area() << endl;
    cout << "rectB area: " << rectB.area() << endl;

    return 0;
}
```

**Output:**

```
rectA area: 12
rectB area: 30
```

### 2.7.3. Pointers (Con trỏ)

```
#include <iostream>
using namespace std;

int main() {
    int value = 93;
    int* p;

    p = &value;        // p lưu địa chỉ của value
    cout << "Address of value: " << p << endl;
    cout << "Value of value: " << *p << endl;

    *p = 100;          // Thay đổi value thông qua con trỏ
    cout << "New value: " << value << endl;

    return 0;
}
```

**Output:**

```
Address of value: 0x7ffeea8b9a3c
Value of value: 93
New value: 100
```

### 2.7.4. Arrays (Mảng)

```
#include <iostream>
using namespace std;

int main() {
    int fibonacci[8] = {1, 2, 3, 5, 8, 13, 21, 34};

    // In các phần tử
    for (int i = 0; i < 8; i++) {
        cout << "fibonacci[" << i << "] = " << fibonacci[i] << endl;
    }

    // Array và pointer
    int* p = fibonacci;
    cout << "\nUsing pointer:" << endl;
    cout << "First element: " << *p << endl;
    cout << "Second element: " << *(p + 1) << endl;

    return 0;
}
```

### 2.7.5. Pointer to Struct

```
#include <iostream>
#include <string>
using namespace std;

struct Student {
    string name;
    string firstname;
    int age;
};

void printStudent(Student* s) {
    cout << "My name is " << s->name << " " << s->firstname << endl;
    cout << "I am " << s->age << " years old." << endl;
}
```

```
int main() {
    Student sv;
    sv.name = "Nguyen";
    sv.firstname = "An";
    sv.age = 20;

    // Con trỏ đến struct
    Student* psv = &sv;

    // Truy xuất qua con trỏ
    printStudent(psv);

    return 0;
}
```

**Output:**

```
My name is Nguyen An
I am 20 years old.
```

### 2.7.6. Pointer to Class

```
#include <iostream>
using namespace std;

class Rectangle {
    int width, height;

public:
    Rectangle(int w, int h) : width(w), height(h) {}

    int area() {
        return width * height;
    }
};

int main() {
    Rectangle rectA(3, 4);
    Rectangle* rectB = &rectA;              // Con trỏ tới object có sẵn
    Rectangle* rectC = new Rectangle(5, 6); // Con trỏ tới object mới (heap)

    cout << "rectA area: " << rectA.area() << endl;
```

```
    cout << "rectB area: " << rectB->area() << endl;
    cout << "rectC area: " << rectC->area() << endl;

    delete rectC;  // Giải phóng bộ nhớ

    return 0;
}
```

**Output:**

```
rectA area: 12
rectB area: 12
rectC area: 30
```

## 2.8. Những lỗi phổ biến khi code

### 2.8.1. Lỗi với Pointers

**1. Dangling Pointer (Con trỏ lơ lửng):**

```
int* p = new int(10);
delete p;
cout << *p << endl;  // ❌ LỖI: p trỏ tới vùng nhớ đã được giải phóng
```

**Cách sửa:**

```
int* p = new int(10);
delete p;
p = nullptr;  // ✅ Gán nullptr sau khi delete
if (p != nullptr) {
    cout << *p << endl;
}
```

**2. Memory Leak (Rò rỉ bộ nhớ):**

```
void func() {
    int* p = new int(10);
    // Quên delete
}  // ❌ Bộ nhớ không được giải phóng
```

**Cách sửa:**

```
void func() {
    int* p = new int(10);
    // ... sử dụng p ...
    delete p;  // ✅ Nhớ giải phóng
}
```

**3. Null Pointer Dereference:**

```
int* p = nullptr;
cout << *p << endl;  // ❌ LỖI: Dereference null pointer
```

**Cách sửa:**

```
int* p = nullptr;
if (p != nullptr) {  // ✅ Kiểm tra trước khi dereference
    cout << *p << endl;
}
```

### 2.8.2. Lỗi với Arrays

**1. Array Out of Bounds:**

```
int arr[5] = {1, 2, 3, 4, 5};
cout << arr[5] << endl;  // ❌ LỖI: Chỉ có index 0-4
```

**Cách sửa:**

```
int arr[5] = {1, 2, 3, 4, 5};
for (int i = 0; i < 5; i++) {  // ✅ i < 5, không phải i <= 5
    cout << arr[i] << endl;
}
```

**2. Array Not Initialized:**

```
int arr[100];
cout << arr[0] << endl;  // ❌ Giá trị rác (garbage value)
```

**Cách sửa:**

```
int arr[100] = {0};  // ✅ Khởi tạo tất cả phần tử = 0
```

### 2.8.3. Lỗi với Struct/Class

**1. Quên khởi tạo:**

```
struct Point {
    int x, y;
};

Point p;
cout << p.x << endl;  // ❌ Giá trị rác
```

**Cách sửa:**

```
Point p = {0, 0};  // ✅ Khởi tạo
// Hoặc dùng constructor
```

## 2.9. Khi nào nên dùng / không nên dùng

### 2.9.1. Struct vs Class

| Tiêu chí | Struct | Class |
|---|---|---|
| **Mục đích** | Nhóm dữ liệu đơn giản | ADT phức tạp với behavior |
| **Mặc định** | Public | Private |
| **Khi nào dùng** | POD (Plain Old Data), không logic phức tạp | Có methods, invariants cần bảo vệ |
| **Ví dụ** | `Point {x, y}`, `Color {r, g, b}` | `Stack`, `Queue`, `BinaryTree` |

**Ví dụ:**

```
// ✅ Dùng struct cho dữ liệu đơn giản
struct Point {
    int x, y;
};

// ✅ Dùng class cho ADT
class Stack {
private:
    int data[100];
    int top;
public:
    void push(int x);
    int pop();
};
```

### 2.9.2. Array vs Pointer

| Tiêu chí | Array | Pointer |
|---|---|---|
| **Kích thước** | Cố định (compile-time) | Động (runtime) |
| **Bộ nhớ** | Stack (local) hoặc Data (global) | Heap (với `new`) |
| **An toàn** | Tự động giải phóng | Phải `delete` thủ công |
| **Hiệu suất** | Nhanh hơn (cache-friendly) | Chậm hơn một chút |
| **Khi nào dùng** | Kích thước biết trước | Kích thước động, cần resize |

**Ví dụ:**

```
// ✅ Dùng array khi biết kích thước
int scores[100];  // Lưu điểm của 100 sinh viên cố định

// ✅ Dùng pointer khi kích thước động
int n;
cin >> n;
int* scores = new int[n];  // Số sinh viên nhập từ user
// ...
delete[] scores;
```

## 2.10. Ứng dụng thực tế

### 2.10.1. Data Structures trong Hệ điều hành

**1. Process Management:**

- **Queue:** Ready queue, waiting queue cho CPU scheduling
- **Stack:** Call stack cho mỗi process/thread
- **Tree:** Process tree (parent-child relationships)

**2. Memory Management:**

- **Linked List:** Free memory blocks
- **Tree:** Page table (virtual memory)
- **Hash Table:** File descriptor table

**3. File System:**

- **Tree:** Directory structure (folder hierarchy)
- **Linked List:** File allocation (linked allocation)

### 2.10.2. Data Structures trong Database

**1. Indexing:**

- **B-Tree / B+ Tree:** Index structure (MySQL, PostgreSQL)
- **Hash Table:** Hash index
- **Trie:** Full-text search

**2. Query Processing:**

- **Stack:** Expression evaluation
- **Queue:** Buffer pool management

**3. Transaction Management:**

- **Graph:** Deadlock detection
- **Queue:** Transaction queue

### 2.10.3. Data Structures trong Compiler

**1. Lexical Analysis:**

- **Hash Table:** Symbol table (lưu biến, hàm)
- **Trie:** Keyword lookup

**2. Syntax Analysis:**

- **Stack:** Parsing stack
- **Tree:** Abstract Syntax Tree (AST)

**3. Code Optimization:**

- **Graph:** Control Flow Graph (CFG), Data Flow Graph

### 2.10.4. Data Structures trong Web

**1. Browser:**

- **Stack:** Back/Forward navigation history
- **Tree:** DOM (Document Object Model)
- **Hash Table:** Cache

**2. Web Server:**

- **Queue:** Request queue
- **Hash Table:** Session storage
- **Tree:** URL routing

**3. Search Engine:**

- **Trie:** Autocomplete
- **Graph:** PageRank
- **Inverted Index:** Document search

### 2.10.5. Data Structures trong Game

**1. Graphics:**

- **Quadtree / Octree:** Spatial partitioning
- **BSP Tree:** 3D rendering

**2. AI:**

- **Graph:** Pathfinding (A*)
- **Tree:** Decision tree, Behavior tree

**3. Game State:**

- **Stack:** Undo/Redo
- **Queue:** Event queue

## 2.11. Tóm tắt chương

**Những điểm quan trọng nhất cần nhớ:**

1. **Data** là nguyên liệu thô, **Data Type** xác định cách lưu và xử lý, **Data Structure** tổ chức dữ liệu hiệu quả.
2. **Abstract Data Type (ADT)** tách biệt "làm gì" (interface) khỏi "làm thế nào" (implementation).
3. **Algorithm** là các bước logic để giải quyết vấn đề. Chương trình = Data Structures + Algorithms.
4. **Pseudocode** giúp mô tả giải thuật độc lập với ngôn ngữ lập trình.
5. **Trong bộ nhớ:**
   - Stack: Biến local, tự động quản lý
   - Heap: Bộ nhớ động (`new` / `delete`), phải quản lý thủ công
   - Data/BSS: Biến global
6. **Struct** dùng cho dữ liệu đơn giản, **Class** dùng cho ADT phức tạp.
7. **Pointers** mạnh mẽ nhưng dễ lỗi: Nhớ kiểm tra `nullptr`, giải phóng bộ nhớ, tránh dangling pointer.
8. **Arrays** nhanh nhưng cố định kích thước; **Dynamic arrays** (pointer + `new`) linh hoạt hơn.
9. **Ứng dụng thực tế:** DSA xuất hiện ở mọi nơi - OS, database, compiler, web, game…
10. **Tư duy quan trọng:** Luôn hỏi WHAT, WHY, HOW khi học cấu trúc dữ liệu hoặc giải thuật mới.

# 3. Độ phức tạp giải thuật (Algorithm Complexity)

## 3.1. Tổng quan chương

### 3.1.1. Nội dung chính của chương

Chương này giúp bạn hiểu cách đánh giá hiệu suất của giải thuật - một kỹ năng cốt lõi trong lập trình. Nội dung bao gồm:

1. **Algorithm Efficiency (Hiệu quả giải thuật):**
   - Khái niệm computational complexity
   - Time complexity và Space complexity
2. **Big-O Notation:**
   - Ký hiệu toán học để biểu diễn độ phức tạp
   - Các quy tắc tính Big-O
3. **Các loại vòng lặp và độ phức tạp:**
   - Linear loops: O(n)
   - Logarithmic loops: O(log n)
   - Nested loops: $O(n^2)$, O(n log n)
4. **Best, Worst, Average case**
5. **Các complexity classes phổ biến:**
   - Constant, Logarithmic, Linear, Linearithmic, Quadratic, Polynomial, Exponential, Factorial
6. **P và NP Problems**

### 3.1.2. Kiến thức nền cần biết trước

- Toán học cơ bản: logarithm, luỹ thừa
- Vòng lặp trong lập trình (for, while)
- Hiểu cách đọc và phân tích code

## 3.2. Giải thích từng khái niệm

### 3.2.1. Algorithm Efficiency (Hiệu quả giải thuật)

**WHAT - Hiệu quả giải thuật là gì?**

**Computational Complexity** (Độ phức tạp tính toán) là thước đo mức độ khó khăn (về thời gian và/hoặc bộ nhớ) của một giải thuật.

Hai câu hỏi quan trọng:

1. **Time Complexity:** Giải thuật chạy nhanh thế nào?
2. **Space Complexity:** Giải thuật tốn bao nhiêu bộ nhớ?

Dạng tổng quát:

$$\text{efficiency} = f(n)$$

Trong đó n là **kích thước bài toán** (số lượng phần tử đầu vào).

Tại sao cần đánh giá hiệu quả giải thuật?

1. **So sánh các giải thuật:** Một bài toán có nhiều cách giải, cái nào tốt hơn?
2. **Dự đoán hiệu suất:** Liệu giải thuật có chạy được trên dữ liệu lớn không?
3. **Tối ưu hóa code:** Tìm điểm nghẽn (bottleneck) để cải thiện

**Ví dụ thực tế:**

- Tìm kiếm trong Google: Phải xử lý hàng tỷ trang web → Cần giải thuật hiệu quả
- Sắp xếp 1 triệu sản phẩm trên Shopee → Giải thuật $O(n^2)$ sẽ quá chậm
- GPS tìm đường đi ngắn nhất → Phải tính nhanh để người dùng không phải đợi

**Cách đo hiệu quả:**

1. **Đếm số phép toán cơ bản:**
   - So sánh (comparisons)
   - Phép tính (arithmetic operations)
   - Gán (assignments)
   - Truy xuất bộ nhớ (memory access)
2. **Tập trung vào phép toán tốn thời gian nhất:**
   - Data movement to/from memory/storage (chậm nhất)
   - CPU operations (nhanh hơn)
3. **Phân tích theo kích thước đầu vào n:**
   - n càng lớn, thời gian/bộ nhớ tăng như thế nào?

### 3.2.2. Big-O Notation

**WHAT - Big-O là gì?**

**Big-O Notation** là ký hiệu toán học để mô tả **tốc độ tăng trưởng** của hàm khi $n \to \infty$.

Big-O chỉ quan tâm đến **xu hướng tăng trưởng**, không quan tâm đến:

- Hằng số nhân (constant factor)
- Các số hạng nhỏ hơn

**Quy tắc rút gọn:**

1. Bỏ hệ số: `5n → n`
2. Giữ số hạng lớn nhất: `n² + n → n²`

**Ví dụ:**

| Hàm f(n) | Rút gọn | Big-O |
|---|---|---|
| `f(n) = 5n` | Bỏ hệ số 5 | `O(n)` |
| `f(n) = n²/2 + n/2` | Giữ $n^2$, bỏ n/2 | `O(n²)` |
| `f(n) = 3n² + 100n + 50` | Giữ $n^2$ | `O(n²)` |
| `f(n) = log₂n + 100` | Giữ log n | `O(log n)` |

Tại sao dùng Big-O?

1. **Đơn giản hóa:** Không cần tính toán chính xác số phép toán
2. **Tập trung vào điều quan trọng:** Khi n lớn, chỉ số hạng lớn nhất mới quan trọng
3. **Độc lập với hardware:** Không phụ thuộc vào tốc độ CPU cụ thể
4. **Dễ so sánh:** $O(n) < O(n \log n) < O(n^2)$

**Ví dụ minh họa:**

- Khi n = 1,000,000:
  - `n² = 1,000,000,000,000`
  - `100n = 100,000,000`
- Rõ ràng $n^2$ lớn hơn 100n rất nhiều, nên 100n không còn quan trọng

**Định nghĩa chính thức (toán học):**

$f(n) = O(g(n))$ nếu tồn tại hằng số $c > 0$ và $n_0$ sao cho:

$$f(n) \leq c \times g(n) \text{ với mọi } n \geq n_0$$

**Nói đơn giản:** f(n) không tăng nhanh hơn g(n) (với hệ số c nào đó) khi n đủ lớn.

### 3.2.3. Các loại Complexity Classes

Sau đây là các độ phức tạp phổ biến, từ tốt nhất đến tệ nhất:

| Tên | Big-O | n=10,000 | Thời gian ước tính |
|---|---|---|---|
| Constant | `O(1)` | 1 | 10 ns |
| Logarithmic | `O(log n)` | 14 | 140 ns |
| Linear | `O(n)` | 10,000 | 0.1 ms |
| Linearithmic | `O(n log n)` | 140,000 | 1.4 ms |
| Quadratic | $O(n^2)$ | 100,000,000 | 15-20 phút |
| Cubic | $O(n^3)$ | $10^{12}$ | vài giờ |
| Exponential | $O(2^n)$ | $2^{10000}$ | Không khả thi |
| Factorial | `O(n!)` | 10000! | Không khả thi |

**Giả sử:**

- 1 phép toán = 1 microsecond ($10^{-6}$ giây)
- Mỗi vòng lặp có 10 phép toán

**Biểu đồ so sánh:**

```
Thời gian
    ^
    |                                          n²
    |                                      ..
    |                                  ..
    |                              .....
    |                       .....    n log n
    |                .......
    |          .....      n
    |    .....
    | .............  log n
    |_________________> n (kích thước dữ liệu)
```

**Ghi nhớ:** Khi n = 1 triệu:

- O(log n): 20 bước
- O(n): 1 triệu bước
- O(n log n): 20 triệu bước
- O($n^2$): 1 nghìn tỷ bước (quá chậm!)

### 3.2.4. Phân tích các loại vòng lặp

#### 3.2.4.1. Linear Loops (Vòng lặp tuyến tính)

**Ví dụ 1:**

```
for (int i = 0; i < 1000; i++) {
    // application code
}
```

- Số lần lặp: 1000
- f(n) = n → **O(n)**

**Ví dụ 2:**

```
for (int i = 0; i < 1000; i += 2) {
    // application code
}
```

- Số lần lặp: 500
- f(n) = n/2 → **O(n)** (bỏ hằng số 1/2)

**Nhận xét:** Vòng lặp tăng/giảm theo **cộng** (i++, i+=2, i+=k) → O(n)

#### 3.2.4.2. Logarithmic Loops (Vòng lặp logarit)

**Ví dụ 1: Multiply loop (nhân đôi)**

```
int i = 1;
while (i <= n) {
    // application code
    i = i * 2;  // i nhân đôi mỗi lần
}
```

- Các giá trị i: 1, 2, 4, 8, 16, …, n
- Số bước: $\log_2(n)$
- **O(log n)**

**Ví dụ 2: Divide loop (chia đôi)**

```
int i = n;
while (i >= 1) {
    // application code
    i = i / 2;  // i giảm một nửa mỗi lần
}
```

- Các giá trị i: n, n/2, n/4, n/8, …, 1

- Số bước: $\log_2(n)$
- **O(log n)**

**Nhận xét:** Vòng lặp tăng/giảm theo **nhân/chia** (i*2, i/2) → O(log n)

**Tại sao log n nhanh?**

- n = 1,000,000 → $\log_2(n) \approx 20$ bước
- Giảm kích thước bài toán rất nhanh!

#### 3.2.4.3. Nested Loops (Vòng lặp lồng nhau)

**Quy tắc:**

$$\text{Iterations} = \text{Outer loop iterations} \times \text{Inner loop iterations}$$

**Ví dụ 1: Quadratic (Bậc 2)**

```
int i = 1;
while (i <= n) {
    int j = 1;
    while (j <= n) {
        // application code
        j = j + 1;  // O(n)
    }
    i = i + 1;  // O(n)
}
```

- Vòng ngoài: n lần
- Vòng trong: n lần (mỗi lần vòng ngoài)
- Tổng: $n \times n = n^2$
- **$O(n^2)$**

**Ví dụ 2: Linearithmic (n log n)**

```
int i = 1;
while (i <= n) {
    int j = 1;
    while (j <= n) {
        // application code
        j = j * 2;  // O(log n)
    }
    i = i + 1;  // O(n)
}
```

- Vòng ngoài: n lần
- Vòng trong: log n lần

- Tổng: n × log n
- **O(n log n)**

**Ví dụ 3: Dependent Quadratic (Vòng trong phụ thuộc vòng ngoài)**

```
int i = 1;
while (i <= n) {
    int j = 1;
    while (j <= i) {  // j <= i (không phải j <= n)
        // application code
        j = j + 1;
    }
    i = i + 1;
}
```

- Vòng ngoài: n lần
- Vòng trong: 1, 2, 3, …, n lần (tùy theo i)
- Tổng: 1 + 2 + 3 + … + n = n(n+1)/2
- **O(n²)** (bỏ hằng số 1/2 và n)

### 3.2.5. Best, Worst, Average Case

**WHAT - Ba trường hợp phân tích**

1. **Best Case:** Trường hợp tốt nhất - số bước ít nhất
2. **Worst Case:** Trường hợp xấu nhất - số bước nhiều nhất
3. **Average Case:** Trường hợp trung bình

Tại sao phân tích 3 trường hợp?

1. **Best case:** Hiểu giới hạn dưới của giải thuật
2. **Worst case:** Đảm bảo giải thuật không chậm quá mức cho phép
3. **Average case:** Ước lượng hiệu suất thực tế

Trong thực tế, ta thường quan tâm đến **Worst Case** vì nó đảm bảo giải thuật luôn chạy trong thời gian cho phép.

## 3.3. Bản chất trong máy tính

### 3.3.1. Tại sao Big-O quan trọng?

**Ví dụ thực tế: Tìm kiếm trong mảng**

Giả sử bạn có 1 tỷ số ($n = 10^9$) và cần tìm một số cụ thể:

**Cách 1: Linear Search (O(n))**

```
for (int i = 0; i < n; i++) {
    if (arr[i] == target) return i;
}
```

- Thời gian: $10^9$ phép so sánh
- Với CPU 1 GHz: 1 giây

**Cách 2: Binary Search (O(log n))**

```
// Mảng đã sắp xếp
int left = 0, right = n-1;
while (left <= right) {
    int mid = (left + right) / 2;
    if (arr[mid] == target) return mid;
    else if (arr[mid] < target) left = mid + 1;
    else right = mid - 1;
}
```

- Thời gian: $\log_2(10^9) \approx 30$ phép so sánh
- Với CPU 1 GHz: 0.00003 giây

**Kết luận:** Binary Search nhanh hơn 33 triệu lần!

### 3.3.2. Cache và Memory Access

**Tại sao array nhanh hơn linked list?**

```
Array (cache-friendly):
Memory: [1][2][3][4][5][6][7][8]  <- Liên tiếp
        ↑  Khi đọc [1], CPU tự động load [2][3][4] vào cache

Linked List (cache-unfriendly):
Memory: [1]→ [2]→ [3]→ [4]  <- Rải rác trong RAM
        ↑    ↑    ↑    ↑
      0x100 0x500 0x300 0x700
      Mỗi node phải đọc riêng → chậm hơn
```

**Hiệu suất thực tế:**

- Truy xuất array: 1-2 ns (cache hit)

- Truy xuất linked list: 100 ns (RAM access)

→ Dù cùng O(n) nhưng array thường nhanh hơn 50-100 lần trong thực tế!

## 3.4. Lịch sử / Nguồn gốc

### 3.4.1. Sự ra đời của Big-O Notation

**1894: Paul Bachmann**

- Nhà toán học Đức đầu tiên đề xuất ký hiệu O trong sách "Analytische Zahlentheorie"
- Mục đích: Mô tả tốc độ tăng trưởng của hàm số

**1914: G.H. Hardy & J.E. Littlewood**

- Phát triển thêm các ký hiệu Big-O, Big-Ω, Big-Θ
- Ứng dụng trong lý thuyết số

**1960s-1970s: Khoa học máy tính**

- Donald Knuth (Cha đẻ của TeX, tác giả "The Art of Computer Programming")
- Đưa Big-O vào phân tích giải thuật
- Phổ biến trong ngành khoa học máy tính

### 3.4.2. Tại sao cần phân tích độ phức tạp?

**Thời kỳ đầu (1950s):**

- Máy tính rất chậm và đắt
- Mỗi giây CPU rất quý giá
- Phải tối ưu hóa từng dòng code

**Ngày nay:**

- Máy tính nhanh hơn hàng triệu lần
- Nhưng dữ liệu cũng lớn hơn hàng tỷ lần!
- Google, Facebook xử lý hàng tỷ requests/ngày
- Giải thuật $O(n^2)$ vs $O(n \log n)$ có thể chênh lệch hàng giờ đồng hồ

**Ví dụ: Google Search**

- Dữ liệu: Hàng tỷ trang web
- Nếu dùng $O(n^2)$: Không bao giờ trả về kết quả
- Thực tế dùng: $O(\log n)$ với index tree → Kết quả trong < 0.5 giây

## 3.5. Phân tích thuật toán

### 3.5.1. Binary Search (Tìm kiếm nhị phân)

**Ý tưởng:**

- Mảng đã sắp xếp
- Mỗi bước chia đôi không gian tìm kiếm
- So sánh phần tử giữa với target

**Các bước:**

1. Tìm phần tử giữa `mid`
2. Nếu `arr[mid] == target` → Tìm thấy
3. Nếu `arr[mid] < target` → Tìm bên phải
4. Nếu `arr[mid] > target` → Tìm bên trái
5. Lặp lại

**Recurrence Equation:**

```
T(n) = 1 + T(n/2)
```

- 1: Phép so sánh ở bước hiện tại
- T(n/2): Bài toán con với kích thước n/2

**Giải:**

```
T(n) = 1 + T(n/2)
     = 1 + [1 + T(n/4)]
     = 1 + 1 + [1 + T(n/8)]
     = ...
     = log₂(n) + T(1)
     = O(log n)
```

**Complexity:**

- **Best case:** O(1) - Target ngay ở giữa
- **Worst case:** O(log n) - Target ở đầu/cuối hoặc không tồn tại
- **Average case:** O(log n)

**Complexity:**

- Time: O(log n)
- Space: O(1)

### 3.5.2. Sequential Search (Tìm kiếm tuần tự)

**Ý tưởng:**

- Duyệt từng phần tử từ đầu đến cuối
- So sánh với target

**Các bước:**

1. Bắt đầu từ phần tử đầu tiên
2. So sánh với target
3. Nếu bằng → Tìm thấy
4. Nếu không → Tiếp tục phần tử tiếp theo
5. Lặp lại cho đến hết mảng

**Complexity:**

- **Best case:** O(1) - Target ở đầu mảng
- **Worst case:** O(n) - Target ở cuối hoặc không có
- **Average case:** O(n)
  - Xác suất target ở vị trí i: $p_i = 1/n$
  - Số bước trung bình: $(1 + 2 + 3 + \ldots + n) / n = (n+1)/2 = O(n)$

**Complexity:**

- Time: Best: O(1), Worst: O(n), Avg: O(n)
- Space: O(1)

### 3.5.3. Quick Sort (Sắp xếp nhanh)

**Ý tưởng:**

- Chọn pivot (phần tử chốt)
- Partition: Chia mảng thành 2 phần (< pivot và > pivot)
- Đệ quy sắp xếp 2 phần

**Recurrence Equation:**

```
T(n) = O(n) + 2T(n/2)
```

- O(n): Thời gian partition
- 2T(n/2): Hai bài toán con, mỗi bài toán kích thước n/2

**Complexity:**

- **Best case:** O(n log n) - Pivot luôn chia đều
- **Worst case:** $O(n^2)$ - Pivot luôn là min/max (mảng đã sắp xếp)
- **Average case:** O(n log n)

**Complexity:**

- Time: Best: O(n log n), Worst: $O(n^2)$, Avg: O(n log n)

- Space: O(log n) - Stack cho đệ quy

## 3.6. Minh họa từng bước

### 3.6.1. Binary Search - Tìm số 21

Mảng đã sắp xếp: `[1, 2, 3, 5, 8, 13, 21, 34, 55, 89]`

Target: 21

```
Bước 1:
[1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
 ↑              ↑                  ↑
left          mid                right
mid = (0+9)/2 = 4, arr[4] = 8
8 < 21 → Tìm bên phải

Bước 2:
[1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
                ↑   ↑           ↑
              left mid        right
mid = (5+9)/2 = 7, arr[7] = 34
34 > 21 → Tìm bên trái

Bước 3:
[1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
                ↑   ↑   ↑
              left mid right
mid = (5+6)/2 = 5, arr[5] = 13
13 < 21 → Tìm bên phải

Bước 4:
[1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
                    ↑
                  left=mid=right
mid = 6, arr[6] = 21
Tìm thấy! → Index = 6
```

Tổng số bước: 4 bước = $\log_2(10) \approx 3.32$ → Làm tròn lên 4

### 3.6.2. Sequential Search - Tìm số 7

Mảng: `[8, 5, 21, 2, 1, 13, 4, 34, 7, 18]`

Target: 7

```
Bước 1: arr[0] = 8  ≠ 7 → Tiếp tục
Bước 2: arr[1] = 5  ≠ 7 → Tiếp tục
Bước 3: arr[2] = 21 ≠ 7 → Tiếp tục
Bước 4: arr[3] = 2  ≠ 7 → Tiếp tục
Bước 5: arr[4] = 1  ≠ 7 → Tiếp tục
Bước 6: arr[5] = 13 ≠ 7 → Tiếp tục
Bước 7: arr[6] = 4  ≠ 7 → Tiếp tục
Bước 8: arr[7] = 34 ≠ 7 → Tiếp tục
Bước 9: arr[8] = 7  = 7 → Tìm thấy! Index = 8
```

Tổng số bước: 9 bước (gần worst case)

## 3.7. Code minh họa bằng C++

### 3.7.1. Binary Search

```
#include <iostream>
using namespace std;

// Binary Search - Yêu cầu mảng đã sắp xếp
int binarySearch(int arr[], int n, int target) {
    int left = 0;
    int right = n - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;  // Tránh overflow

        if (arr[mid] == target) {
            return mid;  // Tìm thấy
        }
        else if (arr[mid] < target) {
            left = mid + 1;  // Tìm bên phải
        }
        else {
            right = mid - 1;  // Tìm bên trái
        }
    }

    return -1;  // Không tìm thấy
}

int main() {
    int arr[] = {1, 2, 3, 5, 8, 13, 21, 34, 55, 89};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 21;

    int index = binarySearch(arr, n, target);

    if (index != -1) {
        cout << "Found " << target << " at index " << index << endl;
    } else {
        cout << target << " not found" << endl;
    }
```

```
    return 0;
}
```

**Output:**

```
Found 21 at index 6
```

### 3.7.2. Sequential Search

```
#include <iostream>
using namespace std;

// Sequential Search - Không yêu cầu mảng sắp xếp
int sequentialSearch(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) {
            return i;  // Tìm thấy
        }
    }
    return -1;  // Không tìm thấy
}

int main() {
    int arr[] = {8, 5, 21, 2, 1, 13, 4, 34, 7, 18};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 7;

    int index = sequentialSearch(arr, n, target);

    if (index != -1) {
        cout << "Found " << target << " at index " << index << endl;
    } else {
        cout << target << " not found" << endl;
    }

    return 0;
}
```

**Output:**

```
Found 7 at index 8
```

### 3.7.3. Code phân tích các loại vòng lặp

```
#include <iostream>
#include <chrono>
using namespace std;
using namespace chrono;

// O(n) - Linear
void linearLoop(int n) {
    int count = 0;
    for (int i = 0; i < n; i++) {
        count++;
    }
    cout << "Linear: " << count << " iterations" << endl;
}

// O(log n) - Logarithmic
void logLoop(int n) {
    int count = 0;
    int i = 1;
    while (i <= n) {
        count++;
        i = i * 2;
    }
    cout << "Logarithmic: " << count << " iterations" << endl;
}

// O(n²) - Quadratic
void quadraticLoop(int n) {
    int count = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            count++;
        }
    }
    cout << "Quadratic: " << count << " iterations" << endl;
}

// O(n log n) - Linearithmic
void nLogNLoop(int n) {
    int count = 0;
    for (int i = 0; i < n; i++) {
        int j = 1;
```

```
        while (j <= n) {
            count++;
            j = j * 2;
        }
    }
    cout << "n log n: " << count << " iterations" << endl;
}

int main() {
    int n = 1000;

    cout << "n = " << n << endl << endl;

    linearLoop(n);       // 1,000 iterations
    logLoop(n);          // ~10 iterations
    nLogNLoop(n);        // ~10,000 iterations

    // Quadratic quá chậm, demo với n nhỏ hơn
    quadraticLoop(100);  // 10,000 iterations

    return 0;
}
```

**Output:**

```
n = 1000

Linear: 1000 iterations
Logarithmic: 10 iterations
n log n: 10000 iterations
Quadratic: 10000 iterations
```

### 3.7.4. Đo thời gian thực tế

```
#include <iostream>
#include <chrono>
#include <vector>
#include <algorithm>
using namespace std;
using namespace chrono;

int main() {
    int n = 100000;
    vector<int> arr(n);

    // Tạo mảng ngẫu nhiên
    for (int i = 0; i < n; i++) {
        arr[i] = rand();
    }

    // Đo thời gian Binary Search (yêu cầu mảng sắp xếp)
    sort(arr.begin(), arr.end());

    auto start = high_resolution_clock::now();
    binary_search(arr.begin(), arr.end(), arr[n/2]);
    auto end = high_resolution_clock::now();

    auto duration = duration_cast<microseconds>(end - start);
    cout << "Binary Search time: " << duration.count() << " microseconds" <<
endl;

    // Đo thời gian Linear Search
    start = high_resolution_clock::now();
    find(arr.begin(), arr.end(), arr[n/2]);
    end = high_resolution_clock::now();

    duration = duration_cast<microseconds>(end - start);
    cout << "Linear Search time: " << duration.count() << " microseconds" <<
endl;

    return 0;
}
```

## 3.8. Những lỗi phổ biến khi code

### 1. Tính mid sai (Integer overflow):

```
// ❌ SAI: Có thể bị overflow khi left + right > INT_MAX
int mid = (left + right) / 2;

// ✅ ĐÚNG: Tránh overflow
int mid = left + (right - left) / 2;
```

### 2. Điều kiện dừng sai trong Binary Search:

```
// ❌ SAI: while (left < right) → Có thể bỏ sót phần tử cuối
while (left < right) {
    // ...
}

// ✅ ĐÚNG: while (left <= right)
while (left <= right) {
    // ...
}
```

### 3. Không kiểm tra mảng rỗng:

```
// ❌ SAI: Không kiểm tra n <= 0
int binarySearch(int arr[], int n, int target) {
    int left = 0, right = n - 1;
    // ...
}

// ✅ ĐÚNG: Kiểm tra trước
int binarySearch(int arr[], int n, int target) {
    if (n <= 0) return -1;
    // ...
}
```

### 4. Dùng Binary Search trên mảng chưa sắp xếp:

```
int arr[] = {8, 5, 21, 2, 1};  // ❌ Chưa sắp xếp
int index = binarySearch(arr, 5, 2);  // ❌ Kết quả sai!

// ✅ ĐÚNG: Sắp xếp trước
```

```
sort(arr, arr + 5);
int index = binarySearch(arr, 5, 2);
```

### 5. Lãng phí thời gian phân tích sai:

```
// Lập trình viên nghĩ đây là O(n)
for (int i = 0; i < n; i++) {
    sort(arr, arr + n);  // ❌ sort() là O(n log n)!
}
// Thực tế: O(n² log n) - rất chậm!

// ✅ ĐÚNG: Sắp xếp 1 lần
sort(arr, arr + n);  // O(n log n)
for (int i = 0; i < n; i++) {
    // ...
}
```

## 3.9. Khi nào nên dùng / không nên dùng

### 3.9.1. Binary Search vs Sequential Search

| Tiêu chí | Binary Search | Sequential Search |
|---|---|---|
| **Yêu cầu** | Mảng đã sắp xếp | Không yêu cầu |
| **Time Complexity** | O(log n) | O(n) |
| **Ưu điểm** | Rất nhanh với dữ liệu lớn | Đơn giản, không cần sort |
| **Nhược điểm** | Phải sắp xếp trước (O(n log n)) | Chậm với dữ liệu lớn |
| **Khi nào dùng** | Mảng đã sắp xếp, tìm kiếm nhiều lần | Mảng nhỏ, tìm 1 lần |

**Ví dụ:**

```
// Tìm 1 lần trong mảng chưa sort → Dùng Sequential
int arr[] = {8, 5, 21, 2, 1};
sequentialSearch(arr, 5, 2);  // ✅ O(n)

// Tìm nhiều lần → Sort 1 lần rồi dùng Binary
sort(arr, arr + 5);  // O(n log n) - chỉ 1 lần
binarySearch(arr, 5, 2);  // ✅ O(log n) × k lần
binarySearch(arr, 5, 8);
binarySearch(arr, 5, 21);
```

### 3.9.2. So sánh Complexity Classes

| Complexity | Ưu | Nhược |
|---|---|---|
| O(1) | Nhanh nhất, không phụ thuộc n | Hiếm gặp |
| O(log n) | Rất nhanh, scale tốt | Cần cấu trúc đặc biệt (sorted, tree) |
| O(n) | Chấp nhận được, đơn giản | Chậm với n lớn (> $10^6$) |
| O(n log n) | Tốt nhất cho sorting | Hơi chậm với n rất lớn |
| $O(n^2)$ | OK với n nhỏ (< 1000) | Quá chậm với n > $10^4$ |
| $O(2^n)$, O(n!) | Giải được bài toán | Chỉ dùng với n rất nhỏ (< 20) |

## 3.10. Ứng dụng thực tế

### 3.10.1. Hệ điều hành (Operating System)

**1. Process Scheduling:**

- O(1) scheduler: Linux CFS (Completely Fair Scheduler)
- Priority queue: O(log n) insert/delete

**2. Memory Management:**

- Buddy system: O(log n) allocation
- Page replacement algorithms: O(1) LRU with hash table

**3. File System:**

- B-Tree index: O(log n) search
- Directory lookup: O(log n) with balanced tree

### 3.10.2. Database

**1. Indexing:**

- B+ Tree: O(log n) search, insert, delete
- Hash index: O(1) average case

**2. Query Optimization:**

- Join algorithms: O(n log n) - Merge Join
- Sorting: O(n log n) - External Sort

**3. Full-text Search:**

- Inverted index: O(k + log n) k = số kết quả

### 3.10.3. Web Applications

**1. Google Search:**

- PageRank: O(n log n) sorting
- Autocomplete: Trie - O(k) k = độ dài từ khóa

**2. Facebook Newsfeed:**

- Sorting posts: O(n log n)
- Friend suggestions: O(V + E) - Graph algorithms

**3. E-commerce (Shopee, Lazada):**

- Product search: O(log n) with indexing
- Price sorting: O(n log n)
- Recommendation: $O(n^2) \rightarrow$ Optimized to O(n log n)

### 3.10.4. Game Development

**1. Collision Detection:**

- Brute force: $O(n^2)$
- Spatial partitioning (Quadtree): O(n log n)

**2. Pathfinding:**

- Dijkstra: O((V + E) log V)
- A*: O(b^d) - Heuristic optimization

**3. Rendering:**

- Z-sorting: O(n log n)

### 3.10.5. Machine Learning / AI

**1. Training:**

- Linear Regression: $O(n^3)$ - Matrix inversion
- Neural Network: O(n × m × e) n=samples, m=features, e=epochs

**2. Prediction:**

- K-Nearest Neighbors: O(n) per query
- Decision Tree: O(log n) per query

**3. Data Processing:**

- Feature scaling: O(n)
- Sorting features: O(n log n)

## 3.11. Tóm tắt chương

**Những điểm quan trọng nhất cần nhớ:**

1. **Big-O Notation** biểu diễn tốc độ tăng trưởng của hàm khi n → ∞
   - Bỏ hằng số
   - Giữ số hạng lớn nhất
2. **Thứ tự tốc độ** (từ nhanh đến chậm):

   ```
   O(1) < O(log n) < O(n) < O(n log n) < O(n²) < O(2ⁿ) < O(n!)
   ```

3. **Các loại vòng lặp:**
   - Tăng/giảm theo **cộng** (i++, i+=k) → O(n)
   - Tăng/giảm theo **nhân/chia** (i*2, i/2) → O(log n)
   - Nested loops → Nhân complexity của từng vòng
4. **Ba trường hợp phân tích:**
   - Best case: Tốt nhất
   - Worst case: Xấu nhất (quan trọng nhất!)
   - Average case: Trung bình
5. **Binary Search:**
   - O(log n) - Rất nhanh
   - Yêu cầu mảng đã sắp xếp
   - Chia đôi không gian tìm kiếm mỗi bước
6. **Trong thực tế:**
   - n < 100: Mọi giải thuật đều OK
   - 100 < n < 10,000: Tránh $O(n^2)$
   - n > 10,000: Cần O(n log n) hoặc tốt hơn
   - n > 1,000,000: Cần O(n) hoặc O(log n)
7. **Trade-off:**
   - Time vs Space: Đôi khi tốn thêm bộ nhớ để tăng tốc độ (caching, memoization)
   - Simplicity vs Performance: Code đơn giản vs code tối ưu
8. **P vs NP:**
   - P: Giải được trong thời gian đa thức (polynomial)
   - NP: Kiểm tra lời giải trong thời gian đa thức
   - NP-complete: Bài toán khó nhất trong NP
9. **Thực tế quan trọng hơn lý thuyết:**
   - O(n) với array thường nhanh hơn O(n) với linked list (do cache)

- Hằng số ẩn trong Big-O cũng quan trọng

10. **Kỹ năng cần rèn luyện:**
    - Nhìn code → Nhận diện Big-O
    - Thiết kế giải thuật → Ước lượng complexity
    - Đọc tài liệu → Hiểu performance của thư viện/API

# 4. Đệ quy (Recursion)

## 4.1. Tổng quan chương

### 4.1.1. Nội dung chính của chương

Đệ quy là một trong những kỹ thuật lập trình mạnh mẽ và thanh lịch nhất. Chương này sẽ giúp bạn:

1. **Khái niệm Đệ quy:**
   - Direct recursion
   - Indirect recursion
2. **Thành phần cơ bản:**
   - Base case (Trường hợp dừng)
   - General case (Trường hợp đệ quy)
3. **Tính chất của Đệ quy**
4. **Thiết kế giải thuật đệ quy**
5. **Các ví dụ kinh điển:**
   - Factorial (Giai thừa)
   - GCD (Ước số chung lớn nhất)
   - Fibonacci
   - Towers of Hanoi (Tháp Hà Nội)
   - Print list in reverse
6. **Recursion và Backtracking:**
   - Eight Queens Problem
7. **So sánh Recursion vs Iteration**

### 4.1.2. Kiến thức nền cần biết trước

- Hàm (functions) trong C++
- Stack (khái niệm cơ bản)
- Hiểu cách hàm được gọi và return

## 4.2. Giải thích từng khái niệm

### 4.2.1. Recursion (Đệ quy)

**WHAT - Đệ quy là gì?**

**Đệ quy** (Recursion) là một quá trình lặp lại trong đó một giải thuật (hàm) **gọi chính nó**.

**Hai loại đệ quy:**

1. **Direct Recursion (Đệ quy trực tiếp):**
   - Hàm A gọi chính hàm A
   - A → A
2. **Indirect Recursion (Đệ quy gián tiếp):**
   - Hàm A gọi hàm B, hàm B gọi lại hàm A
   - A → B → A

**Ví dụ trực quan: Giai thừa**

Định nghĩa toán học:

```
n! = 1                          nếu n = 0
n! = n × (n-1) × (n-2) × ... × 1  nếu n > 0
```

Định nghĩa đệ quy:

```
factorial(n) = 1                      nếu n = 0  (base case)
factorial(n) = n × factorial(n-1)     nếu n > 0  (recursive case)
```

Tại sao cần đệ quy?

1. **Code ngắn gọn, dễ hiểu:**
   - Nhiều bài toán có bản chất đệ quy (tree traversal, graph search)
   - Code đệ quy thường ngắn hơn và dễ đọc hơn nhiều so với iteration
2. **Giải quyết bài toán phức tạp:**
   - Chia bài toán lớn thành các bài toán con nhỏ hơn
   - Mỗi bài toán con có cùng cấu trúc với bài toán gốc
3. **Tự nhiên với một số cấu trúc dữ liệu:**
   - Tree: Mỗi node có các subtree
   - Linked List: Head + phần còn lại (tail)
   - Graph: DFS, BFS

**Ví dụ thực tế:**

- Duyệt cây thư mục trong máy tính
- Tính toán biểu thức toán học

- Game: Minimax algorithm (cờ vua, cờ ca rô)
- Chia để trị: Merge Sort, Quick Sort

**Cách đệ quy hoạt động trong máy tính:**

Khi một hàm đệ quy được gọi:

1. **Push vào Stack:**
   - Tất cả biến local, tham số, địa chỉ return được lưu vào stack
2. **Thực thi hàm:**
   - Nếu đến base case → Return kết quả
   - Nếu chưa → Gọi đệ quy (push stack tiếp)
3. **Pop khỏi Stack:**
   - Khi hàm return, stack frame được giải phóng
   - Quay lại hàm gọi trước đó

**Ví dụ: factorial(3)**

```
Call Stack (từ trên xuống):

factorial(0) → return 1                  Stack: [0]
factorial(1) = 1 × factorial(0) = 1      Stack: [1, 0]
factorial(2) = 2 × factorial(1) = 2      Stack: [2, 1, 0]
factorial(3) = 3 × factorial(2) = 6      Stack: [3, 2, 1, 0]

Return process (từ dưới lên):
factorial(0) returns 1
factorial(1) returns 1 × 1 = 1
factorial(2) returns 2 × 1 = 2
factorial(3) returns 3 × 2 = 6
```

### 4.2.2. Thành phần cơ bản của Đệ quy

**WHAT - Hai thành phần bắt buộc**

Mọi giải thuật đệ quy đều phải có 2 thành phần:

1. **Base Case (Trường hợp dừng):**
   - Điều kiện để dừng đệ quy
   - Trường hợp đơn giản nhất, có thể giải trực tiếp
   - KHÔNG gọi đệ quy
2. **General Case (Trường hợp đệ quy):**
   - Chia bài toán thành bài toán con nhỏ hơn
   - Gọi đệ quy với tham số "gần base case hơn"

**Ví dụ: Factorial**

```
int factorial(int n) {
    // Base case: Dừng khi n = 0
    if (n == 0) {
        return 1;
    }

    // General case: n! = n × (n-1)!
    return n * factorial(n - 1);
}
```

**Cảnh báo quan trọng:**

1. **Thiếu Base Case:** Đệ quy vô tận → Stack Overflow!
2. **Base Case sai:** Vẫn đệ quy vô tận
3. **Không tiến về Base Case:** Ví dụ factorial(n+1) thay vì factorial(n-1)

**Quy tắc vàng khi thiết kế đệ quy:**

1. Xác định Base Case (Bài toán đơn giản nhất)
2. Xác định General Case (Làm sao chia nhỏ bài toán?)
3. Đảm bảo mỗi lần gọi đệ quy đều **tiến gần hơn** đến Base Case

### 4.2.3. Properties of Recursion (Tính chất của Đệ quy)

**Tính chất cơ bản:**

1. **Giải bài toán lớn bằng cách dùng lời giải của bài toán con:**
   - Mỗi lần gọi đệ quy giải quyết một phần nhỏ hơn của bài toán
2. **Cuối cùng bài toán con đủ đơn giản:**
   - Đến Base Case → Không cần đệ quy nữa
   - Bắt đầu return kết quả ngược lên
3. **Mỗi lần gọi đệ quy phải làm giảm kích thước bài toán:**
   - factorial(n) → factorial(n-1): n giảm
   - binarySearch(left, right) → binarySearch(left, mid-1): Giảm một nửa

## 4.3. Bản chất trong máy tính

### 4.3.1. Call Stack trong Đệ quy

**Memory Layout của đệ quy:**

```
Stack (phát triển xuống dưới):

┌──────────────────────┐ ← Top of Stack
│  factorial(0)        │
│  n = 0               │
│  return address      │
├──────────────────────┤
│  factorial(1)        │
│  n = 1               │
│  return address      │
├──────────────────────┤
│  factorial(2)        │
│  n = 2               │
│  return address      │
├──────────────────────┤
│  factorial(3)        │
│  n = 3               │
│  return address      │
├──────────────────────┤
│  main()              │
│  ...                 │
└──────────────────────┘ ← Bottom of Stack
```

**Mỗi Stack Frame chứa:**

- Tham số của hàm (n)
- Biến local
- Return address (địa chỉ để quay lại sau khi return)

**Vấn đề: Stack Overflow**

```
int badRecursion(int n) {
    return badRecursion(n);  // ❌ Không có base case!
}
```

→ Stack đầy → Crash!

### 4.3.2. Đệ quy vs Iteration - Hiệu suất

**Time Complexity:**

- Thường giống nhau

- Ví dụ: factorial đệ quy và iteration đều O(n)

**Space Complexity:**

- **Iteration:** O(1) - Chỉ dùng vài biến
- **Recursion:** O(n) - Stack frame cho mỗi lần gọi

**Speed:**

- **Iteration:** Nhanh hơn (không overhead của function call)
- **Recursion:** Chậm hơn một chút (push/pop stack)

**Nhưng:**

- Code đệ quy thường ngắn gọn, dễ hiểu hơn
- Một số bài toán viết đệ quy dễ hơn rất nhiều (tree, graph)

## 4.4. Lịch sử / Nguồn gốc

### 4.4.1. Lịch sử của Đệ quy

**1930s: Lambda Calculus**

- Alonzo Church phát triển λ-calculus
- Nền tảng toán học cho recursion

**1960: LISP (John McCarthy)**

- Ngôn ngữ lập trình đầu tiên hỗ trợ đệ quy mạnh mẽ
- Dùng cho AI research

**1970s-1980s: Functional Programming**

- Haskell, ML, Scheme
- Đệ quy là kỹ thuật lập trình chính

**Ngày nay:**

- Mọi ngôn ngữ lập trình đều hỗ trợ đệ quy
- Quan trọng trong thuật toán: Tree, Graph, Divide & Conquer

### 4.4.2. Towers of Hanoi - Câu chuyện

**Truyền thuyết:**

- Tại một ngôi đền ở Hà Nội, có 3 cây cột và 64 đĩa vàng
- Các nhà sư phải chuyển đĩa từ cột này sang cột khác
- Khi hoàn thành → Thế giới kết thúc

**Thực tế:**

- Bài toán do nhà toán học Pháp Édouard Lucas phát minh (1883)
- Minh họa hoàn hảo cho đệ quy
- Với 64 đĩa: $2^{64}$ - 1 bước ≈ 500 tỷ năm!

## 4.5. Phân tích thuật toán

### 4.5.1. Factorial (Giai thừa)

**Ý tưởng:**

- n! = n × (n-1)!
- Base case: 0! = 1

**Pseudocode:**

```
Algorithm recursiveFactorial(n)
Pre: n >= 0
Post: n! returned

if n = 0 then
    return 1
else
    return n * recursiveFactorial(n-1)
end
End recursiveFactorial
```

**Phân tích:**

- Mỗi lần gọi giảm n đi 1
- Số lần gọi: n+1 (factorial(n) → factorial(0))
- Mỗi lần: O(1) (một phép nhân)

**Complexity:**

- Time: O(n)
- Space: O(n) - Stack depth

### 4.5.2. Fibonacci Numbers

**Định nghĩa:**

```
fib(n) = 0                        nếu n = 0
fib(n) = 1                        nếu n = 1
fib(n) = fib(n-1) + fib(n-2)      nếu n > 1
```

**Cây đệ quy cho fib(4):**

```
                    fib(4)
                   /      \
              fib(3)      fib(2)
             /     \      /     \
        fib(2)  fib(1) fib(1) fib(0)
        /    \
    fib(1) fib(0)
```

**Vấn đề: Tính toán lặp lại!**

- fib(2) được tính 2 lần
- fib(1) được tính 3 lần
- Với n lớn → Cực kỳ chậm!

**Phân tích số lần gọi:**

| n | Số lần gọi | Thời gian |
|---|---|---|
| 10 | 177 | < 1 giây |
| 20 | 21,891 | < 1 giây |
| 30 | 2,692,573 | 7 giây |
| 40 | 331,160,281 | 13 phút |
| 50 | 12 tỷ | Vài giờ! |

**Complexity:**

- Time: $O(2^n)$ - Exponential!
- Space: O(n) - Stack depth

**Fibonacci đệ quy thuần túy rất chậm!**

Giải pháp:

- **Memoization:** Lưu kết quả đã tính → O(n)
- **Iteration:** Dùng vòng lặp → O(n), space O(1)
- **Matrix exponentiation:** O(log n)

### 4.5.3. Towers of Hanoi (Tháp Hà Nội)

**Bài toán:**

- 3 cột: Source (A), Auxiliary (B), Destination (C)
- n đĩa xếp từ nhỏ đến lớn trên cột A
- Mục tiêu: Chuyển tất cả sang cột C
- Quy tắc:
  - Một lần chỉ di chuyển 1 đĩa
  - Đĩa lớn không được đè lên đĩa nhỏ

**Ý tưởng đệ quy:**

Để di chuyển n đĩa từ A → C (dùng B):

1. Di chuyển (n-1) đĩa trên cùng từ A → B (dùng C)
2. Di chuyển đĩa lớn nhất từ A → C
3. Di chuyển (n-1) đĩa từ B → C (dùng A)

**Pseudocode:**

```
Algorithm move(disks, source, destination, auxiliary)
Pre: disks >= 1
Post: steps printed

if disks = 1 then
    print("Move from", source, "to", destination)
else
    move(disks-1, source, auxiliary, destination)
    move(1, source, destination, auxiliary)
    move(disks-1, auxiliary, destination, source)
end
End move
```

**Recurrence Equation:**

```
T(n) = 1 + 2T(n-1)

Giải:
T(n) = 1 + 2T(n-1)
     = 1 + 2[1 + 2T(n-2)]
     = 1 + 2 + 4T(n-2)
     = 1 + 2 + 4 + 8T(n-3)
     = ...
```

```
    = 1 + 2 + 4 + ... + 2^(n-1)
    = 2^n - 1
```

**Complexity:**

- Time: $O(2^n)$ - Exponential
- Space: O(n) - Stack depth

**Ý nghĩa:**

- Số bước tối thiểu = $2^n - 1$
- Với 64 đĩa: $2^{64} - 1 \approx 1.8 \times 10^{19}$ bước
- Nếu 1 bước/giây → Mất 500 tỷ năm!

### 4.5.4. GCD - Euclidean Algorithm (Ước số chung lớn nhất)

**Định nghĩa:**

```
gcd(a, b) = a                  nếu b = 0
gcd(a, b) = b                  nếu a = 0
gcd(a, b) = gcd(b, a mod b)    nếu a, b > 0
```

**Ví dụ: gcd(48, 18)**

```
gcd(48, 18)
= gcd(18, 48 mod 18)
= gcd(18, 12)
= gcd(12, 18 mod 12)
= gcd(12, 6)
= gcd(6, 12 mod 6)
= gcd(6, 0)
= 6
```

**Pseudocode:**

```
Algorithm gcd(a, b)
Pre: a, b >= 0
Post: GCD of a and b returned

if b = 0 then
    return a
end
if a = 0 then
    return b
end
return gcd(b, a mod b)
End gcd
```

**Complexity:**

- Time: O(log n) - n là số nhỏ hơn trong (a, b)
- Space: O(log n) - Stack depth

## 4.6. Minh họa từng bước

### 4.6.1. Factorial(3) - Chi tiết

```
Call: factorial(3)

Step 1: factorial(3) được gọi
        n = 3, không phải base case
        return 3 * factorial(2)
        → Gọi factorial(2)

Step 2: factorial(2) được gọi
        n = 2, không phải base case
        return 2 * factorial(1)
        → Gọi factorial(1)

Step 3: factorial(1) được gọi
        n = 1, không phải base case
        return 1 * factorial(0)
        → Gọi factorial(0)

Step 4: factorial(0) được gọi
        n = 0, BASE CASE!
        return 1

Return process (unwinding):

Step 5: factorial(0) returns 1
        factorial(1) = 1 * 1 = 1

Step 6: factorial(1) returns 1
        factorial(2) = 2 * 1 = 2

Step 7: factorial(2) returns 2
        factorial(3) = 3 * 2 = 6

Final result: 6
```

**Visualization:**

```
factorial(3)              Stack:
    |                     [3]
    ├-> 3 *               [3, 2]
    |       factorial(2)  [3, 2, 1]
```

```
|           |         [3, 2, 1, 0]
|           ├─> 2 *
|           |       factorial(1)
|           |           |
|           |           ├─> 1 *
|           |           |       factorial(0)
|           |           |           |
|           |           |           └─> return 1
|           |           └─> return 1
|           └─> return 2
└─> return 6
```

### 4.6.2. Towers of Hanoi với 3 đĩa

```
Initial:        Goal:
  |               |
 [1]              |
 [2]              |
 [3]             [3]
_____           _____
  A    B    C     A    B    C

move(3, A, C, B):

1. move(2, A, B, C) - Chuyển 2 đĩa trên từ A → B
   a. move(1, A, C, B) → Move A → C
      A: [2][3]  B: []      C: [1]

   b. move(1, A, B, C) → Move A → B
      A: [3]     B: [2]     C: [1]

   c. move(1, C, B, A) → Move C → B
      A: [3]     B: [1][2]  C: []

2. move(1, A, C, B) - Chuyển đĩa lớn nhất A → C
   A: []      B: [1][2]  C: [3]

3. move(2, B, C, A) - Chuyển 2 đĩa từ B → C
   a. move(1, B, A, C) → Move B → A
      A: [1]     B: [2]     C: [3]

   b. move(1, B, C, A) → Move B → C
      A: [1]     B: []      C: [2][3]

   c. move(1, A, C, B) → Move A → C
      A: []      B: []      C: [1][2][3]

Done! Tổng: 2³ - 1 = 7 bước
```

## 4.7. Code minh họa bằng C++

### 4.7.1. Factorial - Đệ quy vs Iteration

**Recursive version:**

```
#include <iostream>
using namespace std;

long recursiveFactorial(int n) {
    // Base case
    if (n == 0) {
        return 1;
    }
    // General case
    return n * recursiveFactorial(n - 1);
}

int main() {
    int n = 5;
    cout << "Factorial of " << n << " = "
         << recursiveFactorial(n) << endl;
    return 0;
}
```

**Iterative version:**

```
long iterativeFactorial(int n) {
    long result = 1;
    for (int i = 1; i <= n; i++) {
        result *= i;
    }
    return result;
}
```

**Output:**

```
Factorial of 5 = 120
```

### 4.7.2. Fibonacci - Đệ quy (Naive)

```
#include <iostream>
using namespace std;

long fib(int n) {
    // Base cases
    if (n == 0 || n == 1) {
        return n;
    }
    // General case
    return fib(n - 1) + fib(n - 2);
}

int main() {
    int n = 10;
    cout << "Fibonacci(" << n << ") = " << fib(n) << endl;

    // Đo thời gian cho các giá trị khác nhau
    for (int i = 1; i <= 40; i++) {
        cout << "fib(" << i << ") = " << fib(i) << endl;
    }

    return 0;
}
```

**Vấn đề:** Với n > 40, chương trình chạy rất chậm!

### 4.7.3. Fibonacci - Tối ưu với Memoization

```
#include <iostream>
#include <vector>
using namespace std;

vector<long> memo;

long fibMemo(int n) {
    // Đã tính rồi → Trả về luôn
    if (memo[n] != -1) {
        return memo[n];
    }

    // Base cases
    if (n == 0 || n == 1) {
        memo[n] = n;
        return n;
    }

    // Tính và lưu kết quả
    memo[n] = fibMemo(n - 1) + fibMemo(n - 2);
    return memo[n];
}

int main() {
    int n = 50;
    memo.resize(n + 1, -1);  // Khởi tạo memo với -1

    cout << "Fibonacci(" << n << ") = " << fibMemo(n) << endl;

    return 0;
}
```

**Kết quả:** Fibonacci(50) chạy gần như tức thì! (thay vì hàng giờ)

### 4.7.4. Towers of Hanoi

```
#include <iostream>
using namespace std;

void move(int n, char source, char destination, char auxiliary) {
    static int step = 0;  // Đếm số bước

    if (n == 1) {
        // Base case: Chỉ có 1 đĩa → Di chuyển trực tiếp
        cout << "Step " << ++step << ": Move from "
             << source << " to " << destination << endl;
    } else {
        // General case
        // 1. Chuyển (n-1) đĩa từ source → auxiliary (dùng destination)
        move(n - 1, source, auxiliary, destination);

        // 2. Chuyển đĩa lớn nhất từ source → destination
        move(1, source, destination, auxiliary);

        // 3. Chuyển (n-1) đĩa từ auxiliary → destination (dùng source)
        move(n - 1, auxiliary, destination, source);
    }
}

int main() {
    int numDisks;
    cout << "Enter number of disks: ";
    cin >> numDisks;

    cout << "\nSolving Towers of Hanoi with " << numDisks << " disks:\n";
    move(numDisks, 'A', 'C', 'B');

    return 0;
}
```

**Output (với 3 đĩa):**

```
Enter number of disks: 3

Solving Towers of Hanoi with 3 disks:
Step 1: Move from A to C
Step 2: Move from A to B
```

```
Step 3: Move from C to B
Step 4: Move from A to C
Step 5: Move from B to A
Step 6: Move from B to C
Step 7: Move from A to C
```

### 4.7.5. GCD (Euclidean Algorithm)

```
#include <iostream>
using namespace std;

int gcd(int a, int b) {
    // Base cases
    if (b == 0) return a;
    if (a == 0) return b;

    // General case
    return gcd(b, a % b);
}

int main() {
    int a = 48, b = 18;
    cout << "GCD of " << a << " and " << b << " = "
         << gcd(a, b) << endl;

    // Test với các giá trị khác
    cout << "GCD(100, 35) = " << gcd(100, 35) << endl;
    cout << "GCD(17, 19) = " << gcd(17, 19) << endl;  // Số nguyên tố cùng nhau

    return 0;
}
```

**Output:**

```
GCD of 48 and 18 = 6
GCD(100, 35) = 5
GCD(17, 19) = 1
```

### 4.7.6. Print Linked List in Reverse (Đệ quy)

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

void printReverse(Node* head) {
    // Base case: Danh sách rỗng
    if (head == NULL) {
        return;
    }

    // General case:
    // 1. In phần còn lại (đệ quy)
    printReverse(head->next);

    // 2. In node hiện tại (sau khi đệ quy return)
    cout << head->data << " ";
}

void printForward(Node* head) {
    if (head == NULL) return;
    cout << head->data << " ";
    printForward(head->next);
}

int main() {
    // Tạo linked list: 1 → 2 → 3 → 4
    Node* head = new Node{1, nullptr};
    head->next = new Node{2, nullptr};
    head->next->next = new Node{3, nullptr};
    head->next->next->next = new Node{4, nullptr};

    cout << "Forward: ";
    printForward(head);
    cout << endl;

    cout << "Reverse: ";
    printReverse(head);
```

```
    cout << endl;

    return 0;
}
```

**Output:**

```
Forward: 1 2 3 4
Reverse: 4 3 2 1
```

## 4.8. Những lỗi phổ biến khi code

**1. Thiếu Base Case → Stack Overflow:**

```
// ❌ SAI: Không có base case
int factorial(int n) {
    return n * factorial(n - 1);  // Lỗi: Đệ quy vô tận!
}

// ✅ ĐÚNG: Có base case
int factorial(int n) {
    if (n == 0) return 1;  // Base case
    return n * factorial(n - 1);
}
```

**2. Base Case sai:**

```
// ❌ SAI: Base case n == 1, nhưng factorial(0) sẽ lỗi
int factorial(int n) {
    if (n == 1) return 1;  // Thiếu n == 0
    return n * factorial(n - 1);
}

// ✅ ĐÚNG
int factorial(int n) {
    if (n == 0 || n == 1) return 1;
    return n * factorial(n - 1);
}
```

**3. Không tiến về Base Case:**

```
// ❌ SAI: factorial(n) gọi factorial(n+1) → Không bao giờ đến base case
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n + 1);  // Sai: n+1 thay vì n-1!
}

// ✅ ĐÚNG: Giảm n mỗi lần
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);  // n-1 tiến về 0
}
```

**4. Quên kiểm tra input âm:**

```
// ❌ SAI: Không kiểm tra n < 0
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);  // n < 0 sẽ lỗi!
}

// ✅ ĐÚNG: Kiểm tra input
int factorial(int n) {
    if (n < 0) return -1;  // Lỗi: factorial không định nghĩa cho số âm
    if (n == 0) return 1;
    return n * factorial(n - 1);
}
```

**5. Stack Overflow với input lớn:**

```
// ❌ VẤN ĐỀ: Với n rất lớn (vd: n = 100000), stack overflow
int sum(int n) {
    if (n == 0) return 0;
    return n + sum(n - 1);
}

// ✅ GIẢI PHÁP: Dùng iteration cho bài toán đơn giản
int sum(int n) {
    int result = 0;
    for (int i = 1; i <= n; i++) {
        result += i;
    }
    return result;  // Hoặc dùng công thức: n * (n + 1) / 2
}
```

**6. Fibonacci không tối ưu:**

```
// ❌ RẤT CHẬM: O(2^n)
long fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);  // Tính lại nhiều lần!
}

// ✅ TỐI ƯU: Dùng memoization hoặc iteration
```

## 4.9. Khi nào nên dùng / không nên dùng

### 4.9.1. Khi nào dùng Đệ quy?

| Tình huống | Lý do |
|---|---|
| Cấu trúc dữ liệu đệ quy | Tree, Graph, Linked List có bản chất đệ quy |
| Divide & Conquer | Merge Sort, Quick Sort, Binary Search |
| Backtracking | N-Queens, Sudoku, Maze solving |
| Mathematical problems | Factorial, Fibonacci, GCD, Towers of Hanoi |
| Code ngắn gọn hơn nhiều | 10 dòng đệ quy vs 50 dòng iteration |

**Ví dụ tốt cho đệ quy:**

```
// Tree traversal - Rất tự nhiên với đệ quy
void inorder(Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

// Nếu dùng iteration → Phức tạp hơn rất nhiều (cần stack tự quản lý)
```

### 4.9.2. Khi nào KHÔNG nên dùng Đệ quy?

| Tình huống | Lý do |
|---|---|
| Bài toán đơn giản | Iteration nhanh hơn, tiết kiệm bộ nhớ |
| n rất lớn | Stack overflow risk (n > 10000) |
| Performance critical | Overhead của function call |
| Tính toán lặp lại nhiều | Fibonacci thuần túy → $O(2^n)$ quá chậm |

**Ví dụ nên dùng Iteration:**

```
// Tính tổng 1 → n: Đơn giản → Iteration tốt hơn
int sum(int n) {
    int result = 0;
    for (int i = 1; i <= n; i++) {
        result += i;
    }
    return result;
}
```

```
// Đệ quy: Dài hơn, chậm hơn, tốn bộ nhớ hơn
int sumRecursive(int n) {
    if (n == 0) return 0;
    return n + sumRecursive(n - 1);
}
```

### 4.9.3. So sánh Recursion vs Iteration

| Tiêu chí | Recursion | Iteration |
|---|---|---|
| **Code** | Ngắn gọn, dễ hiểu | Dài hơn, phức tạp hơn |
| **Speed** | Chậm hơn (function call overhead) | Nhanh hơn |
| **Memory** | O(n) stack space | O(1) - Chỉ vài biến |
| **Risk** | Stack overflow với n lớn | An toàn hơn |
| **Sử dụng** | Tree, Graph, Divide & Conquer | Simple loops, large n |

## 4.10. Ứng dụng thực tế

**1. File System (Hệ thống file):**

Duyệt thư mục và tất cả thư mục con:

```
void listFiles(string path) {
    // Base case: path is a file
    if (isFile(path)) {
        cout << path << endl;
        return;
    }

    // General case: path is a directory
    for (auto& entry : listDirectory(path)) {
        listFiles(path + "/" + entry);  // Đệ quy
    }
}
```

**Trong Windows Explorer / Mac Finder:**

- Tìm kiếm file đệ quy qua tất cả thư mục con
- Tính tổng dung lượng thư mục

**2. Compiler (Trình biên dịch):**

**Parsing expressions (Phân tích biểu thức):**

```
Expression: 3 + 4 * 2

Parse Tree (đệ quy):
       +
      / \
     3   *
        / \
       4   2
```

**Đánh giá biểu thức:**

```
int evaluate(Node* root) {
    if (root->isNumber()) {
        return root->value;
    }
    int left = evaluate(root->left);
    int right = evaluate(root->right);
```

```
    return apply(root->op, left, right);
}
```

**3. Game Development:**

**Minimax Algorithm (Game AI):**

Cờ vua, cờ ca rô: Tìm nước đi tốt nhất

```
int minimax(Board board, int depth, bool isMax) {
    if (depth == 0 || gameOver(board)) {
        return evaluate(board);
    }

    if (isMax) {
        int maxEval = -INFINITY;
        for (auto& move : possibleMoves(board)) {
            int eval = minimax(makeMove(board, move), depth-1, false);
            maxEval = max(maxEval, eval);
        }
        return maxEval;
    } else {
        // Similar for MIN player
    }
}
```

**4. Web Crawling:**

Google Bot, Facebook Crawler:

```
void crawl(string url, int depth) {
    if (depth == 0 || visited(url)) return;

    markVisited(url);
    processPage(url);

    for (auto& link : getLinks(url)) {
        crawl(link, depth - 1);  // Đệ quy crawl các link
    }
}
```

**5. JSON / XML Parsing:**

```
{
    "name": "John",
    "address": {
        "city": "NYC",
        "zip": {
            "code": "10001"
        }
    }
}
```

Đệ quy để parse nested objects:

```
void parseJSON(JSONObject obj) {
    for (auto& [key, value] : obj) {
        if (value.isObject()) {
            parseJSON(value);  // Đệ quy
        } else {
            cout << key << ": " << value << endl;
        }
    }
}
```

## 4.11. Tóm tắt chương

**Những điểm quan trọng nhất cần nhớ:**

1. **Đệ quy** là hàm gọi chính nó để giải bài toán nhỏ hơn.
2. **Hai thành phần bắt buộc:**
   - **Base Case:** Điều kiện dừng (không đệ quy)
   - **General Case:** Gọi đệ quy với bài toán nhỏ hơn
3. **Quy tắc vàng:**
   - Mỗi lần đệ quy phải tiến gần hơn đến Base Case
   - Nếu không → Stack Overflow!
4. **Recursion vs Iteration:**
   - Recursion: Ngắn gọn, dễ hiểu, tốn bộ nhớ
   - Iteration: Nhanh hơn, tiết kiệm bộ nhớ
5. **Các ví dụ kinh điển:**
   - Factorial: O(n) time, O(n) space
   - Fibonacci (naive): $O(2^n)$ - RẤT CHẬM!
   - Fibonacci (memoization): O(n) - Nhanh!
   - Towers of Hanoi: $O(2^n)$ - Exponential
   - GCD (Euclidean): O(log n) - Rất nhanh
   - Binary Search: O(log n)
6. **Backtracking:**
   - Thử các khả năng, quay lui nếu sai
   - N-Queens, Sudoku, Maze
7. **Khi nào dùng đệ quy:**
   - Tree, Graph traversal
   - Divide & Conquer (Merge Sort, Quick Sort)
   - Backtracking problems
   - Code ngắn gọn hơn nhiều so với iteration
8. **Khi nào KHÔNG dùng:**
   - Bài toán đơn giản (sum, factorial…)
   - n rất lớn (risk stack overflow)
   - Performance critical
   - Tính toán lặp lại (Fibonacci naive)
9. **Tối ưu đệ quy:**
   - **Memoization:** Lưu kết quả đã tính

- **Tail recursion:** Một số compiler tối ưu thành iteration
- Chuyển sang iteration nếu cần

10. **Call Stack:**
    - Mỗi lần gọi hàm → Push vào stack
    - Return → Pop khỏi stack
    - Stack có giới hạn → Quá nhiều đệ quy → Stack Overflow

# 5. Danh sách liên kết (Linked Lists)

## 5.1. Tổng quan chương

### 5.1.1. Nội dung chính của chương

Danh sách (List) là một trong những cấu trúc dữ liệu cơ bản và quan trọng nhất. Chương này sẽ giúp bạn hiểu sâu về Lists qua các nội dung:

1. **Linear List Concepts:**
   - Định nghĩa List
   - General List vs Restricted List
   - List ADT (Abstract Data Type)
2. **Array Implementation:**
   - Static Array
   - Dynamic Array
   - Các phép toán cơ bản
3. **Singly Linked List:**
   - Node structure
   - Các phép toán: Insert, Delete, Search
   - Traversal
4. **Other Linked Lists:**
   - Doubly Linked List
   - Circular Linked List
5. **So sánh các cách triển khai:**
   - Array vs Linked List
   - Ưu/nhược điểm
   - Khi nào dùng cái gì

### 5.1.2. Kiến thức nền cần biết trước

- Con trỏ (Pointers) trong C++
- Cấp phát bộ nhớ động ( `new` , `delete` )
- Struct/Class trong C++

## 5.2. Giải thích từng khái niệm

### 5.2.1. Linear List (Danh sách tuyến tính)

**WHAT - List là gì?**

**Linear List** (Danh sách tuyến tính) là một chuỗi hữu hạn, có thứ tự các phần tử dữ liệu.

**"Có thứ tự"** nghĩa là:

- Mỗi phần tử có một vị trí (position) trong danh sách
- Có phần tử đầu (head) và phần tử cuối (tail)
- Mỗi phần tử (trừ đầu/cuối) có phần tử đứng trước và đứng sau

**Ví dụ:**

```
List of students: [An, Binh, Cuong, Dung, Em]
                   ↑                         ↑
                 head                      tail
```

**Các loại List:**

1. **General List (Danh sách tổng quát):**
   - Không hạn chế phép toán nào
   - Có thể insert/delete ở bất kỳ vị trí nào
   - **Unsorted List:** Phần tử không sắp xếp theo thứ tự cụ thể
   - **Sorted List:** Phần tử được sắp xếp theo key
2. **Restricted List (Danh sách hạn chế):**
   - Chỉ cho phép một số phép toán
   - Insert/delete chỉ ở đầu hoặc cuối
   - **Stack:** LIFO (Last-In-First-Out)
   - **Queue:** FIFO (First-In-First-Out)

**Tại sao cần List?**

1. **Lưu trữ tập hợp phần tử:**
   - Danh sách sinh viên
   - Lịch sử duyệt web
   - Playlist nhạc
2. **Duy trì thứ tự:**
   - Vị trí phần tử quan trọng
   - Có thể truy xuất theo vị trí
3. **Kích thước động:**
   - Có thể thêm/xóa phần tử

- Không cần biết trước số lượng

**Ví dụ thực tế:**

- Facebook News Feed: List of posts
- Gmail: List of emails
- Music Player: Playlist
- E-commerce: Shopping cart

### 5.2.2. List ADT (Abstract Data Type)

**WHAT - List ADT**

List ADT định nghĩa các phép toán trên List mà không quan tâm đến cách triển khai.

**Basic Concepts:**

- Empty list: Không có phần tử nào
- Length/Size: Số lượng phần tử hiện tại
- Head: Đầu danh sách
- Tail: Cuối danh sách

**Basic Operations:**

1. `Create()` - Tạo list rỗng
2. `Insert(element, position)` - Chèn phần tử
3. `Remove(position)` - Xóa phần tử
4. `Search(element)` - Tìm kiếm
5. `Retrieve(position)` - Lấy phần tử tại vị trí
6. `Traverse()` - Duyệt qua tất cả phần tử

**Extended Operations:**

1. `isEmpty()` - Kiểm tra rỗng
2. `isFull()` - Kiểm tra đầy (với array)
3. `Size()` - Lấy số lượng phần tử
4. `Clear()` - Xóa tất cả
5. `Replace(position, element)` - Thay thế
6. `Merge(list1, list2)` - Gộp 2 list
7. `Append(list1, list2)` - Nối 2 list

**Điều kiện thành công:**

- Insert: List chưa đầy
- Remove/Retrieve: List không rỗng và vị trí hợp lệ

### 5.2.3. Array-based List (Danh sách dựa trên mảng)

**WHAT - Array-based List**

List được triển khai bằng mảng (array), các phần tử được lưu liên tiếp trong bộ nhớ.

**Cấu trúc:**

```
class ArrayList {
private:
    int* data;      // Mảng lưu dữ liệu
    int size;       // Số phần tử hiện tại
    int capacity;   // Dung lượng tối đa
};
```

**Dynamic Array:**

Để linh hoạt hơn, ta dùng **Dynamic Array** - mảng có thể tự động mở rộng:

```
class DynamicArray {
private:
    int* storage;
    int size;
    int capacity;

public:
    DynamicArray() {
        capacity = 10;  // Khởi tạo với 10 phần tử
        size = 0;
        storage = new int[capacity];
    }

    void ensureCapacity(int minCapacity) {
        if (minCapacity > capacity) {
            int newCapacity = (capacity * 3) / 2 + 1;  // Tăng 1.5 lần
            if (newCapacity < minCapacity) {
                newCapacity = minCapacity;
            }
            resize(newCapacity);
        }
    }

    void resize(int newCapacity) {
        int* newStorage = new int[newCapacity];
        memcpy(newStorage, storage, sizeof(int) * size);
```

```
        delete[] storage;
        storage = newStorage;
        capacity = newCapacity;
    }
};
```

**Tại sao dùng Dynamic Array?**

1. **Flexible size:** Không cần biết trước số lượng phần tử
2. **Fast access:** Truy xuất O(1) theo index
3. **Cache-friendly:** Dữ liệu liên tiếp → CPU cache hiệu quả

**Nhược điểm:**

- Insert/Delete ở giữa → O(n) (phải dịch chuyển phần tử)
- Resize tốn thời gian (copy toàn bộ mảng)

### 5.2.4. Singly Linked List (Danh sách liên kết đơn)

**WHAT - Linked List**

**Linked List** là danh sách các node, mỗi node chứa:

1. **Data:** Dữ liệu
2. **Link (next):** Con trỏ tới node tiếp theo

```
[Data|next] → [Data|next] → [Data|next] → NULL
  head
```

**Node Structure:**

```
struct Node {
    int data;
    Node* next;

    Node() : next(nullptr) {}
    Node(int d) : data(d), next(nullptr) {}
};
```

**Linked List Structure:**

```
class LinkedList {
private:
    Node* head;
    int count;

public:
    LinkedList() : head(nullptr), count(0) {}

    ~LinkedList() {
        // Giải phóng tất cả node
        Node* current = head;
        while (current != nullptr) {
            Node* temp = current;
            current = current->next;
            delete temp;
        }
    }
};
```

**Tại sao dùng Linked List?**

1. **Dynamic size:** Dễ dàng thêm/xóa phần tử

2. **Efficient insert/delete:** O(1) nếu biết vị trí (không cần dịch chuyển)
3. **No resize:** Không cần cấp phát lại bộ nhớ như array

**Nhược điểm:**

- Truy xuất chậm: O(n) để tìm phần tử thứ i
- Tốn bộ nhớ: Mỗi node cần lưu con trỏ next
- Cache-unfriendly: Node rải rác trong RAM

**Trong bộ nhớ:**

Array-based List:

```
Memory: [1][2][3][4][5] ← Liên tiếp
Address: 0x100, 0x104, 0x108, 0x10C, 0x110
```

Linked List:

```
Memory:
Node 1: [data: 1, next: 0x500] ← 0x100
Node 2: [data: 2, next: 0x300] ← 0x500
Node 3: [data: 3, next: 0x700] ← 0x300
Node 4: [data: 4, next: NULL]  ← 0x700

Nodes rải rác trong RAM, không liên tiếp!
```

## 5.3. Bản chất trong máy tính

### 5.3.1. Array-based List trong Memory

```
ArrayList (capacity = 10, size = 5):

Stack:
  [data = 0x1000]  ← Con trỏ tới heap
  [size = 5]
  [capacity = 10]

Heap:
  0x1000: [10]  ← data[0]
  0x1004: [20]  ← data[1]
  0x1008: [30]  ← data[2]
  0x100C: [40]  ← data[3]
  0x1010: [50]  ← data[4]
  0x1014: [?]   ← data[5] (unused)
  ...
  0x1024: [?]   ← data[9] (unused)
```

**Truy xuất data[i]:**

```
Address = base_address + i * sizeof(int)
Address of data[3] = 0x1000 + 3 * 4 = 0x100C
→ O(1) - Rất nhanh!
```

### 5.3.2. Linked List trong Memory

```
LinkedList:

Stack:
  [head = 0x2000]
  [count = 4]

Heap (rải rác):
  0x2000: [data: 10, next: 0x3500]  ← Node 1 (head)
  0x3500: [data: 20, next: 0x2800]  ← Node 2
  0x2800: [data: 30, next: 0x4100]  ← Node 3
  0x4100: [data: 40, next: NULL]    ← Node 4
```

**Truy xuất phần tử thứ 3:**

```
current = head;           // 0x2000
current = current->next;  // 0x3500
current = current->next;  // 0x2800
return current->data;     // 30
→ O(n) - Phải duyệt qua từng node!
```

**Tại sao Linked List chậm hơn?**

1. Phải duyệt tuần tự (không random access)
2. Cache miss: Node rải rác → CPU phải đọc từ RAM nhiều lần

## 5.4. Lịch sử / Nguồn gốc

### 5.4.1. Lịch sử của List

**1955: Linked List**

- Được phát minh bởi Allen Newell, Cliff Shaw, và Herbert A. Simon
- Dùng trong ngôn ngữ IPL (Information Processing Language)
- Mục đích: Lưu trữ biểu tượng trong AI research

**1958: LISP (List Processing)**

- John McCarthy tạo ngôn ngữ LISP
- Linked List là cấu trúc dữ liệu chính
- `car` (head) và `cdr` (tail) - Thuật ngữ vẫn dùng đến giờ

**1960s-1970s:**

- Doubly Linked List
- Circular Linked List
- Skip List (1989)

**Ngày nay:**

- `std::list` (C++ STL) - Doubly Linked List
- `std::vector` (C++ STL) - Dynamic Array
- `ArrayList` (Java)
- `LinkedList` (Java)

## 5.5. Phân tích thuật toán

### 5.5.1. Array-based List Operations

**Insert at position i:**

```
Algorithm insertAt(arr, i, value, size, capacity)
Pre: 0 <= i <= size, size < capacity
Post: value inserted at position i

1. Ensure capacity (resize if needed)
2. Shift elements from i to size-1 right by 1
3. arr[i] = value
4. size++
End insertAt
```

**Phân tích:**

- Best case: Insert ở cuối → O(1)
- Worst case: Insert ở đầu → O(n) (shift n phần tử)
- Average: O(n/2) = O(n)

**Remove at position i:**

```
Algorithm removeAt(arr, i, size)
Pre: 0 <= i < size
Post: element at position i removed

1. Shift elements from i+1 to size-1 left by 1
2. size--
End removeAt
```

**Phân tích:**

- Best case: Remove cuối → O(1)
- Worst case: Remove đầu → O(n)
- Average: O(n)

**Bảng tổng hợp:**

**Complexity:**

- Time: Insert/Remove: Best O(1), Worst O(n), Avg O(n); Access: O(1); Search: O(n) unsorted, O(log n) sorted
- Space: O(n)

### 5.5.2. Singly Linked List Operations

**Insert at head:**

```
Algorithm insertHead(list, value)
Pre: list is initialized
Post: new node inserted at head

1. newNode = create new node with value
2. newNode->next = list.head
3. list.head = newNode
4. list.count++
End insertHead
```

**Complexity:**

- Time: O(1) - Chỉ thay đổi con trỏ
- Space: O(1)

**Insert after a node:**

```
Algorithm insertAfter(pPre, value)
Pre: pPre points to a node in list
Post: new node inserted after pPre

1. newNode = create new node with value
2. newNode->next = pPre->next
3. pPre->next = newNode
4. list.count++
End insertAfter
```

**Complexity:**

- Time: O(1) - Nếu biết pPre
- Space: O(1)

**Delete at head:**

```
Algorithm deleteHead(list)
Pre: list is not empty
Post: first node deleted

1. temp = list.head
2. list.head = list.head->next
3. delete temp
4. list.count--
End deleteHead
```

**Complexity:**

- Time: O(1)
- Space: O(1)

**Search:**

```
Algorithm search(list, value)
Pre: list is initialized
Post: return node with value or NULL

1. current = list.head
2. while current != NULL
3.     if current->data == value
4.         return current
5.     current = current->next
6. return NULL
End search
```

**Complexity:**

- Time: Best: O(1), Worst: O(n), Avg: O(n)
- Space: O(1)

## 5.6. Minh họa từng bước

### 5.6.1. Array List - Insert at position 2

```
Initial: [10, 20, 30, 40, 50]  size = 5
Insert 99 at position 2

Step 1: Shift elements from position 2 to 4
  [10, 20, 30, 40, 50, ?]
              ↓   ↓   ↓
  [10, 20, ?, 30, 40, 50]

Step 2: Insert 99 at position 2
  [10, 20, 99, 30, 40, 50]

Step 3: size++
  size = 6

Result: [10, 20, 99, 30, 40, 50]
```

### 5.6.2. Linked List - Insert at head

```
Initial:
  head → [20|●] → [30|●] → [40|NULL]

Insert 10 at head:

Step 1: Create new node
  newNode → [10|NULL]

Step 2: newNode->next = head
  newNode → [10|●]
              ↓
  head → [20|●] → [30|●] → [40|NULL]

Step 3: head = newNode
  head → [10|●] → [20|●] → [30|●] → [40|NULL]

Result:
  head → [10|●] → [20|●] → [30|●] → [40|NULL]
```

### 5.6.3. Linked List - Delete first node

```
Initial:
  head → [10|●] → [20|●] → [30|NULL]
```

```
Step 1: temp = head
  temp ────┐
  head → [10|●] → [20|●] → [30|NULL]

Step 2: head = head->next
  temp → [10|●] ╳
  head ──────→ [20|●] → [30|NULL]

Step 3: delete temp
  head → [20|●] → [30|NULL]

Result:
  head → [20|●] → [30|NULL]
```

## 5.7. Code minh họa bằng C++

### 5.7.1. Dynamic Array List - Complete Implementation

```
#include <iostream>
#include <cstring>
using namespace std;

class DynamicArray {
private:
    int* storage;
    int size;
    int capacity;

    void setCapacity(int newCapacity) {
        int* newStorage = new int[newCapacity];
        memcpy(newStorage, storage, sizeof(int) * size);
        delete[] storage;
        storage = newStorage;
        capacity = newCapacity;
    }

    void ensureCapacity(int minCapacity) {
        if (minCapacity > capacity) {
            int newCapacity = (capacity * 3) / 2 + 1;
            if (newCapacity < minCapacity) {
                newCapacity = minCapacity;
            }
            setCapacity(newCapacity);
        }
    }

    void pack() {
        if (size <= capacity / 2) {
            int newCapacity = (size * 3) / 2 + 1;
            setCapacity(newCapacity);
        }
    }

    void rangeCheck(int index) {
        if (index < 0 || index >= size) {
            throw "Index out of bounds!";
        }
```

```
    }

public:
    DynamicArray(int cap = 10) {
        capacity = cap;
        size = 0;
        storage = new int[capacity];
    }

    ~DynamicArray() {
        delete[] storage;
    }

    int getSize() { return size; }
    bool isEmpty() { return size == 0; }

    int get(int index) {
        rangeCheck(index);
        return storage[index];
    }

    void set(int index, int value) {
        rangeCheck(index);
        storage[index] = value;
    }

    void insertAt(int index, int value) {
        if (index < 0 || index > size) {
            throw "Index out of bounds!";
        }
        ensureCapacity(size + 1);

        // Shift elements right
        int moveCount = size - index;
        if (moveCount > 0) {
            memmove(storage + index + 1,
                    storage + index,
                    sizeof(int) * moveCount);
        }

        storage[index] = value;
        size++;
```

```
    }

    void add(int value) {
        insertAt(size, value);  // Insert at end
    }

    void removeAt(int index) {
        rangeCheck(index);

        // Shift elements left
        int moveCount = size - index - 1;
        if (moveCount > 0) {
            memmove(storage + index,
                    storage + index + 1,
                    sizeof(int) * moveCount);
        }

        size--;
        pack();
    }

    int search(int value) {
        for (int i = 0; i < size; i++) {
            if (storage[i] == value) {
                return i;
            }
        }
        return -1;  // Not found
    }

    void print() {
        cout << "[";
        for (int i = 0; i < size; i++) {
            cout << storage[i];
            if (i < size - 1) cout << ", ";
        }
        cout << "]" << endl;
    }
};

int main() {
    DynamicArray arr;
```

```
    arr.add(10);
    arr.add(20);
    arr.add(30);
    arr.print();  // [10, 20, 30]

    arr.insertAt(1, 99);
    arr.print();  // [10, 99, 20, 30]

    arr.removeAt(2);
    arr.print();  // [10, 99, 30]

    cout << "Index of 99: " << arr.search(99) << endl;  // 1

    return 0;
}
```

### 5.7.2. Singly Linked List - Complete Implementation

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;

    Node() : next(nullptr) {}
    Node(int d) : data(d), next(nullptr) {}
};

class LinkedList {
private:
    Node* head;
    int count;

public:
    LinkedList() : head(nullptr), count(0) {}

    ~LinkedList() {
        clear();
    }

    int getSize() { return count; }
    bool isEmpty() { return head == nullptr; }

    // Insert at head - O(1)
    void insertHead(int value) {
        Node* newNode = new Node(value);
        newNode->next = head;
        head = newNode;
        count++;
    }

    // Insert at tail - O(n)
    void insertTail(int value) {
        Node* newNode = new Node(value);

        if (head == nullptr) {
            head = newNode;
```

```
        } else {
            Node* current = head;
            while (current->next != nullptr) {
                current = current->next;
            }
            current->next = newNode;
        }
        count++;
    }

    // Insert after a specific node
    void insertAfter(Node* pPre, int value) {
        if (pPre == nullptr) {
            cout << "Previous node cannot be NULL!" << endl;
            return;
        }

        Node* newNode = new Node(value);
        newNode->next = pPre->next;
        pPre->next = newNode;
        count++;
    }

    // Delete at head - O(1)
    void deleteHead() {
        if (head == nullptr) {
            cout << "List is empty!" << endl;
            return;
        }

        Node* temp = head;
        head = head->next;
        delete temp;
        count--;
    }

    // Delete node with specific value - O(n)
    bool deleteNode(int value) {
        if (head == nullptr) return false;

        // Special case: head node
        if (head->data == value) {
```

```
            deleteHead();
            return true;
        }

        // Find predecessor of node to delete
        Node* current = head;
        while (current->next != nullptr && current->next->data != value) {
            current = current->next;
        }

        if (current->next == nullptr) {
            return false;  // Not found
        }

        // Delete node
        Node* temp = current->next;
        current->next = temp->next;
        delete temp;
        count--;
        return true;
    }

    // Search - O(n)
    Node* search(int value) {
        Node* current = head;
        while (current != nullptr) {
            if (current->data == value) {
                return current;
            }
            current = current->next;
        }
        return nullptr;
    }

    // Get node at position - O(n)
    Node* getAt(int position) {
        if (position < 0 || position >= count) {
            return nullptr;
        }

        Node* current = head;
        for (int i = 0; i < position; i++) {
```

```
            current = current->next;
        }
        return current;
    }

    // Clear all nodes
    void clear() {
        Node* current = head;
        while (current != nullptr) {
            Node* temp = current;
            current = current->next;
            delete temp;
        }
        head = nullptr;
        count = 0;
    }

    // Print list
    void print() {
        Node* current = head;
        cout << "[";
        while (current != nullptr) {
            cout << current->data;
            if (current->next != nullptr) cout << " -> ";
            current = current->next;
        }
        cout << "]" << endl;
    }

    // Print reverse (using recursion)
    void printReverseHelper(Node* node) {
        if (node == nullptr) return;
        printReverseHelper(node->next);
        cout << node->data;
        if (node != head) cout << " <- ";
    }

    void printReverse() {
        cout << "[";
        printReverseHelper(head);
        cout << "]" << endl;
    }
```

```
};

int main() {
    LinkedList list;

    list.insertHead(30);
    list.insertHead(20);
    list.insertHead(10);
    list.print();  // [10 -> 20 -> 30]

    list.insertTail(40);
    list.print();  // [10 -> 20 -> 30 -> 40]

    Node* node20 = list.search(20);
    if (node20 != nullptr) {
        list.insertAfter(node20, 25);
    }
    list.print();  // [10 -> 20 -> 25 -> 30 -> 40]

    list.deleteNode(25);
    list.print();  // [10 -> 20 -> 30 -> 40]

    list.printReverse();  // [40 <- 30 <- 20 <- 10]

    return 0;
}
```

## 5.8. Những lỗi phổ biến khi code

### 1. Array: Index out of bounds

```
// ❌ SAI: Không kiểm tra index
void insertAt(int index, int value) {
    storage[index] = value;  // Nguy hiểm!
}

// ✅ ĐÚNG: Kiểm tra
void insertAt(int index, int value) {
    if (index < 0 || index > size) {
        throw "Index out of bounds!";
    }
    // ...
}
```

### 2. Array: Quên resize khi đầy

```
// ❌ SAI: Không kiểm tra capacity
void add(int value) {
    storage[size++] = value;  // Overflow khi size >= capacity!
}

// ✅ ĐÚNG
void add(int value) {
    ensureCapacity(size + 1);
    storage[size++] = value;
}
```

### 3. Linked List: Memory leak

```
// ❌ SAI: Không giải phóng node
void deleteNode(int value) {
    if (head->data == value) {
        head = head->next;  // Mất node cũ!
    }
}

// ✅ ĐÚNG
void deleteNode(int value) {
    if (head->data == value) {
        Node* temp = head;
```

```
        head = head->next;
        delete temp;  // Giải phóng bộ nhớ
    }
}
```

### 4. Linked List: Dangling pointer

```
// ❌ SAI: Thay đổi next trước khi lưu
void insertAfter(Node* pPre, int value) {
    Node* newNode = new Node(value);
    pPre->next = newNode;          // Mất link tới node tiếp theo!
    newNode->next = pPre->next;    // newNode->next trỏ tới chính nó!
}

// ✅ ĐÚNG: Lưu next trước
void insertAfter(Node* pPre, int value) {
    Node* newNode = new Node(value);
    newNode->next = pPre->next;    // Lưu next trước
    pPre->next = newNode;
}
```

### 5. Linked List: NULL pointer dereference

```
// ❌ SAI: Không kiểm tra NULL
void deleteAt(int index) {
    Node* current = head;
    for (int i = 0; i < index - 1; i++) {
        current = current->next;  // Có thể NULL!
    }
    current->next = current->next->next;  // Crash nếu current NULL!
}

// ✅ ĐÚNG
void deleteAt(int index) {
    if (index < 0 || index >= count) return;
    // ... kiểm tra NULL ở mọi bước
}
```

## 5.9. Khi nào nên dùng / không nên dùng

### 5.9.1. ArrayList vs LinkedList

| Tiêu chí | ArrayList | LinkedList |
|---|---|---|
| **Access** | O(1) - Random access | O(n) - Sequential only |
| **Insert/Delete đầu** | O(n) - Shift elements | O(1) - Change pointer |
| **Insert/Delete cuối** | O(1) amortized | O(n) - Must traverse |
| **Insert/Delete giữa** | O(n) - Shift | O(1) if know position |
| **Search** | O(n) unsorted, O(log n) sorted | O(n) always |
| **Memory** | Liên tiếp, cache-friendly | Rải rác, cache-unfriendly |
| **Space overhead** | Capacity > size | Extra pointer per node |
| **Resize** | Cần khi đầy (O(n)) | Không cần |

### 5.9.2. Khi nào dùng ArrayList?

**Dùng ArrayList khi:**

1. **Cần random access (truy xuất theo index):**
   - `arr[5]` → O(1)
2. **Ít insert/delete, nhiều read:**
   - Đọc dữ liệu cấu hình
   - Display list (không thay đổi)
3. **Thao tác chủ yếu ở cuối:**
   - Add/remove at end → O(1)
4. **Cần sort hiệu quả:**
   - Quick Sort, Merge Sort hoạt động tốt trên array

**Ví dụ:**

- Lưu điểm sinh viên (100 sinh viên cố định)
- Reading list of products (display only)
- Image pixels trong game

### 5.9.3. Khi nào dùng LinkedList?

**Dùng LinkedList khi:**

1. **Nhiều insert/delete ở đầu:**
   - Browser history (thêm URL mới)
   - Undo/Redo stack
2. **Không biết trước kích thước:**
   - Chat messages (vô hạn)

- Streaming data

3. **Không cần random access:**
   - Chỉ duyệt tuần tự from head to tail
4. **Cần insert/delete giữa list thường xuyên:**
   - Music playlist (insert, remove songs)
   - Task queue

**Ví dụ:**

- Browser history
- Music player playlist
- LRU Cache implementation

## 5.10. Ứng dụng thực tế

**1. Operating System:**

**Process Queue:**

```
// Ready queue cho CPU scheduler
LinkedList<Process> readyQueue;

void addProcess(Process p) {
    readyQueue.insertTail(p);  // FIFO
}

Process getNextProcess() {
    Process p = readyQueue.head->data;
    readyQueue.deleteHead();
    return p;
}
```

**LRU Cache:**

- Doubly Linked List + Hash Map
- Least Recently Used page replacement

**2. Web Browser:**

**History (Back/Forward buttons):**

```
// Browser history using Doubly Linked List
DoublyLinkedList<URL> history;
Node* current;  // Current page

void visitPage(URL url) {
    // Delete all forward history
    while (current->next != nullptr) {
        deleteAfter(current);
    }
    insertAfter(current, url);
    current = current->next;
}

void goBack() {
    if (current->prev != nullptr) {
        current = current->prev;
    }
}
```

```
void goForward() {
    if (current->next != nullptr) {
        current = current->next;
    }
}
```

**3. Music Player:**

**Playlist:**

```
// Doubly Linked List để navigate forward/backward
class Playlist {
    DoublyLinkedList<Song> songs;
    Node* currentSong;

    void next() {
        if (currentSong->next != nullptr) {
            currentSong = currentSong->next;
            playSong(currentSong->data);
        }
    }

    void previous() {
        if (currentSong->prev != nullptr) {
            currentSong = currentSong->prev;
            playSong(currentSong->data);
        }
    }

    void shuffle() {
        // Convert to ArrayList, shuffle, convert back
    }
};
```

**4. Text Editor:**

**Undo/Redo:**

```
// Stack (using LinkedList)
LinkedList<Action> undoStack;
LinkedList<Action> redoStack;
```

```
void execute(Action action) {
    performAction(action);
    undoStack.insertHead(action);
    redoStack.clear();  // Clear redo history
}

void undo() {
    if (!undoStack.isEmpty()) {
        Action action = undoStack.head->data;
        undoStack.deleteHead();
        reverseAction(action);
        redoStack.insertHead(action);
    }
}

void redo() {
    if (!redoStack.isEmpty()) {
        Action action = redoStack.head->data;
        redoStack.deleteHead();
        performAction(action);
        undoStack.insertHead(action);
    }
}
```

**5. E-commerce:**

**Shopping Cart:**

```
// ArrayList cho shopping cart (ít thay đổi sau khi add)
ArrayList<Product> cart;

void addToCart(Product p) {
    cart.add(p);  // O(1) - add at end
}

void updateQuantity(int index, int quantity) {
    cart.get(index).quantity = quantity;  // O(1) - random access
}

double calculateTotal() {
    double total = 0;
    for (int i = 0; i < cart.size(); i++) {
```

```
        total += cart.get(i).price * cart.get(i).quantity;
    }
    return total;
}
```

## 5.11. Tóm tắt chương

**Những điểm quan trọng nhất cần nhớ:**

1. **List ADT** có 2 cách triển khai chính:
   - **Array-based:** Fast access O(1), slow insert/delete O(n)
   - **Linked List:** Slow access O(n), fast insert/delete O(1) at known position
2. **Array-based List:**
   - Truy xuất: O(1)
   - Insert/Delete: O(n) (shift elements)
   - Memory: Liên tiếp, cache-friendly
   - Space: O(n), có thể lãng phí (capacity > size)
3. **Singly Linked List:**
   - Truy xuất: O(n)
   - Insert/Delete at head: O(1)
   - Insert/Delete after known node: O(1)
   - Memory: Rải rác, cache-unfriendly
   - Space: O(n) + O(n) pointers
4. **Khi nào dùng gì:**
   - ArrayList: Cần random access, ít insert/delete
   - LinkedList: Nhiều insert/delete, không cần random access
5. **Các biến thể Linked List:**
   - Singly: Chỉ có next → Duyệt một chiều
   - Doubly: Có next + prev → Duyệt 2 chiều, dễ delete
   - Circular: Cuối nối đầu → Vòng lặp vô hạn
6. **Lỗi thường gặp:**
   - Array: Index out of bounds, quên resize
   - Linked List: Memory leak, NULL pointer, dangling pointer
7. **Ứng dụng thực tế:**
   - ArrayList: Shopping cart, Fixed data, Configuration
   - LinkedList: Browser history, Music playlist, Undo/Redo
8. **Trade-offs:**
   - Time vs Space: Linked List tốn bộ nhớ hơn (pointers)
   - Simplicity vs Flexibility: Array đơn giản nhưng Linked List linh hoạt
9. **C++ STL:**
   - `std::vector` → Dynamic Array

- `std::list` → Doubly Linked List
- `std::forward_list` → Singly Linked List

10. **Luôn nhớ:**
    - Array: Fast random access, slow modification
    - Linked List: Slow access, fast modification
    - Choose based on your use case!

# 6. Stack và Queue

## 6.1. Tổng quan chương

### 6.1.1. Nội dung chính

Stack và Queue là hai cấu trúc dữ liệu **Restricted List** - chỉ cho phép thêm/xóa ở các đầu cụ thể:

**Stack (Ngăn xếp):**

- LIFO: Last-In-First-Out
- Thêm/xóa ở một đầu (top)
- Ứng dụng: Undo/Redo, Function call stack, Expression evaluation

**Queue (Hàng đợi):**

- FIFO: First-In-First-Out
- Thêm ở cuối (rear), xóa ở đầu (front)
- Ứng dụng: Task scheduling, BFS, Print queue

### 6.1.2. Kiến thức nền

- List ADT (Chapter 4)
- Array và Linked List implementation

### 6.1.3. Linear List Concepts

**General list:**

- Không có hạn chế về các thao tác có thể sử dụng
- Không có hạn chế về vị trí thêm/xóa dữ liệu

**Restricted list:**

- Chỉ một số thao tác nhất định có thể sử dụng
- Dữ liệu chỉ có thể thêm/xóa ở các đầu của list

## 6.2. Basic Operations of Stacks

### 6.2.1. Định nghĩa Stack

**Stack** là một dãy hữu hạn các phần tử kiểu T, trong đó tất cả các thao tác thêm và xóa được giới hạn ở một đầu gọi là **top** (đỉnh).

Stack là cấu trúc dữ liệu **LIFO (Last In - First Out)**:

- Phần tử cuối cùng được thêm vào là phần tử đầu tiên được lấy ra

**Ví dụ trực quan:**

```
Stack of plates (chồng đĩa):

    ┌───┐
    │ 3 │ ← Top (thêm/xóa ở đây)
    ├───┤
    │ 2 │
    ├───┤
    │ 1 │
    └───┘


Push 4:     Pop:

┌───┐       ┌───┐
│ 4 │ →     │ 3 │ ← Removed
├───┤       ├───┤
│ 3 │       │ 2 │
├───┤       └───┘
│ 2 │
├───┤
│ 1 │
└───┘
```

### 6.2.2. Các thao tác cơ bản

**Basic operations:**

- Construct: Tạo stack rỗng
- Push: Đưa phần tử mới lên đỉnh stack
- Pop: Lấy phần tử ở đỉnh stack ra
- Top: Truy xuất phần tử ở đỉnh (không xóa)

**Extended operations:**

- isEmpty(): Kiểm tra stack rỗng
- isFull(): Kiểm tra stack đầy
- size(): Số lượng phần tử
- clear(): Xóa toàn bộ stack

### 6.2.3. Tại sao cần Stack?

**Function calls trong chương trình:**

- Mỗi lần gọi hàm → Push vào call stack
- Return → Pop khỏi stack

**Undo/Redo:**

- Mỗi action → Push vào stack
- Undo → Pop ra

**Expression evaluation:**

- Tính toán biểu thức toán học
- Chuyển infix → postfix

## 6.3. Implementation of Stacks

### 6.3.1. Linked-list Implementation

**Stack structure:**

```
stack
  count <integer>
  top <node pointer>
end stack
```

**Stack node structure:**

```
node
  data <dataType>
  next <node pointer>
end node
```

**C++ Implementation:**

```
template <class ItemType>
struct Node {
    ItemType data;
    Node<ItemType>* next;
};

template <class List_ItemType>
class Stack {
public:
    Stack();
    ~Stack();
    void Push(List_ItemType dataIn);
    int Pop(List_ItemType& dataOut);
    int GetStackTop(List_ItemType& dataOut);
    void Clear();
    int IsEmpty();
    int GetSize();
    Stack<List_ItemType>* Clone();
    void Print2Console();

private:
    Node<List_ItemType>* top;
    int count;
};
```

### 6.3.2. Create an Empty Linked Stack

**Algorithm createStack:**

```
Algorithm createStack(ref stack <metadata>)
  Pre: stack is a metadata structure of a stack
  Post: metadata initialized

  stack.count = 0
  stack.top = null
  return
End createStack
```

**C++ Implementation:**

```
template <class List_ItemType>
Stack<List_ItemType>::Stack() {
    this->top = NULL;
    this->count = 0;
}

template <class List_ItemType>
Stack<List_ItemType>::~Stack() {
    this->Clear();
}
```

### 6.3.3. Push Data into a Linked Stack

**Các bước thực hiện:**

- Cấp phát bộ nhớ cho node mới và thiết lập dữ liệu
- Cập nhật con trỏ: Node mới trỏ đến top hiện tại
- Con trỏ top trỏ đến node mới
- Tăng count

**Algorithm pushStack:**

```
Algorithm pushStack(ref stack <metadata>, val data <dataType>)
  Pre: stack is a metadata structure to a valid stack
       data contains value to be pushed into the stack
  Post: data have been pushed in stack
  Return: true if successful; false if memory overflow

  if stack full then
    success = false
  else
    allocate(pNew)
    pNew->data = data
    pNew->next = stack.top
    stack.top = pNew
    stack.count = stack.count + 1
    success = true
  end
  return success
End pushStack
```

**C++ Implementation:**

```
template <class List_ItemType>
void Stack<List_ItemType>::Push(List_ItemType value) {
    Node<List_ItemType>* pNew = new Node<List_ItemType>();
    pNew->data = value;
    pNew->next = this->top;
    this->top = pNew;
    this->count++;
}
```

**Lưu ý:**

- Push thành công khi cấp phát bộ nhớ thành công
- Không có sự khác biệt giữa push vào stack có phần tử và stack rỗng

### 6.3.4. Pop Linked Stack

**Các bước thực hiện:**

- dltPtr giữ phần tử ở đỉnh stack
- top trỏ đến phần tử kế tiếp
- Giải phóng dltPtr và giảm count

**Algorithm popStack:**

```
Algorithm popStack(ref stack <metadata>, ref dataOut <dataType>)
  Pre: stack is a metadata structure to a valid stack
       dataOut is to receive the popped data
  Post: data have been returned to caller
  Return: true if successful; false if stack is empty

  if stack empty then
    success = false
  else
    dltPtr = stack.top
    dataOut = stack.top->data
    stack.top = stack.top->next
    stack.count = stack.count - 1
    recycle(dltPtr)
    success = true
  end
  return success
End popStack
```

**C++ Implementation:**

```
template <class List_ItemType>
int Stack<List_ItemType>::Pop(List_ItemType& dataOut) {
    if (this->GetSize() == 0)
        return 0;

    Node<List_ItemType>* dltPtr = this->top;
    dataOut = dltPtr->data;
    this->top = dltPtr->next;
    this->count--;
    delete dltPtr;
    return 1;
}
```

### 6.3.5. Stack Top

**Algorithm stackTop:**

```
Algorithm stackTop(ref stack <metadata>, ref dataOut <dataType>)
  Pre: stack is a metadata structure to a valid stack
       dataOut is to receive top stack data
  Post: data have been returned to caller
  Return: true if successful; false if stack is empty

  if stack empty then
    success = false
  else
    dataOut = stack.top->data
    success = true
  end
  return success
End stackTop
```

**C++ Implementation:**

```
template <class List_ItemType>
int Stack<List_ItemType>::GetStackTop(List_ItemType& dataOut) {
    if (this->GetSize() == 0)
        return 0;
    dataOut = this->top->data;
    return 1;
}
```

### 6.3.6. Destroy Stack

**Algorithm destroyStack:**

```
Algorithm destroyStack(ref stack <metadata>)
  Pre: stack is a metadata structure to a valid stack
  Post: stack empty and all nodes recycled

  if stack not empty then
    while stack.top not null do
      temp = stack.top
      stack.top = stack.top->next
      recycle(temp)
    end
  end
  stack.count = 0
```

```
  return
End destroyStack
```

**C++ Implementation:**

```
template <class List_ItemType>
void Stack<List_ItemType>::Clear() {
    Node<List_ItemType>* temp;
    while (this->top != NULL) {
        temp = this->top;
        this->top = this->top->next;
        delete temp;
    }
    this->count = 0;
}
```

### 6.3.7. isEmpty và isFull Linked Stack

**isEmpty Algorithm:**

```
Algorithm isEmpty(ref stack <metadata>)
  Pre: stack is a metadata structure to a valid stack
  Post: return stack status
  Return: true if the stack is empty, false otherwise

  if count = 0 then
    Return true
  else
    Return false
  end
End isEmpty
```

**C++ Implementation:**

```
template <class List_ItemType>
int Stack<List_ItemType>::IsEmpty() {
    return (count == 0);
}

template <class List_ItemType>
int Stack<List_ItemType>::GetSize() {
    return count;
}

template <class List_ItemType>
int Stack<List_ItemType>::IsFull() {
    Node<List_ItemType>* pNew = new Node<List_ItemType>();
    if (pNew != NULL) {
        delete pNew;
        return 0;
    } else {
        return 1;
    }
}
```

### 6.3.8. Print Stack

```
template <class List_ItemType>
void Stack<List_ItemType>::Print2Console() {
    Node<List_ItemType>* p;
```

```
    p = this->top;
    while (p != NULL) {
        cout << p->data << " ";
        p = p->next;
    }
    cout << endl;
}
```

### 6.3.9. Using Linked Stack

```
int main(int argc, char* argv[]) {
    Stack<int>* myStack = new Stack<int>();
    int val;

    myStack->Push(7);
    myStack->Push(9);
    myStack->Push(10);
    myStack->Push(8);

    myStack->Print2Console();  // 8 10 9 7

    myStack->Pop(val);
    myStack->Print2Console();  // 10 9 7

    delete myStack;
    return 0;
}
```

## 6.4. Array-based Stack Implementation

### 6.4.1. Nguyên lý

Implementation của array-based stack rất đơn giản. Sử dụng biến `top` để trỏ đến phần tử trên cùng của stack trong mảng.

**Các bước:**

- Ban đầu: top = −1
- Push: Tăng top lên 1 và ghi phần tử vào storage[top]
- Pop: Kiểm tra top != −1 và giảm top đi 1
- getTop: Kiểm tra top != −1 và trả về storage[top]
- isEmpty: Trả về boolean nếu top == −1

### 6.4.2. C++ Implementation

```
#include <string>
using namespace std;

class ArrayStack {
private:
    int top;
    int capacity;
    int* storage;

public:
    ArrayStack(int capacity) {
        storage = new int[capacity];
        this->capacity = capacity;
        top = -1;
    }

    ~ArrayStack() {
        delete[] storage;
    }

    void push(int value) {
        if (top == capacity - 1)
            throw string("Stack is overflow");
        top++;
        storage[top] = value;
    }

    void pop(int& dataOut) {
        if (top == -1)
```

```
            throw string("Stack is empty");
        dataOut = storage[top];
        top--;
    }

    int getTop() {
        if (top == -1)
            throw string("Stack is empty");
        return storage[top];
    }

    bool isEmpty() {
        return (top == -1);
    }

    bool isFull() {
        return (top == capacity - 1);
    }

    int getSize() {
        return top + 1;
    }

    void print2Console() {
        if (top > -1) {
            for (int i = top; i >= 0; i--) {
                cout << storage[i] << " ";
            }
            cout << endl;
        }
    }
};
```

### 6.4.3. Using Array-based Stack

```
int main(int argc, char* argv[]) {
    ArrayStack* myStack = new ArrayStack(10);
    int val;

    myStack->push(7);
    myStack->push(9);
    myStack->push(10);
    myStack->push(8);

    myStack->print2Console();  // 8 10 9 7

    myStack->pop(val);
    myStack->print2Console();  // 10 9 7

    delete myStack;
    return 0;
}
```

### 6.4.4. So sánh Array vs Linked Stack

**Array-based Stack:**

- Độ phức tạp: Push/Pop/Top: O(1)
- Không gian: Fixed size O(MAX_SIZE)
- Ưu điểm: Đơn giản, truy xuất nhanh
- Nhược điểm: Kích thước cố định

**Linked Stack:**

- Độ phức tạp: Push/Pop/Top: O(1)
- Không gian: Dynamic O(n)
- Ưu điểm: Kích thước linh hoạt
- Nhược điểm: Overhead của con trỏ

## 6.5. Applications of Stack

Stack được sử dụng trong nhiều ứng dụng quan trọng:

**Reversing Data:**

- Đảo ngược danh sách
- Chuyển đổi Decimal sang Binary

**Parsing:**

- Kiểm tra cặp ngoặc (Brackets Parse)
- Phân tích cú pháp

**Postponement of Processing:**

- Chuyển đổi Infix sang Postfix
- Tính giá trị biểu thức Postfix

**Backtracking:**

- Bài toán Goal Seeking
- Knight's Tour
- Thoát khỏi mê cung
- Bài toán Eight Queens

**Function Call Stack:**

```
void c() { /* ... */ }
void b() { c(); }
void a() { b(); }
int main() { a(); }

Call Stack:
┌─────────┐
│  c()    │ ← Top
├─────────┤
│  b()    │
├─────────┤
│  a()    │
├─────────┤
│ main()  │
└─────────┘
```

Khi c() return → Pop khỏi stack → Quay lại b()

**Stack Overflow:**

```
void infiniteRecursion() {
    infiniteRecursion();  // Không có base case!
}
// → Stack tràn → Program crash!
```

## 6.6. Basic Operations of Queues

### 6.6.1. Định nghĩa Queue

**Queue** là một dãy hữu hạn các phần tử kiểu T, trong đó dữ liệu chỉ có thể được thêm vào ở một đầu gọi là **rear** (cuối) và xóa từ đầu kia gọi là **front** (đầu).

Queue là cấu trúc dữ liệu **FIFO (First In - First Out)**:

- Phần tử đầu tiên được thêm vào là phần tử đầu tiên được lấy ra

**Ví dụ trực quan:**

```
Queue at ticket counter (hàng mua vé):
                    Rear (thêm vào đây)
                      ↓
Front → [1][2][3][4][5]
↑
Dequeue (lấy ra ở đây)

Enqueue 6:          Dequeue:
Front → [1][2][3][4][5][6]    Front → [2][3][4][5]
                                      (1 removed)
```

### 6.6.2. Các thao tác cơ bản

**Basic operations:**

- Construct: Tạo queue rỗng
- Enqueue: Đưa phần tử mới vào cuối queue
- Dequeue: Lấy phần tử ở đầu queue ra
- Queue Front: Truy xuất phần tử đầu (không xóa)
- Queue Rear: Truy xuất phần tử cuối (không xóa)

### 6.6.3. Tại sao cần Queue?

**Task Scheduling:**

- CPU scheduling: Các process chờ CPU
- Print queue: Các file chờ in

**BFS (Breadth-First Search):**

- Duyệt graph/tree theo từng cấp độ

**Buffering:**

- Keyboard buffer
- Network packet queue

## 6.7. Implementation of Queue

### 6.7.1. Linked-list Implementation

**Queue structure:**

```
queue
  count <integer>
  front <node pointer>
  rear <node pointer>
end queue
```

**Queue node structure:**

```
node
  data <dataType>
  next <node pointer>
end node
```

**C++ Implementation:**

```
template <class ItemType>
struct Node {
    ItemType data;
    Node<ItemType>* next;
};

template <class List_ItemType>
class Queue {
public:
    Queue();
    ~Queue();
    void Enqueue(List_ItemType dataIn);
    int Dequeue(List_ItemType& dataOut);
    int GetQueueFront(List_ItemType& dataOut);
    int GetQueueRear(List_ItemType& dataOut);
    void Clear();
    int IsEmpty();
    int GetSize();
    void Print2Console();

private:
    Node<List_ItemType>* front;
    Node<List_ItemType>* rear;
```

```
    int count;
};
```

### 6.7.2. Create Queue

**Algorithm createQueue:**

```
Algorithm createQueue(ref queue <metadata>)
  Pre: queue is a metadata structure of a queue
  Post: metadata initialized

  queue.count = 0
  queue.front = null
  queue.rear = null
  return
End createQueue
```

**C++ Implementation:**

```
template <class List_ItemType>
Queue<List_ItemType>::Queue() {
    this->count = 0;
    this->front = NULL;
    this->rear = NULL;
}

template <class List_ItemType>
Queue<List_ItemType>::~Queue() {
    this->Clear();
}
```

### 6.7.3. Enqueue

**Algorithm enqueue:**

```
Algorithm enqueue(ref queue <metadata>, val data <dataType>)
  Pre: queue is a metadata structure of a valid queue
       data contains data to be inserted into queue
  Post: data have been inserted in queue
  Return: true if successful, false if memory overflow

  if queue full then
    return false
  end

  allocate(newPtr)
  newPtr->data = data
  newPtr->next = null
```

```
  if queue.count = 0 then
    queue.front = newPtr  // Insert into an empty queue
  else
    queue.rear->next = newPtr  // Insert into a queue with data
  end

  queue.rear = newPtr
  queue.count = queue.count + 1
  return true
End enqueue
```

**C++ Implementation:**

```
template <class List_ItemType>
void Queue<List_ItemType>::Enqueue(List_ItemType value) {
    Node<List_ItemType>* newPtr = new Node<List_ItemType>();
    newPtr->data = value;
    newPtr->next = NULL;

    if (this->count == 0)
        this->front = newPtr;
    else
        this->rear->next = newPtr;

    this->rear = newPtr;
    this->count++;
}
```

### 6.7.4. Dequeue

**Algorithm dequeue:**

```
Algorithm dequeue(ref queue <metadata>, ref dataOut <dataType>)
  Pre: queue is a metadata structure of a valid queue
       dataOut is to receive dequeued data
  Post: front data have been returned to caller
  Return: true if successful, false if queue is empty

  if queue empty then
    return false
  end

  dataOut = queue.front->data
  dltPtr = queue.front

  if queue.count = 1 then
    queue.rear = NULL  // Delete data in a queue with only one item
  end

  queue.front = queue.front->next
  queue.count = queue.count - 1
  recycle(dltPtr)
  return true
End dequeue
```

**C++ Implementation:**

```
template <class List_ItemType>
int Queue<List_ItemType>::Dequeue(List_ItemType& dataOut) {
    if (count == 0)
        return 0;

    dataOut = front->data;
    Node<List_ItemType>* dltPtr = this->front;

    if (count == 1)
        this->rear = NULL;

    this->front = this->front->next;
    this->count--;
    delete dltPtr;
    return 1;
}
```

### 6.7.5. Queue Front and Rear

```
template <class List_ItemType>
int Queue<List_ItemType>::GetQueueFront(List_ItemType& dataOut) {
    if (count == 0)
        return 0;
    dataOut = this->front->data;
    return 1;
}

template <class List_ItemType>
int Queue<List_ItemType>::GetQueueRear(List_ItemType& dataOut) {
    if (count == 0)
        return 0;
    dataOut = this->rear->data;
    return 1;
}
```

### 6.7.6. Destroy Queue

**Algorithm destroyQueue:**

```
Algorithm destroyQueue(ref queue <metadata>)
  Pre: queue is a metadata structure of a valid queue
  Post: queue empty and all nodes recycled

  if queue not empty then
    while queue.front not null do
      temp = queue.front
      queue.front = queue.front->next
      recycle(temp)
    end
  end

  queue.front = NULL
  queue.rear = NULL
  queue.count = 0
  return
End destroyQueue
```

**C++ Implementation:**

```
template <class List_ItemType>
void Queue<List_ItemType>::Clear() {
    Node<List_ItemType>* temp;
    while (this->front != NULL) {
        temp = this->front;
        this->front = this->front->next;
        delete temp;
    }
    this->front = NULL;
    this->rear = NULL;
    this->count = 0;
}
```

### 6.7.7. Queue Empty

```
template <class List_ItemType>
int Queue<List_ItemType>::IsEmpty() {
    return (this->count == 0);
}
```

```
template <class List_ItemType>
int Queue<List_ItemType>::GetSize() {
    return this->count;
}
```

### 6.7.8. Print Queue

```
template <class List_ItemType>
void Queue<List_ItemType>::Print2Console() {
    Node<List_ItemType>* p;
    p = this->front;
    cout << "Front: ";
    while (p != NULL) {
        cout << p->data << " ";
        p = p->next;
    }
    cout << endl;
}
```

### 6.7.9. Using Linked Queue

```
int main(int argc, char* argv[]) {
    Queue<int>* myQueue = new Queue<int>();
    int val;

    myQueue->Enqueue(7);
    myQueue->Enqueue(9);
    myQueue->Enqueue(10);
    myQueue->Enqueue(8);

    myQueue->Print2Console();  // Front: 7 9 10 8

    myQueue->Dequeue(val);
    myQueue->Print2Console();  // Front: 9 10 8

    delete myQueue;
    return 1;
}
```

## 6.8. Array-based Queue Implementation

### 6.8.1. C++ Implementation

```
#include <string>
using namespace std;

class ArrayQueue {
private:
    int capacity;
    int front;
    int rear;
    int* storage;

public:
    ArrayQueue(int capacity) {
        storage = new int[capacity];
        this->capacity = capacity;
        front = -1;
        rear = -1;
    }

    ~ArrayQueue() {
        delete[] storage;
    }

    void enQueue(int value) {
        if (isFull())
            throw string("Queue is full");

        if (front == -1)
            front = 0;

        rear++;
        storage[rear % capacity] = value;
    }

    void deQueue(int& valueOut) {
        if (isEmpty())
            throw string("Queue is empty");

        valueOut = storage[front % capacity];
        front++;
```

```
    }

    int getFront() {
        if (isEmpty())
            throw string("Queue is empty");
        return storage[front % capacity];
    }

    int getRear() {
        if (isEmpty())
            throw string("Queue is empty");
        return storage[rear % capacity];
    }

    bool isEmpty() {
        return (front > rear || front == -1);
    }

    bool isFull() {
        return (rear - front + 1 == capacity);
    }

    int getSize() {
        return rear - front + 1;
    }
};
```

### 6.8.2. Using Array-based Queue

```
int main(int argc, char* argv[]) {
    ArrayQueue* myQueue = new ArrayQueue(10);
    int val;

    myQueue->enQueue(7);
    myQueue->enQueue(9);
    myQueue->enQueue(10);
    myQueue->enQueue(8);

    myQueue->deQueue(val);  // val = 7

    delete myQueue;
    return 1;
}
```

### 6.8.3. So sánh Array vs Linked Queue

**Array-based Queue (Circular):**

- Độ phức tạp: Enqueue/Dequeue: O(1)
- Không gian: Fixed O(MAX_SIZE)
- Ưu điểm: Đơn giản, hiệu quả
- Nhược điểm: Kích thước cố định

**Linked Queue:**

- Độ phức tạp: Enqueue/Dequeue: O(1)
- Không gian: Dynamic O(n)
- Ưu điểm: Kích thước linh hoạt
- Nhược điểm: Overhead của con trỏ

## 6.9. Applications of Queue

Queue được sử dụng trong nhiều ứng dụng:

**Polynomial Arithmetic:**

- Tính toán đa thức

**Categorizing Data:**

- Phân loại dữ liệu

**Expression Evaluation:**

- Tính giá trị biểu thức Prefix

**Radix Sort:**

- Thuật toán sắp xếp cơ số

**Queue Simulation:**

- Mô phỏng hàng đợi

**Task Scheduling:**

- CPU scheduling
- Print spooler
- Network packet routing

**BFS Algorithm:**

- Duyệt đồ thị theo chiều rộng

## 6.10. Lỗi phổ biến

**Stack Overflow:**

```
// ❌ Không kiểm tra full
void push(int x) {
    data[++top] = x;  // Nguy hiểm!
}

// ✅ ĐÚNG
void push(int x) {
    if (isFull()) {
        throw "Stack overflow!";
    }
    data[++top] = x;
}
```

**Queue: Quên dùng modulo trong circular queue:**

```
// ❌ SAI
rear = rear + 1;

// ✅ ĐÚNG
rear = (rear + 1) % MAX_SIZE;
```

**Pop/Dequeue từ cấu trúc rỗng:**

```
// ❌ Không kiểm tra empty
int pop() {
    return data[top--];  // Lỗi nếu stack rỗng!
}

// ✅ ĐÚNG
int pop() {
    if (isEmpty()) {
        throw "Stack underflow!";
    }
    return data[top--];
}
```

## 6.11. Khi nào dùng Stack và Queue

**Dùng Stack khi:**

- Cần LIFO (phần tử gần nhất vào - gần nhất ra)
- Undo/Redo operations
- Backtracking algorithms
- Expression evaluation
- DFS (Depth-First Search)
- Function call management

**Dùng Queue khi:**

- Cần FIFO (xử lý theo thứ tự đến)
- Task scheduling
- BFS (Breadth-First Search)
- Buffering (keyboard, network)
- Print queue management
- Request handling

## 6.12. Tóm tắt

**Stack - LIFO:**

- Thêm và xóa ở một đầu (top)
- Operations: Push, Pop, Top - tất cả O(1)
- Implementation: Array (fixed size) hoặc Linked List (dynamic)
- Ứng dụng: Function calls, Undo/Redo, Expression evaluation, DFS

**Queue - FIFO:**

- Thêm ở rear, xóa ở front
- Operations: Enqueue, Dequeue, Front, Rear - tất cả O(1)
- Implementation: Array (circular, fixed) hoặc Linked List (dynamic)
- Ứng dụng: Scheduling, BFS, Buffering, Print queue

**Lựa chọn Implementation:**

- Array: Khi biết trước kích thước tối đa, cần truy xuất nhanh
- Linked List: Khi kích thước thay đổi động, không biết trước giới hạn

# 7. Cây (Tree)

## 7.1. Tổng quan chương

### 7.1.1. Nội dung chính của chương

Tree là cấu trúc dữ liệu phi tuyến tính (non-linear), có tính phân cấp (hierarchical), là nền tảng cho nhiều cấu trúc dữ liệu và thuật toán phức tạp.

Chương này sẽ giúp bạn nắm vững:

1. **Basic Tree Concepts:**
   - Định nghĩa Tree
   - Các thuật ngữ: Root, Leaf, Parent, Child, Sibling, Height, Depth
   - Properties của Tree
2. **Binary Tree:**
   - Định nghĩa và tính chất
   - Complete Tree, Balanced Tree
   - Array vs Linked implementation
3. **Tree Traversals:**
   - Depth-First: Preorder, Inorder, Postorder
   - Breadth-First (Level-order)
4. **Expression Trees:**
   - Biểu diễn biểu thức toán học
   - Infix, Prefix, Postfix notation
5. **Binary Search Tree (BST):**
   - Properties
   - Search, Insert, Delete operations
   - Complexity analysis

### 7.1.2. Kiến thức nền cần biết trước

- Recursion (Chapter 3)
- Linked List (Chapter 4)
- Queue (Chapter 5)
- Pointers và Dynamic memory allocation

## 7.2. Giải thích từng khái niệm

### 7.2.1. Tree (Cây)

**WHAT - Cây là gì?**

**Tree** là một tập hợp hữu hạn các **nodes** (đỉnh) được kết nối bởi các **branches** (cạnh), trong đó:

- Có một node đặc biệt gọi là **root** (gốc)
- Mọi node khác đều có đúng một parent (cha)
- Không có chu trình (cycle)

**Minh họa:**

```
        a         ← Root
      / | \
     b  c  d      ← Level 1
    / \     |\  \
   e   f   g h  i ← Level 2 (Leaves)
```

**Các thuật ngữ quan trọng:**

| Thuật ngữ | Định nghĩa |
|---|---|
| Root | Node đầu tiên, không có parent. Indegree = 0 |
| Leaf | Node không có child. Outdegree = 0 |
| Internal node | Node không phải root hoặc leaf |
| Parent | Node có outdegree > 0 |
| Child | Node có indegree = 1 |
| Siblings | Các nodes có cùng parent |
| Ancestors | Tất cả nodes từ root đến node hiện tại |
| Descendants | Tất cả nodes từ node hiện tại đến leaves |
| Path | Chuỗi nodes liên tiếp từ node này đến node khác |
| Level | Khoảng cách từ root (root ở level 0) |
| Height | Level của leaf sâu nhất + 1 |
| Degree | Số children của một node |
| Subtree | Bất kỳ node nào và tất cả descendants của nó |

**Ví dụ với tree trên:**

- Root: a
- Leaves: c, e, f, g, h, i

- Internal nodes: b, d
- Siblings: {b, c, d}, {e, f}, {g, h, i}
- Height = 3
- Parent of e: b
- Children of d: g, h, i

**Tại sao cần Tree?**

1. **Biểu diễn dữ liệu phân cấp:**
   - File system: Folders chứa subfolders và files
   - Organization chart: CEO → Managers → Employees
   - DOM tree: HTML elements lồng nhau
2. **Search hiệu quả:**
   - Binary Search Tree: O(log n) search
   - Nhanh hơn nhiều so với linear search O(n)
3. **Lưu trữ sorted data:**
   - BST inorder traversal → sorted list
4. **Routing algorithms:**
   - Network routing
   - Decision trees trong AI

**Tree hoạt động thế nào trong máy tính?**

Có 2 cách triển khai chính:

1. **Linked Representation:**

```
struct Node {
    int data;
    Node* children[];  // Mảng con trỏ tới children
};
```

2. **Array Representation (cho Complete Tree):**

```
array[i] → Node i
Parent of i: (i-1)/2
Left child of i: 2i+1
Right child of i: 2i+2
```

### 7.2.2. Binary Tree (Cây nhị phân)

**WHAT - Binary Tree**

**Binary Tree** là tree mà mỗi node có **tối đa 2 children**:

- Left child (con trái)
- Right child (con phải)

Định nghĩa đệ quy:

- Binary tree rỗng, hoặc
- Có root + left subtree (BT) + right subtree (BT)

**Minh họa:**

```
        a
       / \
      b   d
     / \   \
    e   f   g
```

**Properties của Binary Tree:**

Với N nodes, height H:

| Tính chất | Công thức |
|---|---|
| Min height | $H_{\min} = \lfloor \log_2 N \rfloor + 1$ |
| Max height | $H_{\max} = N$ |
| Min nodes (given H) | $N_{\min} = H$ |
| Max nodes (given H) | $N_{\max} = 2^H - 1$ |

**Các loại Binary Tree đặc biệt:**

1. **Complete Binary Tree:**
   - Tất cả levels đầy, trừ level cuối
   - Level cuối: Nodes fill từ trái sang

```
       a
     /   \
    b     c
   / \   /
  d   e f
```

2. **Full Binary Tree (Perfect):**
   - Tất cả levels đầy

- Số nodes: $N = 2^H - 1$

```
      a
    /   \
   b     c
  / \   / \
 d   e f   g
```

3. **Balanced Binary Tree:**
   - Độ chênh lệch height giữa left và right subtree ≤ 1
   - Balance Factor: BF = Height(left) - Height(right)
   - BF ∈ {-1, 0, 1}

**Implementation:**

1. **Linked Implementation:**

```
struct Node {
    int data;
    Node* left;
    Node* right;
};

struct BinaryTree {
    Node* root;
};
```

2. **Array Implementation (cho Complete Tree):**

```
int tree[MAX_SIZE];
// Parent của i: (i-1)/2
// Left của i: 2i+1
// Right của i: 2i+2
```

### 7.2.3. Tree Traversals (Duyệt cây)

**WHAT - Traversal**

**Tree Traversal** là quá trình visit mỗi node trong tree đúng một lần theo một thứ tự nhất định.

2 loại chính:

- **Depth-First Traversal**: Đi sâu trước
- **Breadth-First Traversal**: Đi rộng trước (level-by-level)

**Depth-First Traversals:**

1. **Preorder (NLR - Node-Left-Right):**
   - Process node
   - Traverse left subtree
   - Traverse right subtree

   ```
   Algorithm preorder(root):
       if root ≠ null:
           visit(root)
           preorder(root.left)
           preorder(root.right)
   ```

2. **Inorder (LNR - Left-Node-Right):**
   - Traverse left subtree
   - Process node
   - Traverse right subtree

   ```
   Algorithm inorder(root):
       if root ≠ null:
           inorder(root.left)
           visit(root)
           inorder(root.right)
   ```

3. **Postorder (LRN - Left-Right-Node):**
   - Traverse left subtree
   - Traverse right subtree
   - Process node

   ```
   Algorithm postorder(root):
       if root ≠ null:
           postorder(root.left)
           postorder(root.right)
           visit(root)
   ```

**Ví dụ với tree:**

```
      1
     / \
    2   3
   / \
  4   5
```

- Preorder: 1, 2, 4, 5, 3
- Inorder: 4, 2, 5, 1, 3
- Postorder: 4, 5, 2, 3, 1

**Breadth-First (Level-order):**

- Duyệt từng level từ trái sang phải
- Dùng Queue

```
Algorithm levelorder(root):
    queue = createQueue()
    enqueue(queue, root)
    while not empty(queue):
        node = dequeue(queue)
        visit(node)
        if node.left ≠ null: enqueue(queue, node.left)
        if node.right ≠ null: enqueue(queue, node.right)
```

- Kết quả: 1, 2, 3, 4, 5

### 7.2.4. Binary Search Tree (BST)

**WHAT - BST**

**Binary Search Tree** là Binary Tree với tính chất:

1. Tất cả nodes trong **left subtree < root**
2. Tất cả nodes trong **right subtree ≥ root**
3. Mỗi subtree cũng là BST (đệ quy)

**Ví dụ BST:**

```
        23
       /  \
     18    44
    / \    / \
  12  20  35  52
```

**Valid:** 12 < 18 < 20 < 23 < 35 < 44 < 52

**Invalid BST:**

```
      10
     /  \
    5    15
   / \
  2   20  ← SAI: 20 > 10 nhưng ở left subtree!
```

**Tính chất quan trọng:**

- **Inorder traversal của BST → Sorted list!**
- BST kết hợp ưu điểm:
  - Binary search (O(log n) search)
  - Linked list (O(1) insert/delete nếu biết vị trí)

**Tại sao BST quan trọng?**

1. **Fast search:** O(log n) average, O(h) với h = height
2. **Dynamic:** Dễ dàng insert/delete
3. **Sorted order:** Inorder traversal → sorted
4. **Range queries:** Tìm tất cả phần tử trong [x, y] hiệu quả

## 7.3. Bản chất trong máy tính

### 7.3.1. Memory Layout của Binary Tree

**Linked Representation:**

```
Binary Tree:
        10
       /  \
      5    15

Memory:
┌──────────────────────┐
│ Node (root): 10      │ 0x1000
│ left: 0x2000         │
│ right: 0x3000        │
├──────────────────────┤
│ Node: 5              │ 0x2000
│ left: NULL           │
│ right: NULL          │
├──────────────────────┤
│ Node: 15             │ 0x3000
│ left: NULL           │
│ right: NULL          │
└──────────────────────┘
```

Nodes rải rác trong heap, kết nối bởi pointers.

**Array Representation:**

```
Binary Tree:       Array:
      0           [0, 1, 2, 3, 4, 5, 6]
     / \
    1   2
   / \  /
  3  4 5

array[0] = root
array[1] = left(0)
array[2] = right(0)
array[3] = left(1)
array[4] = right(1)
array[5] = left(2)
```

**Space efficiency:**

- Linked: O(n) nodes + O(n) pointers = O(2n) ≈ O(n)
- Array (Complete Tree): O(n) - No pointers!
- Array (Sparse Tree): Lãng phí space cho NULL nodes

### 7.3.2. Recursion trong Tree Operations

Tree operations thường dùng recursion vì tree có cấu trúc đệ quy tự nhiên:

```
int height(Node* root) {
    if (root == NULL) return 0;  // Base case

    // Recursive case
    int leftHeight = height(root->left);
    int rightHeight = height(root->right);

    return 1 + max(leftHeight, rightHeight);
}
```

**Call Stack cho tree:**

```
        1
       / \
      2   3

height(1):
  height(2):
    height(4): return 1
    height(5): return 1
    return 2
  height(3): return 1
  return 3
```

## 7.4. Lịch sử / Nguồn gốc

### 7.4.1. Lịch sử của Tree Structure

**1950s: Khởi đầu**

- Trees được dùng trong compilers (syntax trees)
- Expression trees cho mathematical expressions

**1960: Binary Search Trees**

- P.F. Windley, A.D. Booth đề xuất BST
- Đột phá: Kết hợp binary search và linked list

**1962: AVL Trees**

- Adelson-Velsky và Landis phát minh AVL tree
- First self-balancing BST
- Đảm bảo O(log n) worst-case

**1970: B-Trees**

- Rudolf Bayer và Ed McCreight (Boeing)
- Multi-way search trees cho databases
- Tối ưu cho disk I/O

**1972: Red-Black Trees**

- Rudolf Bayer
- Dùng trong C++ STL `std::map`, `std::set`

**Ngày nay:**

- Trees everywhere: File systems, databases, compilers, AI
- Advanced variants: Splay trees, Treaps, Segment trees

### 7.4.2. Tại sao Trees ra đời?

**Vấn đề với Array và Linked List:**

- Array: Fast search O(log n) nhưng slow insert/delete O(n)
- Linked List: Fast insert/delete O(1) nhưng slow search O(n)

**Giải pháp: Binary Search Tree**

- Search: O(log n) - Gần như array
- Insert/Delete: O(log n) - Tốt hơn array nhiều

**Nhưng còn vấn đề:** BST có thể unbalanced → O(n) worst case

**Giải pháp: Self-balancing trees (AVL, Red-Black)**

- Đảm bảo O(log n) mọi trường hợp

## 7.5. Phân tích thuật toán

### 7.5.1. BST Search

```
Algorithm searchBST(root, target):
    if root == NULL:
        return NULL

    if target == root.data:
        return root
    else if target < root.data:
        return searchBST(root.left, target)
    else:
        return searchBST(root.right, target)
```

**Phân tích:**

- Mỗi lần so sánh loại đi một nửa tree (như binary search)
- Số bước tối đa = height của tree

**Time Complexity:** Best: O(1) - target = root, Average: O(log n) - balanced tree, Worst: O(n) - skewed tree (như linked list)

**Space Complexity:** O(h) - recursion stack, h = height

**Skewed Tree (worst case):**

```
1
 \
  2
   \
    3
     \
      4  → height = 4, search = O(4) = O(n)
```

**Balanced Tree (best case):**

```
      2
     / \
    1   3
         \
          4  → height = 3, search = O(log 4) ≈ O(2)
```

### 7.5.2. BST Insert

```
Algorithm insertBST(root, value):
    if root == NULL:
```

```
        return createNode(value)

    if value < root.data:
        root.left = insertBST(root.left, value)
    else:
        root.right = insertBST(root.right, value)

    return root
```

**Complexity:**

- Time: Best: O(log n) - balanced, Worst: O(n) - skewed
- Space: O(h) recursion

**Insert luôn diễn ra ở leaf hoặc near-leaf position!**

### 7.5.3. BST Delete

**3 trường hợp:**

1. **Delete leaf:** Đơn giản, set parent pointer = NULL
2. **Delete node with 1 child:** Replace node bằng child
3. **Delete node with 2 children:**
   - Tìm inorder predecessor (max của left subtree)
   - Hoặc inorder successor (min của right subtree)
   - Replace data, delete predecessor/successor

**Time Complexity:** O(h) - phải traverse để tìm node và predecessor/successor

**Space Complexity:** O(h) recursion

### 7.5.4. Tree Traversals

**Time Complexity:** O(n) - visit mỗi node đúng 1 lần

**Space Complexity:** Depth-First: O(h) recursion stack, Breadth-First: O(w) queue, w = max width

## 7.6. Minh họa từng bước

### 7.6.1. BST Insert

Insert: 50, 30, 70, 20, 40, 60, 80

```
Step 1: Insert 50
    50

Step 2: Insert 30 (30 < 50 → left)
    50
   /
  30

Step 3: Insert 70 (70 > 50 → right)
    50
   /  \
  30   70

Step 4: Insert 20 (20 < 50 → left, 20 < 30 → left)
      50
     /  \
   30    70
  /
 20

Step 5: Insert 40 (40 < 50 → left, 40 > 30 → right)
      50
     /  \
   30    70
  /  \
 20   40

Steps 6-7: Insert 60, 80
        50
       /  \
     30    70
    / \    / \
  20  40  60  80
```

### 7.6.2. BST Delete node with 2 children

Delete 50 from above tree:

```
Initial:
        50 ← Delete this
       /  \
     30    70
    / \    / \
  20  40  60  80

Step 1: Find inorder predecessor (max of left subtree)
        → 40 is the largest in left subtree

Step 2: Replace 50 with 40
        40
       /  \
     30    70
    / \    / \
  20  ??  60  80

Step 3: Delete original 40 (leaf node)
        40
       /  \
     30    70
    /     / \
  20     60  80
```

### 7.6.3. Traversals

Tree:

```
      1
     / \
    2   3
   / \
  4   5
```

**Preorder (NLR):**

```
Visit 1 → Visit 2 → Visit 4 → Visit 5 → Visit 3
Result: 1, 2, 4, 5, 3
```

**Inorder (LNR):**

```
Visit 4 → Visit 2 → Visit 5 → Visit 1 → Visit 3
Result: 4, 2, 5, 1, 3
```

**Postorder (LRN):**

```
Visit 4 → Visit 5 → Visit 2 → Visit 3 → Visit 1
Result: 4, 5, 2, 3, 1
```

**Level-order:**

```
Level 0: 1
Level 1: 2, 3
Level 2: 4, 5
Result: 1, 2, 3, 4, 5
```

## 7.7. Code minh họa bằng C++

### 7.7.1. Binary Tree Node Structure

```
#include <iostream>
#include <queue>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};
```

### 7.7.2. Binary Search Tree - Complete Implementation

```
class BST {
private:
    Node* root;

    // Helper: Insert recursively
    Node* insertRec(Node* node, int value) {
        if (node == nullptr) {
            return new Node(value);
        }

        if (value < node->data) {
            node->left = insertRec(node->left, value);
        } else {
            node->right = insertRec(node->right, value);
        }

        return node;
    }

    // Helper: Search recursively
    Node* searchRec(Node* node, int value) {
        if (node == nullptr || node->data == value) {
            return node;
        }

        if (value < node->data) {
```

```
            return searchRec(node->left, value);
        } else {
            return searchRec(node->right, value);
        }
    }

    // Helper: Find minimum node
    Node* findMin(Node* node) {
        while (node->left != nullptr) {
            node = node->left;
        }
        return node;
    }

    // Helper: Delete recursively
    Node* deleteRec(Node* node, int value) {
        if (node == nullptr) return nullptr;

        if (value < node->data) {
            node->left = deleteRec(node->left, value);
        } else if (value > node->data) {
            node->right = deleteRec(node->right, value);
        } else {
            // Found node to delete

            // Case 1: Leaf node
            if (node->left == nullptr && node->right == nullptr) {
                delete node;
                return nullptr;
            }

            // Case 2: One child
            if (node->left == nullptr) {
                Node* temp = node->right;
                delete node;
                return temp;
            }
            if (node->right == nullptr) {
                Node* temp = node->left;
                delete node;
                return temp;
            }
```

```
            // Case 3: Two children
            // Find inorder successor (min in right subtree)
            Node* successor = findMin(node->right);
            node->data = successor->data;
            node->right = deleteRec(node->right, successor->data);
        }

        return node;
    }

    // Helper: Inorder traversal
    void inorderRec(Node* node) {
        if (node == nullptr) return;
        inorderRec(node->left);
        cout << node->data << " ";
        inorderRec(node->right);
    }

    // Helper: Preorder traversal
    void preorderRec(Node* node) {
        if (node == nullptr) return;
        cout << node->data << " ";
        preorderRec(node->left);
        preorderRec(node->right);
    }

    // Helper: Postorder traversal
    void postorderRec(Node* node) {
        if (node == nullptr) return;
        postorderRec(node->left);
        postorderRec(node->right);
        cout << node->data << " ";
    }

    // Helper: Height
    int heightRec(Node* node) {
        if (node == nullptr) return 0;
        return 1 + max(heightRec(node->left), heightRec(node->right));
    }

public:
```

```
    BST() : root(nullptr) {}

    void insert(int value) {
        root = insertRec(root, value);
    }

    bool search(int value) {
        return searchRec(root, value) != nullptr;
    }

    void remove(int value) {
        root = deleteRec(root, value);
    }

    void inorder() {
        inorderRec(root);
        cout << endl;
    }

    void preorder() {
        preorderRec(root);
        cout << endl;
    }

    void postorder() {
        postorderRec(root);
        cout << endl;
    }

    // Level-order (Breadth-First)
    void levelOrder() {
        if (root == nullptr) return;

        queue<Node*> q;
        q.push(root);

        while (!q.empty()) {
            Node* current = q.front();
            q.pop();

            cout << current->data << " ";
```

```
            if (current->left) q.push(current->left);
            if (current->right) q.push(current->right);
        }
        cout << endl;
    }

    int height() {
        return heightRec(root);
    }
};

int main() {
    BST tree;

    // Insert nodes
    tree.insert(50);
    tree.insert(30);
    tree.insert(70);
    tree.insert(20);
    tree.insert(40);
    tree.insert(60);
    tree.insert(80);

    cout << "Inorder (sorted): ";
    tree.inorder();  // 20 30 40 50 60 70 80

    cout << "Preorder: ";
    tree.preorder();  // 50 30 20 40 70 60 80

    cout << "Postorder: ";
    tree.postorder();  // 20 40 30 60 80 70 50

    cout << "Level-order: ";
    tree.levelOrder();  // 50 30 70 20 40 60 80

    cout << "Height: " << tree.height() << endl;  // 3

    // Search
    cout << "Search 40: " << (tree.search(40) ? "Found" : "Not found") << endl;
    cout << "Search 100: " << (tree.search(100) ? "Found" : "Not found") << endl;

    // Delete
```

```
    tree.remove(50);
    cout << "After deleting 50: ";
    tree.inorder();  // 20 30 40 60 70 80

    return 0;
}
```

**Output:**

```
Inorder (sorted): 20 30 40 50 60 70 80
Preorder: 50 30 20 40 70 60 80
Postorder: 20 40 30 60 80 70 50
Level-order: 50 30 70 20 40 60 80
Height: 3
Search 40: Found
Search 100: Not found
After deleting 50: 20 30 40 60 70 80
```

## 7.8. Những lỗi phổ biến khi code

### 1. Memory Leak - Quên delete nodes

```
// ❌ SAI: Không giải phóng bộ nhớ
void deleteTree(Node* root) {
    if (root == nullptr) return;
    deleteTree(root->left);
    deleteTree(root->right);
    // Quên: delete root;
}

// ✅ ĐÚNG
void deleteTree(Node* root) {
    if (root == nullptr) return;
    deleteTree(root->left);
    deleteTree(root->right);
    delete root;  // Giải phóng node
}
```

### 2. NULL Pointer Dereference

```
// ❌ SAI: Không kiểm tra NULL
int height(Node* root) {
    return 1 + max(height(root->left), height(root->right));
    // Crash khi root == NULL!
}

// ✅ ĐÚNG
int height(Node* root) {
    if (root == nullptr) return 0;  // Base case
    return 1 + max(height(root->left), height(root->right));
}
```

### 3. Sai logic Delete với 2 children

```
// ❌ SAI: Quên delete successor node
Node* deleteNode(Node* root, int value) {
    // ...
    // Case 3: Two children
    Node* successor = findMin(root->right);
    root->data = successor->data;
    // Quên: root->right = deleteRec(root->right, successor->data);
```

```
    return root;  // Successor vẫn còn trong tree!
}

// ✅ ĐÚNG: Phải delete successor
root->right = deleteRec(root->right, successor->data);
```

### 4. Infinite Recursion

```
// ❌ SAI: Không có base case
void inorder(Node* root) {
    inorder(root->left);   // Crash khi root == NULL
    cout << root->data;
    inorder(root->right);
}

// ✅ ĐÚNG
void inorder(Node* root) {
    if (root == nullptr) return;  // Base case
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}
```

### 5. Insert sai thứ tự BST

```
// ❌ SAI: Không maintain BST property
Node* insert(Node* root, int value) {
    if (root == nullptr) return new Node(value);
    root->left = insert(root->left, value);  // Luôn insert vào left!
    return root;
}

// ✅ ĐÚNG: So sánh value
Node* insert(Node* root, int value) {
    if (root == nullptr) return new Node(value);
    if (value < root->data) {
        root->left = insert(root->left, value);
    } else {
        root->right = insert(root->right, value);
    }
    return root;
}
```

## 7.9. Khi nào nên dùng / không nên dùng

### 7.9.1. Khi nào dùng Binary Search Tree?

| Tình huống | Lý do |
|---|---|
| Cần search + insert/delete | O(log n) cho cả 3 operations (balanced) |
| Dữ liệu động | Dễ dàng thêm/xóa, không cần resize như array |
| Cần sorted order | Inorder traversal → sorted list |
| Range queries | Tìm tất cả x trong [a, b] hiệu quả |
| Priority queue (nâng cao) | Với some modifications |

**Ví dụ ứng dụng:**

```
// Dictionary với fast search
BST<string, string> dictionary;
dictionary.insert("apple", "a fruit");
dictionary.insert("banana", "yellow fruit");
string meaning = dictionary.search("apple");  // O(log n)

// Auto-complete suggestions
BST<string> words;
// Find all words starting with "app"
words.rangeSearch("app", "apq");
```

### 7.9.2. Khi nào KHÔNG nên dùng BST?

| Tình huống | Lý do |
|---|---|
| Dữ liệu đã sorted và static | Dùng sorted array + binary search → O(log n), no overhead |
| Cần O(1) search | Dùng Hash Table thay vì BST O(log n) |
| Dữ liệu insert theo thứ tự | BST thành skewed → O(n), cần AVL/Red-Black |
| Memory critical | BST tốn memory cho pointers |

### 7.9.3. So sánh với các cấu trúc khác

| Operation | BST | Array | Hash Table |
|---|---|---|---|
| Search | O(log n) | O(n) unsorted \ O(log n) sorted | O(1) |
| Insert | O(log n) | O(n) | O(1) |
| Delete | O(log n) | O(n) | O(1) |

| | | | |
|---|---|---|---|
| Sorted order | ✓ Easy | ✓ Manual sort | ✗ No |
| Range query | ✓ Efficient | ✓ OK | ✗ Bad |
| Memory | O(n) + pointers | O(n) | O(n) + overhead |

**Kết luận:**

- BST: Balanced solution cho search, insert, delete + sorted order
- Array: Tốt khi static, cần random access
- Hash Table: Tốt nhất cho pure search/insert/delete, không cần order

## 7.10. Ứng dụng thực tế

**1. File System:**

```
Directory Structure:
    /
   /|\
  usr var home
  / |    |
bin lib  john
        / | \
      docs music videos
```

- Mỗi folder = node
- Subfolders = children
- Traversal = list all files
- Operations:
  - Create folder: Insert
  - Delete folder: Delete subtree
  - Find file: Search

**2. Database Indexing:**

B-Tree (multi-way BST) trong MySQL, PostgreSQL:

```
// Simplified B-Tree node
struct BTreeNode {
    int keys[M-1];      // Max M-1 keys
    BTreeNode* children[M];  // Max M children
};
```

- Fast search: O(log n)
- Range queries: Find all records with age between 20-30
- Maintain sorted order for ORDER BY queries

**3. Compiler - Expression Trees:**

Expression: `(a + b) * (c - d)`

```
          *
       /   \
      +     -
     / \   / \
    a   b c   d
```

- Parse expression into tree
- Evaluate: Postorder traversal
- Optimize: Tree transformations
- Generate code: Traversal

**4. Auto-complete / Spell Checker:**

```
class Autocomplete {
    BST<string> dictionary;

public:
    void addWord(string word) {
        dictionary.insert(word);
    }

    vector<string> getSuggestions(string prefix) {
        // Find all words >= prefix and < prefix+"z"
        return dictionary.rangeSearch(prefix, prefix + "z");
    }
};

// Usage
Autocomplete ac;
ac.addWord("apple");
ac.addWord("application");
ac.addWord("apply");

vector<string> suggestions = ac.getSuggestions("app");
// Returns: ["apple", "application", "apply"]
```

**5. Game Development - Spatial Partitioning:**

Quadtree (2D BST) cho collision detection:

```
Screen divided into 4 quadrants:
        Root
       / |\ \
     NW NE SW SE
```

- Each node represents a region
- Objects stored at appropriate nodes
- Collision check: Only check objects in same/nearby regions
- Much faster than checking all pairs: $O(n^2) \rightarrow O(n \log n)$

**6. Router Tables (Network):**

IP routing table dùng Trie (variant của tree):

```
Routing Table:
192.168.0.0/16
  ├── 192.168.1.0/24 → Interface A
  └── 192.168.2.0/24 → Interface B
```

- Fast longest prefix match
- O(k) lookup, k = IP address length

## 7.11. Tóm tắt chương

**Những điểm quan trọng nhất cần nhớ:**

1. **Tree** = Cấu trúc phân cấp, phi tuyến tính
   - Root, Leaves, Height, Depth
   - Parent-Child relationships
2. **Binary Tree:** Mỗi node ≤ 2 children
   - Complete Tree: All levels full except last
   - Balanced Tree: |Height(left) - Height(right)| ≤ 1
3. **Traversals:**
   - Depth-First: Preorder (NLR), Inorder (LNR), Postorder (LRN)
   - Breadth-First: Level-order (dùng Queue)
4. **Binary Search Tree:**
   - Left < Root < Right (recursively)
   - Inorder traversal → Sorted list
   - Search/Insert/Delete: O(log n) balanced, O(n) skewed
5. **Implementation:**
   - Linked: Flexible, good for general trees
   - Array: Efficient for complete trees (heap)
6. **Operations Complexity:**

```
| Operation | BST (balanced) | BST (skewed) |
|-----------|----------------|--------------|
| Search    | O(log n)       | O(n)         |
| Insert    | O(log n)       | O(n)         |
| Delete    | O(log n)       | O(n)         |
| Traversal | O(n)           | O(n)         |
```

7. **Khi nào dùng BST:**
   - Cần search + insert/delete nhanh
   - Cần maintain sorted order
   - Range queries
8. **Lỗi thường gặp:**
   - Memory leak (quên delete)
   - NULL pointer dereference
   - Sai logic delete node with 2 children
9. **Ứng dụng:**

- File systems
- Database indexing (B-Trees)
- Compilers (Expression trees, AST)
- Auto-complete
- Game development (Quadtrees)

10. **Vấn đề của Basic BST:** Có thể unbalanced
    - Giải pháp: AVL Trees, Red-Black Trees (Chapter 7)
    - Đảm bảo O(log n) worst-case

# 8. Cây AVL (AVL Tree)

## 8.1. Tổng quan chương

### 8.1.1. Nội dung chính

AVL Tree là Binary Search Tree **tự cân bằng** (self-balancing), đảm bảo operations luôn O(log n) ngay cả worst case.

Nội dung:

1. AVL Tree Concepts và Balance Factor
2. 4 trường hợp mất cân bằng
3. Rotations: Left, Right, Left-Right, Right-Left
4. Insert và Delete operations
5. So sánh với BST thường

### 8.1.2. Kiến thức nền

- Binary Search Tree (Chapter 6)
- Recursion
- Tree rotations

## 8.2. Giải thích từng khái niệm

### 8.2.1. AVL Tree

**WHAT - AVL Tree**

**AVL Tree** là Binary Search Tree với điều kiện cân bằng:

- Với mọi node: |Height(left subtree) - Height(right subtree)| ≤ 1
- Left và right subtrees cũng là AVL trees (đệ quy)

Tên gọi: Adelson-Velsky và Landis (1962)

**Balance Factor (BF):**

```
BF = Height(left) - Height(right)

Trong AVL: BF ∈ {-1, 0, 1}
```

**3 trạng thái:**

- BF = −1: Right subtree cao hơn 1 (RH - Right Higher)
- BF = 0: Cân bằng hoàn hảo (EH - Equal Height)
- BF = +1: Left subtree cao hơn 1 (LH - Left Higher)

**Ví dụ AVL hợp lệ:**

```
      8 (BF=0)
     / \
    5   10 (BF=-1)
   / \    \
  3   6    12
```

**Không phải AVL:**

```
      8 (BF=2) ← |2| > 1, Vi phạm!
     /
    5
   /
  3
```

**Tại sao cần AVL?**

**Vấn đề của BST thường:**

Insert data theo thứ tự: 1, 2, 3, 4, 5

```
1
 \
```

```
2
 \
  3
   \
    4  → Degenerate tree!
     \
      5  → Height = 5 = O(n)
```

Search(5) = O(5) = O(n) - Tệ như linked list!

**Giải pháp: AVL Tree**

Insert 1, 2, 3, 4, 5 vào AVL:

```
     2
    / \
   1   4
      / \
     3   5  → Height = 3 = O(log 5)
```

Search(5) = O(log n) - Luôn đảm bảo!

**Lợi ích:**

1. **Guaranteed O(log n):** Worst case vẫn O(log n)
2. **Predictable performance:** Không bị degraded
3. **Tốt cho real-time systems:** Đảm bảo response time

### 8.2.2. Rotations (Xoay cây)

**WHAT - Rotation**

**Rotation** là thao tác thay đổi cấu trúc tree bằng cách đổi pointers, **không thay đổi thứ tự inorder**.

Mục đích: Giảm height, cân bằng tree.

**Single Rotations:**

1. **Right Rotation:**

```
      y                x
     / \              / \
    x   C    →       A   y
   / \                  / \
  A   B                B   C

Pseudocode:
temp = y.left
y.left = temp.right
temp.right = y
return temp
```

2. **Left Rotation:**

```
    x                  y
   / \                / \
  A   y      →       x   C
     / \            / \
    B   C          A   B

Pseudocode:
temp = x.right
x.right = temp.left
temp.left = x
return temp
```

**Double Rotations:**

3. **Left-Right Rotation:**

```
      z              z              y
     /              /              / \
    x      →       y      →       x   z
     \            /
      y          x
```

Step 1: Left rotate x Step 2: Right rotate z

4. **Right-Left Rotation:**

```
    x              x              y
     \              \            / \
      z      →       y    →     x   z
     /                \
    y                  z
```

Step 1: Right rotate z Step 2: Left rotate x

### 8.2.3. 4 Cases Unbalanced

Sau khi insert/delete, AVL có thể mất cân bằng. Có 4 trường hợp:

**Case 1: Left-Left (LL)**

```
      z (BF=2)
     /
    y (BF=1)
   /
  x

→ Right Rotation at z
```

**Case 2: Right-Right (RR)**

```
  x (BF=-2)
   \
    y (BF=-1)
     \
      z

→ Left Rotation at x
```

**Case 3: Left-Right (LR)**

```
    z (BF=2)
   /
  x (BF=-1)
   \
    y

→ Left rotate x, then Right rotate z
```

**Case 4: Right-Left (RL)**

```
  x (BF=-2)
   \
    z (BF=1)
   /
  y

→ Right rotate z, then Left rotate x
```

## 8.3. Bản chất trong máy tính

### 8.3.1. AVL Node Structure

```
struct AVLNode {
    int data;
    AVLNode* left;
    AVLNode* right;
    int height;  // Lưu height để tính BF nhanh
};

// BF calculation
int getBalance(AVLNode* node) {
    if (node == nullptr) return 0;
    return height(node->left) - height(node->right);
}
```

**Memory:**

- BST thường: 2 pointers (8-16 bytes overhead)
- AVL: 2 pointers + 1 int height (12-20 bytes overhead)

Trade-off: Tốn thêm 4 bytes để đảm bảo O(log n)!

### 8.3.2. Rotation Operations

Rotations chỉ thay đổi **pointers**, không di chuyển data:

```
Before Right Rotation:
  y (0x1000)
  ├─ left: 0x2000 (x)
  └─ right: 0x3000 (C)

x (0x2000)
├─ left: 0x4000 (A)
└─ right: 0x5000 (B)

After Right Rotation:
  x (0x2000) ← New root
  ├─ left: 0x4000 (A)
  └─ right: 0x1000 (y)

y (0x1000)
├─ left: 0x5000 (B)
└─ right: 0x3000 (C)
```

→ Chỉ thay đổi 3 pointers → O(1)!

## 8.4. Lịch sử / Nguồn gốc

**1962: AVL Tree được phát minh**

- Georgy Adelson-Velsky và Evgenii Landis (Liên Xô)
- First self-balancing BST!
- Published in "An algorithm for the organization of information"

**Động lực:**

- BST có thể degenerate thành linked list → O(n)
- Cần đảm bảo O(log n) mọi trường hợp
- Quan trọng cho database và information retrieval

**Sau AVL:**

- 1972: Red-Black Trees (more relaxed balancing)
- 1985: Splay Trees
- Modern databases chủ yếu dùng B-Trees (multiway AVL)

**Ngày nay:**

- AVL: Dùng khi cần search nhiều (read-heavy workloads)
- Red-Black: Dùng khi insert/delete nhiều (write-heavy)
- C++ `std::map`, `std::set` dùng Red-Black (not AVL)

## 8.5. Phân tích thuật toán

### 8.5.1. AVL Insert

```
Algorithm AVLInsert(root, value):
    // 1. Normal BST insert
    if root == NULL:
        return createNode(value)

    if value < root.data:
        root.left = AVLInsert(root.left, value)
    else:
        root.right = AVLInsert(root.right, value)

    // 2. Update height
    root.height = 1 + max(height(root.left), height(root.right))

    // 3. Get balance factor
    balance = height(root.left) - height(root.right)

    // 4. If unbalanced, 4 cases:

    // Left-Left Case
    if balance > 1 and value < root.left.data:
        return rightRotate(root)

    // Right-Right Case
    if balance < -1 and value > root.right.data:
        return leftRotate(root)

    // Left-Right Case
    if balance > 1 and value > root.left.data:
        root.left = leftRotate(root.left)
        return rightRotate(root)

    // Right-Left Case
    if balance < -1 and value < root.right.data:
        root.right = rightRotate(root.right)
        return leftRotate(root)

    return root
```

**Complexity:**

- Time: O(log n) - Guaranteed! Height luôn O(log n)

- Space: O(log n) - Recursion stack

### 8.5.2. AVL Delete

Tương tự Insert:

1. Normal BST delete
2. Update heights
3. Check balance factors
4. Rebalance nếu cần (4 cases)

**Complexity:**

- Time: O(log n) - Guaranteed!
- Space: O(log n)

### 8.5.3. Rotations

**Complexity:**

- Time: O(1) - Chỉ thay đổi pointers
- Space: O(1)

## 8.6. Minh họa từng bước

### 8.6.1. AVL Insert với Rotation

Insert: 10, 20, 30 vào AVL tree

```
Step 1: Insert 10
    10 (BF=0)

Step 2: Insert 20
    10 (BF=-1)
      \
       20 (BF=0)

Step 3: Insert 30
    10 (BF=-2) ← Unbalanced! RR case
      \
       20 (BF=-1)
         \
          30

Step 4: Left Rotation at 10
      20 (BF=0)
     /  \
   10    30

Result: Balanced!
```

### 8.6.2. Left-Right Case

Insert: 30, 10, 20

```
Step 1-2:
    30 (BF=1)
   /
  10 (BF=0)

Step 3: Insert 20
    30 (BF=2) ← Unbalanced! LR case
   /
  10 (BF=-1)
    \
     20

Step 4a: Left Rotation at 10
```

```
    30
   /
  20
 /
10

Step 4b: Right Rotation at 30
      20
     /  \
   10    30

Result: Balanced!
```

## 8.7. Code minh họa C++

```
#include <iostream>
using namespace std;

struct AVLNode {
    int data;
    AVLNode* left;
    AVLNode* right;
    int height;

    AVLNode(int val) : data(val), left(nullptr), right(nullptr), height(1) {}
};

class AVLTree {
private:
    AVLNode* root;

    int height(AVLNode* node) {
        return node ? node->height : 0;
    }

    int getBalance(AVLNode* node) {
        return node ? height(node->left) - height(node->right) : 0;
    }

    void updateHeight(AVLNode* node) {
        if (node) {
            node->height = 1 + max(height(node->left), height(node->right));
        }
    }

    AVLNode* rightRotate(AVLNode* y) {
        AVLNode* x = y->left;
        AVLNode* B = x->right;

        // Rotation
        x->right = y;
        y->left = B;

        // Update heights
        updateHeight(y);
```

```
        updateHeight(x);

        return x;  // New root
    }

    AVLNode* leftRotate(AVLNode* x) {
        AVLNode* y = x->right;
        AVLNode* B = y->left;

        // Rotation
        y->left = x;
        x->right = B;

        // Update heights
        updateHeight(x);
        updateHeight(y);

        return y;  // New root
    }

    AVLNode* insertRec(AVLNode* node, int value) {
        // 1. Normal BST insert
        if (node == nullptr) {
            return new AVLNode(value);
        }

        if (value < node->data) {
            node->left = insertRec(node->left, value);
        } else if (value > node->data) {
            node->right = insertRec(node->right, value);
        } else {
            return node;  // Duplicates not allowed
        }

        // 2. Update height
        updateHeight(node);

        // 3. Get balance factor
        int balance = getBalance(node);

        // 4. Rebalance if needed (4 cases)
```

```
        // Left-Left Case
        if (balance > 1 && value < node->left->data) {
            return rightRotate(node);
        }

        // Right-Right Case
        if (balance < -1 && value > node->right->data) {
            return leftRotate(node);
        }

        // Left-Right Case
        if (balance > 1 && value > node->left->data) {
            node->left = leftRotate(node->left);
            return rightRotate(node);
        }

        // Right-Left Case
        if (balance < -1 && value < node->right->data) {
            node->right = rightRotate(node->right);
            return leftRotate(node);
        }

        return node;
    }

    AVLNode* findMin(AVLNode* node) {
        while (node->left) node = node->left;
        return node;
    }

    AVLNode* deleteRec(AVLNode* node, int value) {
        if (node == nullptr) return nullptr;

        // 1. Normal BST delete
        if (value < node->data) {
            node->left = deleteRec(node->left, value);
        } else if (value > node->data) {
            node->right = deleteRec(node->right, value);
        } else {
            // Found node to delete
            if (!node->left || !node->right) {
                AVLNode* temp = node->left ? node->left : node->right;
```

```
            delete node;
            return temp;
        }

        // Two children
        AVLNode* successor = findMin(node->right);
        node->data = successor->data;
        node->right = deleteRec(node->right, successor->data);
    }

    if (node == nullptr) return nullptr;

    // 2. Update height
    updateHeight(node);

    // 3. Rebalance (same 4 cases)
    int balance = getBalance(node);

    if (balance > 1 && getBalance(node->left) >= 0)
        return rightRotate(node);

    if (balance < -1 && getBalance(node->right) <= 0)
        return leftRotate(node);

    if (balance > 1 && getBalance(node->left) < 0) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }

    if (balance < -1 && getBalance(node->right) > 0) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}

void inorderRec(AVLNode* node) {
    if (!node) return;
    inorderRec(node->left);
    cout << node->data << " ";
    inorderRec(node->right);
```

```
    }

public:
    AVLTree() : root(nullptr) {}

    void insert(int value) {
        root = insertRec(root, value);
    }

    void remove(int value) {
        root = deleteRec(root, value);
    }

    void inorder() {
        inorderRec(root);
        cout << endl;
    }

    int getHeight() {
        return height(root);
    }
};

int main() {
    AVLTree avl;

    // Insert in sorted order (worst for normal BST)
    cout << "Inserting 10, 20, 30, 40, 50..." << endl;
    avl.insert(10);
    avl.insert(20);
    avl.insert(30);
    avl.insert(40);
    avl.insert(50);

    cout << "Inorder: ";
    avl.inorder();  // 10 20 30 40 50

    cout << "Height: " << avl.getHeight() << endl;  // 3 (not 5!)

    // Normal BST would have height 5
    // AVL automatically balances to height 3
```

```
    return 0;
}
```

## 8.8. Lỗi phổ biến

### 1. Quên update height sau rotation

```
// ❌ SAI
AVLNode* rightRotate(AVLNode* y) {
    AVLNode* x = y->left;
    y->left = x->right;
    x->right = y;
    return x;  // Quên update heights!
}

// ✅ ĐÚNG
AVLNode* rightRotate(AVLNode* y) {
    AVLNode* x = y->left;
    AVLNode* B = x->right;
    x->right = y;
    y->left = B;

    updateHeight(y);  // Cập nhật y trước
    updateHeight(x);  // Cập nhật x sau

    return x;
}
```

### 2. Sai thứ tự rotation trong double rotations

```
// ❌ SAI: Right-Left case - Rotate sai thứ tự
if (balance < -1 && getBalance(node->right) > 0) {
    node = rightRotate(node);          // SAI: Rotate root trước
    node->right = leftRotate(node->right);
    return node;
}

// ✅ ĐÚNG
if (balance < -1 && getBalance(node->right) > 0) {
    node->right = rightRotate(node->right);  // Rotate child trước
    return leftRotate(node);                 // Rotate root sau
}
```

## 8.9. Khi nào dùng AVL vs BST vs Red-Black

| | AVL | BST thường | Red-Black |
|---|---|---|---|
| **Search** | O(log n) ✓ | O(n) worst | O(log n) ✓ |
| **Insert** | O(log n) ✓ | O(n) worst | O(log n) ✓ |
| **Balance** | Strict (BF≤1) | No | Relaxed |
| **Rotations** | Nhiều hơn | 0 | Ít hơn AVL |
| **Best for** | Read-heavy | Static sorted | Write-heavy |
| **Ví dụ** | Database index | Fixed config | C++ STL |

**Khi nào dùng AVL:**

- Read >> Write (90% search, 10% insert/delete)
- Cần đảm bảo O(log n) worst-case
- Database read replicas

**Khi nào dùng Red-Black:**

- Write nhiều
- C++ `std::map`, `std::set`
- Linux kernel

## 8.10. Ứng dụng thực tế

**1. Database - In-Memory Index:**

```
// Index cho frequent lookups
AVLTree<int, User> userIndex;  // Key: userID

User getUser(int userID) {
    return userIndex.search(userID);  // O(log n) đảm bảo
}
```

- MySQL, PostgreSQL dùng B-Trees (multi-way AVL)
- Redis dùng Skip List (alternative to AVL)

**2. Auto-complete Systems:**

```
AVLTree<string> dictionary;

// Find all words between "app" and "apq"
vector<string> autocomplete(string prefix) {
    return dictionary.rangeQuery(prefix, prefix + "z");
}
```

**3. Memory Allocators:**

```
// Track free memory blocks
AVLTree<size_t, MemoryBlock> freeBlocks;

void* allocate(size_t size) {
    // Find smallest block >= size → O(log n)
    MemoryBlock block = freeBlocks.findSmallest(size);
    return block.address;
}
```

## 8.11. Tóm tắt

**Điểm quan trọng:**

1. **AVL = BST + Self-balancing**
   - Balance Factor: $|BF| \leq 1$
   - Đảm bảo height = O(log n)
2. **4 Cases unbalanced:**
   - LL → Right rotation
   - RR → Left rotation
   - LR → Left then Right
   - RL → Right then Left
3. **Complexity:**
   - Search/Insert/Delete: O(log n) **đảm bảo**
   - Space: O(n) + height storage
4. **Trade-offs:**
   - AVL: Strict balance → More rotations → Better search
   - Red-Black: Relaxed → Fewer rotations → Better insert
5. **Khi nào dùng:**
   - Read-heavy workloads
   - Cần guaranteed O(log n)
   - Database, Dictionary, Auto-complete
6. **Lỗi thường gặp:**
   - Quên update height
   - Sai thứ tự rotation (double rotations)
   - Không check balance sau insert/delete

# 9. Heap

## 9.1. Tổng quan chương

### 9.1.1. Nội dung chính

Heap là cấu trúc dữ liệu dạng tree đặc biệt, được dùng cho Priority Queue và Heap Sort - một trong những thuật toán sắp xếp hiệu quả nhất.

Nội dung:

1. Heap Definition (Max Heap, Min Heap)
2. Heap Structure và Array Representation
3. Basic Heap Algorithms (ReheapUp, ReheapDown)
4. Insert và Delete operations
5. Build Heap
6. Heap Sort
7. Priority Queue

### 9.1.2. Kiến thức nền

- Binary Tree (Chapter 6)
- Complete Binary Tree
- Array manipulation

## 9.2. Giải thích từng khái niệm

### 9.2.1. Heap

**WHAT - Heap là gì?**

**Heap** là một Complete Binary Tree (hoặc gần Complete) với tính chất heap order:

**Max Heap:**

- Mỗi node ≥ tất cả descendants của nó
- Root = phần tử lớn nhất

**Min Heap:**

- Mỗi node ≤ tất cả descendants của nó
- Root = phần tử nhỏ nhất

**Max Heap example:**

```
        90
       /  \
     80    70
    / \    /
  50  40  60

90 ≥ 80, 70
80 ≥ 50, 40
70 ≥ 60
```

**Min Heap example:**

```
        10
       /  \
     20    30
    / \    /
  40  50  60

10 ≤ 20, 30
20 ≤ 40, 50
30 ≤ 60
```

**Lưu ý quan trọng:**

- Heap ≠ BST! Không có quan hệ left < right
- Chỉ đảm bảo parent-child relationship
- BST: Left < Root < Right (global order)
- Heap: Parent ≥/≤ Children (local order)

**Tại sao cần Heap?**

1. **Priority Queue:**
   - Luôn lấy được max/min trong O(1)
   - Insert/Delete max trong O(log n)
2. **Heap Sort:**
   - O(n log n) worst-case
   - In-place sorting (không cần extra array)
3. **Efficient max/min tracking:**
   - Không cần sort toàn bộ data
   - Chỉ cần maintain heap property

**Ví dụ thực tế:**

- CPU Scheduling: Process với priority cao được xử lý trước
- Dijkstra's Algorithm: Luôn lấy node với distance nhỏ nhất
- Event-driven simulation: Xử lý events theo timestamp
- Top K elements: Tìm K số lớn nhất trong stream

### 9.2.2. Heap Array Representation

**WHAT - Heap trong Array**

Vì Heap là **Complete Binary Tree**, có thể lưu hiệu quả trong array mà không lãng phí space!

**Công thức quan hệ:**

- Node i:
  - Parent: `(i-1)/2`
  - Left child: `2i+1`
  - Right child: `2i+2`

**Ví dụ:**

```
Tree:
        90
       /  \
     80    70
    / \    /
  50  40  60

Array: [90, 80, 70, 50, 40, 60]
Index:  0   1   2   3   4   5

Quan hệ:
- Node 0 (90): left=1(80), right=2(70)
- Node 1 (80): parent=0(90), left=3(50), right=4(40)
- Node 2 (70): parent=0(90), left=5(60), right=6(không có)
- Node 3 (50): parent=1(80)
```

**Lợi ích:**

- Không cần pointers → Tiết kiệm memory
- Cache-friendly (data liên tiếp)
- Dễ navigate (chỉ cần tính toán index)

### 9.2.3. ReheapUp và ReheapDown

**WHAT - Reheap Operations**

**ReheapUp:** Đẩy node lên trên cho đến đúng vị trí (dùng khi insert)

**ReheapDown:** Đẩy node xuống dưới cho đến đúng vị trí (dùng khi delete root)

**ReheapUp Algorithm:**

```
Algorithm reheapUp(heap, position):
    if position > 0:
        parent = (position - 1) / 2
        if heap[position] > heap[parent]:  // Max heap
            swap(heap[position], heap[parent])
            reheapUp(heap, parent)  // Đệ quy lên trên
```

**ReheapDown Algorithm:**

```
Algorithm reheapDown(heap, position, lastPosition):
    leftChild = 2 * position + 1
    rightChild = 2 * position + 2

    if leftChild <= lastPosition:
        // Tìm child lớn hơn
        if rightChild <= lastPosition AND heap[rightChild] > heap[leftChild]:
            largeChild = rightChild
        else:
            largeChild = leftChild

        if heap[largeChild] > heap[position]:
            swap(heap[largeChild], heap[position])
            reheapDown(heap, largeChild, lastPosition)  // Đệ quy xuống
```

**Cách hoạt động:**

**ReheapUp (Insert 95):**

```
Initial:         After insert:    After reheapUp:
    90               90                95
   /  \             /  \              /  \
 80    70    →    80    70     →    90    70
/ \              / \                / \
50  40          50  40              80  40
                   \              /
                   95*          50
```

```
Step 1: Add 95 at end (index 5)
Step 2: Compare with parent 40 → Swap
Step 3: Compare with parent 80 → Swap
Step 4: Compare with parent 90 → Swap
Done!
```

**ReheapDown (Delete root 90):**

```
Initial:         After delete:    After reheapDown:
    90               50                80
   /  \             /  \              /  \
 80    70    →    80    70     →    50    70
/ \              /                 /
50  40          40                 40

Step 1: Move last (50) to root
Step 2: Compare with children (80, 70) → Swap with 80
Step 3: Compare with children (40) → Swap with 40
Done!
```

## 9.3. Bản chất trong máy tính

### 9.3.1. Memory Layout

**Heap trong Array:**

```
Array: [90, 80, 70, 50, 40, 60]

Memory (contiguous):
0x1000: 90  ← heap[0] (root)
0x1004: 80  ← heap[1]
0x1008: 70  ← heap[2]
0x100C: 50  ← heap[3]
0x1010: 40  ← heap[4]
0x1014: 60  ← heap[5]
```

**So với Tree Pointers:**

- Heap: 6 × 4 bytes = 24 bytes
- BST với pointers: 6 × (4 + 8) = 72 bytes (data + 2 pointers)

→ Heap tiết kiệm 3x memory!

### 9.3.2. Cache Performance

Array-based heap rất **cache-friendly**:

- Parent và children gần nhau trong memory
- CPU prefetch hiệu quả
- Thực tế nhanh hơn pointer-based tree

## 9.4. Lịch sử / Nguồn gốc

**1964: Heap Sort**

- J.W.J. Williams phát minh Heap Sort
- First in-place O(n log n) sorting algorithm

**1964: Priority Queue**

- Williams nhận ra Heap hoàn hảo cho Priority Queue

**1970s: Fibonacci Heap**

- Fredman & Tarjan
- Better amortized complexity

**Ngày nay:**

- C++ `std::priority_queue` dùng Heap
- Java `PriorityQueue`
- Python `heapq` module

**Applications mở rộng:**

- Dijkstra's shortest path
- Prim's MST
- Huffman coding
- Median maintenance

## 9.5. Phân tích thuật toán

### 9.5.1. Insert into Heap

```
Algorithm insertHeap(heap, last, data):
    if heap full:
        return false

    last = last + 1
    heap[last] = data
    reheapUp(heap, last)

    return true
```

**Phân tích reheapUp:**

- Worst case: Node bubble up từ leaf đến root
- Số swap tối đa = height = $\lfloor \log_2 n \rfloor$

**Complexity:**

- Time: O(log n)
- Space: O(1) iterative, O(log n) recursive

### 9.5.2. Delete Max (Root)

```
Algorithm deleteHeap(heap, last, dataOut):
    if heap empty:
        return false

    dataOut = heap[0]      // Save root
    heap[0] = heap[last]   // Move last to root
    last = last - 1
    reheapDown(heap, 0, last)

    return true
```

**Complexity:**

- Time: O(log n)
- Space: O(1) iterative, O(log n) recursive

### 9.5.3. Build Heap

**Naive approach:**

```
for i = 1 to n-1:
    reheapUp(heap, i)
```

Time: O(n log n)

**Optimized approach:**

```
for i = (n/2 - 1) down to 0:
    reheapDown(heap, i, n-1)
```

Time: **O(n)** - Tốt hơn!

**Giải thích tại sao O(n):**

- Nodes ở level h: ≤ n/2^(h+1)
- Work per node: h
- Total: Σ (n/2^(h+1)) × h ≈ 2n = O(n)

**Complexity:**

- Time: O(n) - Linear!
- Space: O(1)

### 9.5.4. Heap Sort

```
Algorithm heapSort(array, n):
    // 1. Build max heap
    buildHeap(array, n)  // O(n)

    // 2. Extract max n-1 times
    for i = n-1 down to 1:
        swap(array[0], array[i])  // Move max to end
        reheapDown(array, 0, i-1) // O(log n)

    return array
```

**Complexity:**

- Time: O(n log n) - Guaranteed worst-case!
- Space: O(1) - In-place sorting

**So sánh với Quick Sort:**

- Quick Sort: O(n log n) average, O($n^2$) worst
- Heap Sort: O(n log n) always, but slower in practice (cache-unfriendly swaps)

## 9.6. Minh họa từng bước

### 9.6.1. Build Heap từ array

Array: `[40, 50, 90, 70, 80, 60]`

```
Initial (not heap):
      40
     /  \
   50    90
  / \    /
70  80  60

Step 1: ReheapDown from index 2 (90)
90 ≥ 60 → OK

Step 2: ReheapDown from index 1 (50)
50 < 80 → Swap
      40
     /  \
   80    90
  / \    /
70  50  60

Step 3: ReheapDown from index 0 (40)
40 < max(80, 90) → Swap with 90
      90
     /  \
   80    40
  / \    /
70  50  60

40 < 60 → Swap
      90
     /  \
   80    60
  / \    /
70  50  40

Result: Valid Max Heap!
Array: [90, 80, 60, 70, 50, 40]
```

### 9.6.2. Heap Sort

```
Initial heap: [90, 80, 60, 70, 50, 40]

Iteration 1: Extract max (90)
  Swap 90 with 40: [40, 80, 60, 70, 50 | 90]
  ReheapDown:      [80, 70, 60, 40, 50 | 90]

Iteration 2: Extract max (80)
  Swap 80 with 50: [50, 70, 60, 40 | 80, 90]
  ReheapDown:      [70, 50, 60, 40 | 80, 90]

Iteration 3: Extract max (70)
  Swap 70 with 40: [40, 50, 60 | 70, 80, 90]
  ReheapDown:      [60, 50, 40 | 70, 80, 90]

Iteration 4: Extract max (60)
  Swap 60 with 40: [40, 50 | 60, 70, 80, 90]
  ReheapDown:      [50, 40 | 60, 70, 80, 90]

Iteration 5: Extract max (50)
  Swap 50 with 40: [40 | 50, 60, 70, 80, 90]

Final sorted: [40, 50, 60, 70, 80, 90]
```

## 9.7. Code minh họa C++

```
#include <iostream>
#include <vector>
using namespace std;

class MaxHeap {
private:
    vector<int> heap;

    int parent(int i) { return (i - 1) / 2; }
    int leftChild(int i) { return 2 * i + 1; }
    int rightChild(int i) { return 2 * i + 2; }

    void reheapUp(int position) {
        if (position > 0) {
            int parentPos = parent(position);
            if (heap[position] > heap[parentPos]) {
                swap(heap[position], heap[parentPos]);
                reheapUp(parentPos);
            }
        }
    }

    void reheapDown(int position, int lastPosition) {
        int leftPos = leftChild(position);
        int rightPos = rightChild(position);

        if (leftPos <= lastPosition) {
            int largeChild = leftPos;

            if (rightPos <= lastPosition && heap[rightPos] > heap[leftPos]) {
                largeChild = rightPos;
            }

            if (heap[largeChild] > heap[position]) {
                swap(heap[largeChild], heap[position]);
                reheapDown(largeChild, lastPosition);
            }
        }
    }
```

```
public:
    MaxHeap() {}

    void insert(int value) {
        heap.push_back(value);
        reheapUp(heap.size() - 1);
    }

    bool extractMax(int& maxValue) {
        if (heap.empty()) return false;

        maxValue = heap[0];
        heap[0] = heap.back();
        heap.pop_back();

        if (!heap.empty()) {
            reheapDown(0, heap.size() - 1);
        }

        return true;
    }

    int getMax() {
        return heap.empty() ? -1 : heap[0];
    }

    bool isEmpty() {
        return heap.empty();
    }

    int size() {
        return heap.size();
    }

    void print() {
        cout << "[";
        for (int i = 0; i < heap.size(); i++) {
            cout << heap[i];
            if (i < heap.size() - 1) cout << ", ";
        }
        cout << "]" << endl;
    }
```

```
    // Build heap from array - O(n)
    void buildHeap(vector<int>& arr) {
        heap = arr;
        // Start from last non-leaf node
        for (int i = heap.size() / 2 - 1; i >= 0; i--) {
            reheapDown(i, heap.size() - 1);
        }
    }
};

// Heap Sort
void heapSort(vector<int>& arr) {
    int n = arr.size();

    // 1. Build max heap - O(n)
    for (int i = n / 2 - 1; i >= 0; i--) {
        // ReheapDown inline
        int pos = i;
        while (true) {
            int left = 2 * pos + 1;
            int right = 2 * pos + 2;
            int largest = pos;

            if (left < n && arr[left] > arr[largest])
                largest = left;
            if (right < n && arr[right] > arr[largest])
                largest = right;

            if (largest != pos) {
                swap(arr[pos], arr[largest]);
                pos = largest;
            } else {
                break;
            }
        }
    }

    // 2. Extract max n-1 times - O(n log n)
    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);  // Move max to end
```

```
        // ReheapDown on reduced heap
        int pos = 0;
        while (true) {
            int left = 2 * pos + 1;
            int right = 2 * pos + 2;
            int largest = pos;

            if (left < i && arr[left] > arr[largest])
                largest = left;
            if (right < i && arr[right] > arr[largest])
                largest = right;

            if (largest != pos) {
                swap(arr[pos], arr[largest]);
                pos = largest;
            } else {
                break;
            }
        }
    }
}

int main() {
    MaxHeap heap;

    // Insert elements
    cout << "Inserting 50, 30, 70, 20, 40, 60, 80..." << endl;
    heap.insert(50);
    heap.insert(30);
    heap.insert(70);
    heap.insert(20);
    heap.insert(40);
    heap.insert(60);
    heap.insert(80);

    heap.print();  // [80, 40, 70, 20, 30, 60, 50]
    cout << "Max: " << heap.getMax() << endl;  // 80

    // Extract max
    int maxVal;
    heap.extractMax(maxVal);
    cout << "Extracted: " << maxVal << endl;  // 80
```

```
    heap.print();  // [70, 40, 60, 20, 30, 50]

    // Heap Sort demo
    cout << "\n--- Heap Sort ---" << endl;
    vector<int> arr = {40, 50, 90, 70, 80, 60};
    cout << "Before: ";
    for (int x : arr) cout << x << " ";
    cout << endl;

    heapSort(arr);

    cout << "After: ";
    for (int x : arr) cout << x << " ";
    cout << endl;

    return 0;
}
```

**Output:**

```
Inserting 50, 30, 70, 20, 40, 60, 80...
[80, 40, 70, 20, 30, 60, 50]
Max: 80
Extracted: 80
[70, 40, 60, 20, 30, 50]

--- Heap Sort ---
Before: 40 50 90 70 80 60
After: 40 50 60 70 80 90
```

## 9.8. Lỗi phổ biến

### 1. Sai công thức parent/child

```
// ❌ SAI
int parent(int i) { return i / 2; }        // Thiếu -1
int leftChild(int i) { return 2 * i; }     // Thiếu +1

// ✅ ĐÚNG
int parent(int i) { return (i - 1) / 2; }
int leftChild(int i) { return 2 * i + 1; }
int rightChild(int i) { return 2 * i + 2; }
```

### 2. Không kiểm tra bounds

```
// ❌ SAI
void reheapDown(int pos) {
    int left = leftChild(pos);
    int right = rightChild(pos);
    if (heap[left] > heap[pos]) {  // left có thể out of bounds!
        swap(heap[left], heap[pos]);
    }
}

// ✅ ĐÚNG
void reheapDown(int pos, int lastPos) {
    int left = leftChild(pos);
    if (left <= lastPos) {  // Kiểm tra bounds
        // ...
    }
}
```

### 3. Heap Sort - Quên giảm heap size

```
// ❌ SAI
for (int i = n-1; i > 0; i--) {
    swap(arr[0], arr[i]);
    reheapDown(arr, 0, n-1);  // SAI: Vẫn dùng n-1, không phải i-1!
}

// ✅ ĐÚNG
for (int i = n-1; i > 0; i--) {
    swap(arr[0], arr[i]);
```

```
    reheapDown(arr, 0, i-1);  // Heap size giảm dần
}
```

## 9.9. Khi nào dùng / không dùng

### 9.9.1. Khi nào dùng Heap?

| Tình huống | Lý do |
|---|---|
| Priority Queue | Luôn lấy max/min O(1), insert O(log n) |
| Top K elements | Maintain K largest/smallest elements |
| Median tracking | 2 heaps: max-heap + min-heap |
| Sorting in-place | Heap Sort không cần extra space |
| Graph algorithms | Dijkstra, Prim với priority queue |

### 9.9.2. Heap vs BST vs Array

| Operation | Heap | BST | Sorted Array |
|---|---|---|---|
| Find max/min | O(1) | O(log n) | O(1) |
| Extract max/min | O(log n) | O(log n) | O(n) |
| Insert | O(log n) | O(log n) | O(n) |
| Search arbitrary | O(n) | O(log n) | O(log n) |
| Get sorted | O(n log n) | O(n) | O(1) |
| Space | O(n) | O(n)+pointers | O(n) |

**Kết luận:**

- Heap: Tốt nhất cho **max/min operations**
- BST: Tốt nhất cho **general search**
- Sorted Array: Tốt khi **static data**

## 9.10. Ứng dụng thực tế

**1. CPU Scheduling (OS):**

```
class ProcessScheduler {
    MaxHeap readyQueue;  // Priority = CPU burst time

public:
    void addProcess(Process p) {
        readyQueue.insert(p);  // O(log n)
    }

    Process getNextProcess() {
        Process p;
        readyQueue.extractMax(p);  // Highest priority
        return p;
    }
};
```

Priority levels:

- Real-time processes: Priority 100
- User processes: Priority 50
- Background: Priority 10

**2. Dijkstra's Shortest Path:**

```
void dijkstra(Graph g, int source) {
    MinHeap pq;  // (distance, node)
    vector<int> dist(n, INF);

    dist[source] = 0;
    pq.insert({0, source});

    while (!pq.isEmpty()) {
        auto [d, u] = pq.extractMin();  // O(log n)

        for (auto [v, weight] : g.adj[u]) {
            if (dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                pq.insert({dist[v], v});
            }
        }
```

```
    }
}
```

### 3. Event-Driven Simulation:

```
class EventQueue {
    MinHeap events;  // Sorted by timestamp

public:
    void scheduleEvent(Event e) {
        events.insert(e);
    }

    Event getNextEvent() {
        Event e;
        events.extractMin(e);  // Earliest event
        return e;
    }
};

// Game engine, Network simulator
```

### 4. Top K Elements (Streaming Data):

```
// Find K largest numbers in a stream
class TopK {
    MinHeap heap;  // Size = K
    int k;

public:
    TopK(int k) : k(k) {}

    void add(int num) {
        if (heap.size() < k) {
            heap.insert(num);
        } else if (num > heap.getMin()) {
            heap.extractMin();
            heap.insert(num);
        }
    }

    vector<int> getTopK() {
```

```
        return heap.getAll();
    }
};

// YouTube trending videos, Twitter trending topics
```

**5. Median Maintenance:**

```
class MedianFinder {
    MaxHeap lowerHalf;  // Smaller half
    MinHeap upperHalf;  // Larger half

public:
    void addNumber(int num) {
        if (lowerHalf.isEmpty() || num <= lowerHalf.getMax()) {
            lowerHalf.insert(num);
        } else {
            upperHalf.insert(num);
        }

        // Balance sizes
        if (lowerHalf.size() > upperHalf.size() + 1) {
            int val;
            lowerHalf.extractMax(val);
            upperHalf.insert(val);
        } else if (upperHalf.size() > lowerHalf.size()) {
            int val;
            upperHalf.extractMin(val);
            lowerHalf.insert(val);
        }
    }

    double getMedian() {
        if (lowerHalf.size() == upperHalf.size()) {
            return (lowerHalf.getMax() + upperHalf.getMin()) / 2.0;
        } else {
            return lowerHalf.getMax();
        }
    }
};

// Find median in streaming data - O(log n) per add!
```

## 9.11. Tóm tắt

**Điểm quan trọng:**

1. **Heap:**
   - Complete Binary Tree
   - Max Heap: Parent ≥ Children, Root = max
   - Min Heap: Parent ≤ Children, Root = min
2. **Array Representation:**
   - Node i: left=2i+1, right=2i+2, parent=(i-1)/2
   - Tiết kiệm memory, cache-friendly
3. **Operations:**
   - Insert: Add at end → ReheapUp → O(log n)
   - Extract max: Move last to root → ReheapDown → O(log n)
   - Get max: O(1)
   - Build Heap: O(n) - Not O(n log n)!
4. **Heap Sort:**
   - O(n log n) worst-case guaranteed
   - In-place (không cần extra array)
   - Không stable
5. **Priority Queue:**
   - Best implementation: Heap
   - Insert/Delete: O(log n)
   - Get max/min: O(1)
6. **Ứng dụng:**
   - CPU Scheduling
   - Dijkstra, Prim algorithms
   - Top K problems
   - Median finding
   - Event simulation
7. **Heap vs BST:**
   - Heap: Tốt cho max/min, không tốt cho search
   - BST: Tốt cho search, OK cho max/min
   - Heap: Array-based, BST: Pointer-based
8. **Lỗi thường gặp:**
   - Sai công thức parent/child

- Không check bounds
- Heap Sort quên giảm heap size

9. **Build Heap: O(n) not O(n log n)**
   - Nhiều người nghĩ O(n log n)
   - Thực tế: O(n) với bottom-up approach
10. **C++ STL:**
    - `std::priority_queue` - Max heap default
    - `make_heap()`, `push_heap()`, `pop_heap()`

# 10. Bảng băm (Hash Table)

## 10.1. Tổng quan chương

### 10.1.1. Nội dung chính

Hash Table là cấu trúc dữ liệu cho phép search/insert/delete với **O(1) average case** - nhanh nhất có thể!

Nội dung:

1. Basic Concepts (Hash function, Collision)
2. Hash Functions (Modulo, Mid-square, Folding…)
3. Collision Resolution (Open Addressing, Chaining, Bucket)
4. Load Factor và Performance
5. Applications

### 10.1.2. Kiến thức nền

- Arrays
- Linked Lists
- Modulo arithmetic

## 10.2. Giải thích từng khái niệm

### 10.2.1. Hash Table và Hash Function

**WHAT - Hash Table**

**Hash Table** là cấu trúc dữ liệu sử dụng **hash function** để map keys vào array indices:

```
hash(key) → index
table[index] = value
```

**Thuật ngữ:**

- **Hash Function:** h(key) → address
- **Home Address:** Địa chỉ do hash function tạo ra
- **Prime Area:** Vùng nhớ chứa home addresses
- **Collision:** Khi 2 keys map tới cùng address
- **Synonyms:** Tập keys hash ra cùng location

**Ví dụ:**

```
Hash function: h(key) = key % 10

Keys:      Hash values:
23    →    3
45    →    5
13    →    3  ← Collision với 23!
78    →    8

Table: [_, _, _, 23, _, 45, _, _, 78, _]
       Index: 0  1  2   3  4   5  6  7   8  9
```

**Tại sao cần Hash Table?**

**Câu hỏi: Có thuật toán search O(1) không?**

**CÓ! Hash Table!**

So sánh với các cấu trúc khác:

| n elements | Sequential | Binary | Hash |
|---|---|---|---|
| 1,000 | 500 | 10 | **1** |
| 1,000,000 | 500,000 | 20 | **1** |

**Lợi ích:**

1. **Cực kỳ nhanh:** O(1) average case!
2. **Không phụ thuộc n:** 10 phần tử hay 1 triệu đều O(1)

3. **Đơn giản:** Dễ implement

**Ứng dụng:**

- Dictionary, Map, Set
- Database index
- Caching
- Symbol table trong compiler

### 10.2.2. Hash Functions

**WHAT - Hash Function**

Hash function h(key) biến đổi key thành array index.

**Yêu cầu:**

1. **Deterministic:** Cùng key → cùng hash value
2. **Fast:** Tính toán nhanh (O(1))
3. **Uniform distribution:** Phân bổ đều để giảm collision

**Các loại Hash Functions:**

1. **Direct Hashing:**

```
h(key) = key
```

   - Ưu: Không collision
   - Nhược: Lãng phí space (key space = address space)

2. **Modulo Division (phổ biến nhất):**

```
h(key) = key % m
```

   - m nên là số nguyên tố để giảm collision
   - Ví dụ: h(121267) = 121267 % 307 = 2

3. **Mid-Square:**

```
h(key) = middle digits of (key²)
```

   - Ví dụ: key = 9452
     - $9452^2 = 89340304$
     - Middle: 3403

4. **Digit Extraction:**

```
h(key) = selected digits from key
```

   - Ví dụ: 379452 → digits 1,2,4 → 394

5. **Folding:**

```
Chia key thành parts, cộng lại
```

   - Key = 123456789
   - Parts: 123 | 456 | 789
   - Fold shift: 123 + 456 + 789 = 1368 → 368

6. **Pseudo-Random:**

```
h(key) = (a × key + c) % m
```

- a, c là số nguyên tố

### 10.2.3. Collision (Đụng độ)

**WHAT - Collision**

**Collision** xảy ra khi 2 keys khác nhau hash ra cùng index:

```
h(key1) = h(key2) = i
table[i] đã có key1
→ Làm sao insert key2?
```

**Tại sao có collision?**

- Key space >> Address space
- Ví dụ: 1 triệu employee IDs → Hash table size 1000
- Theo Pigeonhole Principle: Chắc chắn có collisions!

**3 phương pháp giải quyết:**

1. **Open Addressing:** Tìm slot trống khác trong array
2. **Chaining:** Mỗi slot là một linked list
3. **Bucket Hashing:** Mỗi slot chứa nhiều phần tử

### 10.2.4. Open Addressing

**Open Addressing:** Khi collision, tìm slot trống tiếp theo theo một quy luật probe.

Probe function: `hp(key, i) = (h(key) + p(i)) % m`

**3 phương pháp probe:**

1. **Linear Probing:**

```
p(i) = i
hp(key, i) = (h(key) + i) % m
```

   - Thử index: h(key), h(key)+1, h(key)+2, …
   - Vấn đề: **Primary Clustering** (tạo cụm dài)

2. **Quadratic Probing:**

```
p(i) = i²
hp(key, i) = (h(key) + i²) % m
```

   - Thử: h(key), h(key)+1, h(key)+4, h(key)+9, …
   - Giảm clustering nhưng vẫn có **Secondary Clustering**

3. **Double Hashing:**

```
hp(key, i) = (h₁(key) + i × h₂(key)) % m
```

   - Dùng 2 hash functions
   - Ít clustering nhất

**Ví dụ Linear Probing:**

```
h(key) = key % 10
Insert: 23, 45, 13, 78, 33

23: h(23) = 3 → table[3] = 23
45: h(45) = 5 → table[5] = 45
13: h(13) = 3 → Collision! Try 4 → table[4] = 13
78: h(78) = 8 → table[8] = 78
33: h(33) = 3 → Collision! Try 4 (full), 5 (full), 6 → table[6] = 33

Table: [_, _, _, 23, 13, 45, 33, _, 78, _]
```

### 10.2.5. Chaining (Separate Chaining)

**Chaining:** Mỗi slot trong hash table là một linked list.

Collision → Thêm vào linked list tại slot đó.

```
Table: array of linked lists

h(key) = key % 5

Keys: 23, 45, 13, 78, 33, 18

0: NULL
1: NULL
2: NULL
3: 23 → 13 → 33 → 18 → NULL
4: 78 → NULL
```

**Ưu/Nhược:**

- Ưu: Đơn giản, không bao giờ "full"
- Nhược: Tốn memory cho pointers, cache-unfriendly

### 10.2.6. Load Factor

**Load Factor (α):** Tỷ lệ lấp đầy hash table

```
α = n / m
```

n = số elements, m = table size

**Ảnh hưởng đến performance:**

| Load Factor | Open Addressing | Chaining |
|---|---|---|
| $\alpha < 0.5$ | Ít collision, nhanh | List ngắn, nhanh |
| $\alpha \approx 0.7$ | OK | Tốt |
| $\alpha > 0.9$ | Nhiều collision, chậm | List dài, chậm hơn |
| $\alpha > 1.0$ | Không thể (full table) | OK nhưng chậm |

**Khi nào resize?**

- Thường resize khi $\alpha > 0.75$
- Tạo table mới (2× size)
- Rehash tất cả elements → O(n)

## 10.3. Bản chất trong máy tính

### 10.3.1. Memory Layout

**Hash Table with Chaining:**

```
Table Array (Stack/Data segment):
[0]: 0x2000 → [23|0x2100] → [13|NULL]
[1]: NULL
[2]: 0x3000 → [45|NULL]
[3]: 0x4000 → [78|0x4100] → [33|NULL]

Linked List Nodes (Heap):
0x2000, 0x2100: Chain for slot 0
0x3000: Chain for slot 2
0x4000, 0x4100: Chain for slot 3
```

**Hash Table with Open Addressing:**

```
Simple array:
[0]: empty
[1]: empty
[2]: 45
[3]: 23
[4]: 13  ← Probed from slot 3
[5]: 78
[6]: 33  ← Probed from slot 3
```

→ Open Addressing tiết kiệm memory hơn (no pointers)!

### 10.3.2. Cache Performance

**Chaining:** Cache-unfriendly (pointers jump around memory)

**Open Addressing:** Cache-friendly (sequential probing trong array)

→ Trong thực tế, Open Addressing thường nhanh hơn với $\alpha < 0.7$!

## 10.4. Lịch sử / Nguồn gốc

**1953: Hash Table**

- Hans Peter Luhn (IBM) phát minh hashing
- Mục đích: Internal memory lookup

**1954: Open Addressing**

- A.D. Linh
- Linear probing

**1963: Chaining**

- Arnold Dumey
- Separate chaining with linked lists

**1970s: Advanced Hash Functions**

- Universal hashing
- Perfect hashing
- Cryptographic hash (MD5, SHA)

**Ngày nay:**

- Hash tables everywhere!
- C++ `unordered_map`, `unordered_set`
- Java `HashMap`, `HashSet`
- Python `dict`, `set`
- JavaScript objects

**Modern applications:**

- Blockchain (SHA-256 hashing)
- Password storage (bcrypt, scrypt)
- Data deduplication
- Distributed systems (Consistent hashing)

## 10.5. Phân tích thuật toán

### 10.5.1. Hash Function

**Complexity:**

- Time: O(1) - Constant time calculation
- Space: O(1)

### 10.5.2. Search/Insert/Delete

**With Chaining:**

**Time Complexity:**

- Best: O(1) - no collision
- Average: $O(1 + \alpha) \approx O(1)$ if $\alpha$ small
- Worst: O(n) - all keys in same slot

**Space Complexity:** O(n + m) - n elements + m slots

**With Open Addressing:**

**Time Complexity:**

- Best: O(1)
- Average: $O(1/(1-\alpha))$ if $\alpha < 0.7$
- Worst: O(n) - full table

**Space Complexity:** O(m) - fixed array size

**Phân tích chi tiết:**

Với $\alpha = 0.5$:

- Chaining: Average chain length = 0.5 → 1-2 comparisons
- Linear Probing: 1.5 probes
- Quadratic/Double Hashing: 1.4 probes

Với $\alpha = 0.9$:

- Chaining: 1-2 comparisons (still OK)
- Linear Probing: 5-10 probes (degraded!)

## 10.6. Minh họa từng bước

### 10.6.1. Chaining Example

```
h(key) = key % 7
Insert: 23, 45, 13, 78, 33, 18, 51

Step 1: Insert 23
  h(23) = 2
  table[2] = [23] → NULL

Step 2: Insert 45
  h(45) = 3
  table[3] = [45] → NULL

Step 3: Insert 13
  h(13) = 6
  table[6] = [13] → NULL

Step 4: Insert 78
  h(78) = 1
  table[1] = [78] → NULL

Step 5: Insert 33
  h(33) = 5
  table[5] = [33] → NULL

Step 6: Insert 18
  h(18) = 4
  table[4] = [18] → NULL

Step 7: Insert 51
  h(51) = 2  ← Collision với 23!
  table[2] = [23] → [51] → NULL

Final:
[0]: NULL
[1]: [78] → NULL
[2]: [23] → [51] → NULL
[3]: [45] → NULL
[4]: [18] → NULL
[5]: [33] → NULL
[6]: [13] → NULL
```

### 10.6.2. Linear Probing Example

```
Same keys, same h(key) = key % 7
Insert: 23, 45, 13, 78, 33, 18, 51

Insert 23: h(23) = 2 → table[2] = 23
Insert 45: h(45) = 3 → table[3] = 45
Insert 13: h(13) = 6 → table[6] = 13
Insert 78: h(78) = 1 → table[1] = 78
Insert 33: h(33) = 5 → table[5] = 33
Insert 18: h(18) = 4 → table[4] = 18

Insert 51: h(51) = 2 → Collision!
  Try (2+0)%7 = 2 → Full (23)
  Try (2+1)%7 = 3 → Full (45)
  Try (2+2)%7 = 4 → Full (18)
  Try (2+3)%7 = 5 → Full (33)
  Try (2+4)%7 = 6 → Full (13)
  Try (2+5)%7 = 0 → Empty! → table[0] = 51

Final:
[51, 78, 23, 45, 18, 33, 13]
  0   1   2   3   4   5   6
```

## 10.7. Code minh họa C++

### 10.7.1. Hash Table with Chaining

```
#include <iostream>
#include <vector>
#include <list>
using namespace std;

class HashTableChaining {
private:
    vector<list<int>> table;
    int tableSize;
    int numElements;

    int hashFunction(int key) {
        return key % tableSize;
    }

public:
    HashTableChaining(int size = 10) : tableSize(size), numElements(0) {
        table.resize(tableSize);
    }

    void insert(int key) {
        int index = hashFunction(key);

        // Check if already exists
        for (int k : table[index]) {
            if (k == key) return;  // Duplicate
        }

        table[index].push_back(key);
        numElements++;
    }

    bool search(int key) {
        int index = hashFunction(key);

        for (int k : table[index]) {
            if (k == key) return true;
        }
```

```
        return false;
    }

    bool remove(int key) {
        int index = hashFunction(key);

        for (auto it = table[index].begin(); it != table[index].end(); ++it) {
            if (*it == key) {
                table[index].erase(it);
                numElements--;
                return true;
            }
        }

        return false;
    }

    double loadFactor() {
        return (double)numElements / tableSize;
    }

    void print() {
        for (int i = 0; i < tableSize; i++) {
            cout << "[" << i << "]: ";
            for (int key : table[i]) {
                cout << key << " → ";
            }
            cout << "NULL" << endl;
        }
    }
};

int main() {
    HashTableChaining ht(7);

    ht.insert(23);
    ht.insert(45);
    ht.insert(13);
    ht.insert(78);
    ht.insert(33);
    ht.insert(18);
    ht.insert(51);  // Collision with 23
```

```
    ht.print();

    cout << "\nSearch 51: " << (ht.search(51) ? "Found" : "Not found") << endl;
    cout << "Search 100: " << (ht.search(100) ? "Found" : "Not found") << endl;

    cout << "Load Factor: " << ht.loadFactor() << endl;

    ht.remove(51);
    cout << "\nAfter removing 51:" << endl;
    ht.print();

    return 0;
}
```

### 10.7.2. Hash Table with Linear Probing

```
class HashTableLinearProbing {
private:
    vector<int> table;
    vector<bool> occupied;
    int tableSize;
    int numElements;

    int hashFunction(int key) {
        return key % tableSize;
    }

    int probe(int key, int i) {
        return (hashFunction(key) + i) % tableSize;
    }

public:
    HashTableLinearProbing(int size = 10) : tableSize(size), numElements(0) {
        table.resize(tableSize);
        occupied.resize(tableSize, false);
    }

    bool insert(int key) {
        if (numElements >= tableSize) {
            cout << "Table full!" << endl;
            return false;
```

```
        }

        int i = 0;
        while (i < tableSize) {
            int index = probe(key, i);

            if (!occupied[index]) {
                table[index] = key;
                occupied[index] = true;
                numElements++;
                return true;
            }

            if (table[index] == key) {
                return false;  // Duplicate
            }

            i++;  // Probe next
        }

        return false;  // Should not reach here
    }

    bool search(int key) {
        int i = 0;
        while (i < tableSize) {
            int index = probe(key, i);

            if (!occupied[index]) {
                return false;  // Empty slot → not found
            }

            if (table[index] == key) {
                return true;  // Found
            }

            i++;  // Probe next
        }

        return false;
    }
```

```
    void print() {
        for (int i = 0; i < tableSize; i++) {
            if (occupied[i]) {
                cout << "[" << i << "]: " << table[i] << endl;
            } else {
                cout << "[" << i << "]: empty" << endl;
            }
        }
    }

    double loadFactor() {
        return (double)numElements / tableSize;
    }
};
```

## 10.8. Lỗi phổ biến

### 1. Hash function không uniform

```
// ❌ SAI: Collision nhiều
int hash(int key) {
    return key % 100;  // Nếu keys đều chẵn → Chỉ dùng 50 slots!
}

// ✅ TốT HƠN: Dùng số nguyên tố
int hash(int key) {
    return key % 101;  // Prime number
}
```

### 2. Không handle collision

```
// ❌ SAI: Overwrite data khi collision
void insert(int key) {
    int index = hash(key);
    table[index] = key;  // Nếu collision → mất data cũ!
}

// ✅ ĐÚNG: Resolve collision
```

### 3. Linear Probing - Quên modulo

```
// ❌ SAI: Index out of bounds
int probe(int key, int i) {
    return hash(key) + i;  // Có thể > tableSize!
}

// ✅ ĐÚNG
int probe(int key, int i) {
    return (hash(key) + i) % tableSize;
}
```

## 10.9. Khi nào dùng / không dùng

### 10.9.1. Khi nào dùng Hash Table?

- Cần search/insert/delete **cực nhanh** O(1)
- Không cần maintain sorted order
- Không cần range queries
- Key space không quá lớn

**Ví dụ:**

- Dictionary: word → definition
- Cache: URL → cached page
- Database: Primary key index
- Set operations (membership test)

### 10.9.2. Khi nào KHÔNG dùng Hash Table?

| Tình huống | Dùng gì thay thế |
|---|---|
| Cần sorted order | BST, AVL Tree |
| Cần range queries | BST (find all x in [a,b]) |
| Cần min/max frequently | Heap |
| Memory critical | BST (no wasted slots) |
| Worst-case guarantee | AVL Tree O(log n) guaranteed |

## 10.10. Ứng dụng thực tế

**1. Database Indexing:**

```
// Hash index cho primary key
HashTable<int, Record> employeeTable;

void insertEmployee(int id, Record rec) {
    employeeTable.insert(id, rec);  // O(1)
}

Record getEmployee(int id) {
    return employeeTable.search(id);  // O(1)!
}
```

MySQL, PostgreSQL dùng hash index cho equality searches (không dùng cho range).

**2. Caching (Redis, Memcached):**

```
class Cache {
    HashTable<string, string> cache;

public:
    string get(string url) {
        if (cache.contains(url)) {
            return cache.get(url);  // O(1) - Cache hit!
        } else {
            string content = fetchFromNetwork(url);
            cache.insert(url, content);
            return content;
        }
    }
};
```

**3. Compiler - Symbol Table:**

```
// Lưu biến, hàm trong compiler
HashTable<string, SymbolInfo> symbolTable;

void declareVariable(string name, Type type) {
    if (symbolTable.contains(name)) {
        error("Variable already declared!");
    }
    symbolTable.insert(name, {type, address});
```

```
}

Type getVariableType(string name) {
    return symbolTable.get(name).type;  // O(1)
}
```

### 4. Web - Session Management:

```
HashTable<string, Session> sessions;

void createSession(User user) {
    string sessionID = generateRandomID();
    sessions.insert(sessionID, {user, timestamp});
    setCookie("sessionID", sessionID);
}

User getUser(string sessionID) {
    if (sessions.contains(sessionID)) {
        return sessions.get(sessionID).user;  // O(1)
    }
    return null;  // Not logged in
}
```

### 5. Blockchain - Transaction Verification:

```
// Bitcoin transaction pool
HashTable<string, Transaction> mempool;

bool isTransactionValid(string txHash) {
    // Check if transaction already in blockchain
    return mempool.contains(txHash);  // O(1)
}
```

## 10.11. Tóm tắt

**Điểm quan trọng:**

1. **Hash Table = O(1) search/insert/delete!**
   - Nhanh nhất có thể
   - Dùng hash function: key → index
2. **Hash Functions:**
   - Modulo Division (phổ biến): h(k) = k % m, m là prime
   - Mid-Square, Folding, Pseudo-random
   - Yêu cầu: Fast, Uniform distribution
3. **Collision Resolution:**
   - **Chaining:** Linked list tại mỗi slot (đơn giản, no "full")
   - **Open Addressing:** Linear/Quadratic/Double hashing probing
   - **Bucket:** Mỗi slot chứa nhiều elements
4. **Load Factor α = n/m:**
   - α < 0.7: Good performance
   - α > 0.9: Degrade → Cần resize
5. **Complexity:**
   - Average: O(1) - Amazing!
   - Worst: O(n) - Khi all collision
   - Với good hash function + α < 0.7 → Gần O(1)
6. **Trade-offs:**
   - Chaining: Flexible, extra memory for pointers
   - Open Addressing: Cache-friendly, có thể "full"
7. **Hash vs BST vs Heap:**
   - Hash: Fast search, no order
   - BST: Moderate search, sorted order, range queries
   - Heap: Fast max/min, no search
8. **Lỗi thường gặp:**
   - Bad hash function → Nhiều collision
   - Không handle collision
   - Load factor quá cao
9. **Ứng dụng:**
   - Database index
   - Caching systems

- Compiler symbol tables
- Session management
- Blockchain

10. **C++ STL:**
    - `std::unordered_map<K, V>` - Hash table
    - `std::unordered_set<T>`
    - `std::map` - BST (Red-Black Tree)

# 11. Thuật toán Sắp xếp (Sorting Algorithms)

## 11.1. Tổng quan chương

### 11.1.1. Nội dung chính

Sorting là một trong những bài toán cơ bản và quan trọng nhất trong khoa học máy tính. Chương này sẽ phân tích các thuật toán sorting từ đơn giản đến phức tạp:

1. **Sorting Concepts:**
   - Sort Stability
   - Sort Efficiency
   - Comparison-based vs Non-comparison
2. **Simple Sorts $O(n^2)$:**
   - Bubble Sort
   - Selection Sort
   - Insertion Sort
3. **Advanced Sorts O(n log n):**
   - Shell Sort
   - Merge Sort
   - Quick Sort
   - Heap Sort
4. **So sánh các thuật toán**

### 11.1.2. Kiến thức nền

- Arrays
- Recursion (cho Merge Sort, Quick Sort)
- Heap (cho Heap Sort)

## 11.2. Giải thích từng khái niệm

### 11.2.1. Sorting (Sắp xếp)

**WHAT - Sorting**

**Sorting** là quá trình sắp xếp lại các phần tử trong danh sách theo một thứ tự nhất định (tăng dần hoặc giảm dần).

Input: `[5, 2, 8, 1, 9]`

Output: `[1, 2, 5, 8, 9]` (tăng dần)

**Các khái niệm quan trọng:**

1. **Sort Stability:**

   **Stable Sort**

   Các phần tử có **key bằng nhau** giữ nguyên thứ tự tương đối sau khi sort.

   Input: `[(5,a), (3,b), (5,c), (1,d)]`

   Stable: `[(1,d), (3,b), (5,a), (5,c)]` ← a vẫn trước c

   Unstable: `[(1,d), (3,b), (5,c), (5,a)]` ← c trước a (sai!)

   **Stable sorts:** Merge, Insertion, Bubble

   **Unstable sorts:** Quick, Heap, Selection

2. **In-place Sorting:**
   - Không cần extra array
   - Space complexity: O(1)
   - Ví dụ: Bubble, Selection, Insertion, Quick, Heap

3. **Sort Efficiency:**

   ```
   Efficiency = Số comparisons + Số moves (swaps/shifts)
   ```

**Tại sao cần Sorting?**

1. **Tiền xử lý cho search:**
   - Binary search yêu cầu sorted array → O(log n)
2. **Data organization:**
   - Hiển thị data theo thứ tự (tên, điểm, giá…)
3. **Algorithm requirements:**
   - Nhiều thuật toán cần sorted data (merge, median finding…)
4. **Everywhere in real life:**
   - E-commerce: Sort products by price
   - Social media: Sort posts by time

- Database: ORDER BY clauses
- File manager: Sort files by name/date/size

### 11.2.2. Bubble Sort (Sắp xếp nổi bọt)

**WHAT**

**Bubble Sort:** So sánh 2 phần tử liền kề, swap nếu sai thứ tự. Phần tử lớn "nổi" lên cuối sau mỗi pass.

**Algorithm:**

```
for i = 0 to n-2:
    for j = 0 to n-2-i:
        if arr[j] > arr[j+1]:
            swap(arr[j], arr[j+1])
```

**Ví dụ:** `[5, 2, 8, 1]`

```
Pass 1: [2, 5, 1, 8]  (8 nổi lên cuối)
Pass 2: [2, 1, 5, 8]  (5 đúng vị trí)
Pass 3: [1, 2, 5, 8]  (2 đúng vị trí)
Done!
```

**Time Complexity:** Best: O(n) - đã sorted, Average: $O(n^2)$, Worst: $O(n^2)$ - reverse sorted

**Space Complexity:** O(1) - In-place

**Stable:** ✓ Yes

### 11.2.3. Selection Sort

**Selection Sort:** Mỗi lần tìm phần tử nhỏ nhất trong phần chưa sort, swap với vị trí đầu.

```
for i = 0 to n-2:
    minIndex = i
    for j = i+1 to n-1:
        if arr[j] < arr[minIndex]:
            minIndex = j
    swap(arr[i], arr[minIndex])
```

**Time Complexity:** $O(n^2)$ always - Best/Avg/Worst đều như nhau

**Space Complexity:** O(1)

**Stable:** ✗ No

### 11.2.4. Insertion Sort

**Insertion Sort:** Chia array thành sorted và unsorted. Insert từng phần tử từ unsorted vào đúng vị trí trong sorted.

```
for i = 1 to n-1:
    key = arr[i]
    j = i - 1
```

```
    while j >= 0 and arr[j] > key:
        arr[j+1] = arr[j]
        j--
    arr[j+1] = key
```

**Complexity:**

- Time: Best: O(n) - nearly sorted, Average: $O(n^2)$, Worst: $O(n^2)$ - reverse sorted
- Space: O(1)

**Stable:** ✓ Yes

**Khi nào dùng:** Small arrays (n < 50), nearly sorted data

### 11.2.5. Merge Sort

**Merge Sort:** Divide & Conquer

- Chia array thành 2 nửa
- Sort đệ quy từng nửa
- Merge 2 nửa đã sorted

```
Algorithm mergeSort(arr, left, right):
    if left < right:
        mid = (left + right) / 2
        mergeSort(arr, left, mid)
        mergeSort(arr, mid+1, right)
        merge(arr, left, mid, right)
```

**Time Complexity:** O(n log n) - Tất cả trường hợp!

**Space Complexity:** O(n) - Cần extra array cho merge

**Stable:** ✓ Yes

**Recurrence:** T(n) = 2T(n/2) + O(n) = O(n log n)

### 11.2.6. Quick Sort

**Quick Sort:** Divide & Conquer

- Chọn pivot
- Partition: Đưa nhỏ hơn pivot về trái, lớn hơn về phải
- Đệ quy sort 2 phần

```
Algorithm quickSort(arr, low, high):
    if low < high:
        pivotIndex = partition(arr, low, high)
        quickSort(arr, low, pivotIndex - 1)
        quickSort(arr, pivotIndex + 1, high)
```

**Time Complexity:** Best: O(n log n) - pivot chia đều, Average: O(n log n), Worst: O($n^2$) - pivot luôn min/max

**Space Complexity:** O(log n) - Stack cho recursion

**Stable:** ✗ No (có thể làm stable nhưng phức tạp)

**Trong thực tế:** Nhanh nhất! (cache-friendly, in-place)

### 11.2.7. Heap Sort

Đã học ở Chapter 8!

**Time Complexity:** O(n log n) - Guaranteed!

**Space Complexity:** O(1) - In-place

**Stable:** ✗ No

## 11.3. Bản chất trong máy tính

### 11.3.1. Tại sao Quick Sort nhanh trong thực tế?

Dù cùng O(n log n), Quick Sort thường nhanh hơn Merge Sort:

1. **In-place:** Không cần extra array → Ít memory access
2. **Cache-friendly:** Sequential access trong partition
3. **Small constant factor:** Ít overhead

**Benchmark (n = 1,000,000):**

- Quick Sort: 0.5 giây
- Merge Sort: 0.8 giây
- Heap Sort: 1.2 giây

Tuy nhiên, worst-case $O(n^2)$ của Quick Sort đôi khi là vấn đề!

### 11.3.2. Memory Access Patterns

**Merge Sort:**

```
Merge step: Read from 2 arrays → Write to 1 array
→ Many memory writes (chậm)
```

**Quick Sort:**

```
Partition: Swap in-place
→ Fewer memory operations (nhanh)
```

## 11.4. Lịch sử / Nguồn gốc

**1945: Merge Sort**

- John von Neumann
- First O(n log n) algorithm

**1959: Quick Sort**

- Tony Hoare
- Revolutionary! Still most-used today

**1959: Shell Sort**

- Donald Shell
- Improvement over Insertion Sort

**1964: Heap Sort**

- J.W.J. Williams

**Ngày nay:**

- C++ `std::sort`: Introsort (Quick + Heap + Insertion)
- Java `Arrays.sort`: Dual-Pivot Quick Sort
- Python `sorted()`: Timsort (Merge + Insertion)

## 11.5. Phân tích thuật toán

### 11.5.1. Bảng tổng hợp

| Algorithm | Best | Average | Worst | Space | Stable |
|---|---|---|---|---|---|
| Bubble | O(n) | $O(n^2)$ | $O(n^2)$ | O(1) | ✓ |
| Selection | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ | O(1) | ✗ |
| Insertion | O(n) | $O(n^2)$ | $O(n^2)$ | O(1) | ✓ |
| Shell | O(n log n) | O(n^1.25) | $O(n^2)$ | O(1) | ✗ |
| Merge | O(n log n) | O(n log n) | O(n log n) | O(n) | ✓ |
| Quick | O(n log n) | O(n log n) | $O(n^2)$ | O(log n) | ✗ |
| Heap | O(n log n) | O(n log n) | O(n log n) | O(1) | ✗ |

## 11.6. Minh họa từng bước

### 11.6.1. Quick Sort với Partition

Array: `[5, 2, 8, 1, 9]`, pivot = 5

```
Initial: [5, 2, 8, 1, 9]
         ↑ pivot

Partition:
  i = 0, j = 4
  [5, 2, 8, 1, 9]
   ↑           ↑

  2 < 5 → i++
  [5, 2, 8, 1, 9]
      ↑        ↑

  8 > 5, 9 > 5, 1 < 5
  Swap 8 and 1
  [5, 2, 1, 8, 9]
         ↑  ↑

  Final partition:
  [1, 2] 5 [8, 9]
   left    right

Recursively sort:
  Left: [1, 2] → [1, 2]
  Right: [8, 9] → [8, 9]

Result: [1, 2, 5, 8, 9]
```

## 11.7. Code minh họa C++

### 11.7.1. Bubble Sort

```
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;

        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }

        // Optimization: Stop if no swap (already sorted)
        if (!swapped) break;
    }
}
```

### 11.7.2. Selection Sort

```
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;

        // Find minimum in unsorted part
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }

        // Swap with first unsorted element
        swap(arr[i], arr[minIndex]);
    }
}
```

### 11.7.3. Insertion Sort

```
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
```

```
        // Shift elements > key to right
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }

        // Insert key
        arr[j + 1] = key;
    }
}
```

### 11.7.4. Merge Sort

```
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Create temp arrays
    int* L = new int[n1];
    int* R = new int[n2];

    // Copy data
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    // Merge back
    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k++] = L[i++];
        } else {
            arr[k++] = R[j++];
        }
    }

    // Copy remaining
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
```

```
    delete[] L;
    delete[] R;
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

### 11.7.5. Quick Sort

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high];  // Choose last as pivot
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

### 11.7.6. Demo tất cả algorithms

```
#include <iostream>
#include <chrono>
using namespace std;
using namespace chrono;

void testSort(void (*sortFunc)(int[], int), string name, int arr[], int n) {
    int* copy = new int[n];
    for (int i = 0; i < n; i++) copy[i] = arr[i];

    auto start = high_resolution_clock::now();
    sortFunc(copy, n);
    auto end = high_resolution_clock::now();

    auto duration = duration_cast<microseconds>(end - start);
    cout << name << ": " << duration.count() << " µs" << endl;

    delete[] copy;
}

int main() {
    int n = 10000;
    int* arr = new int[n];

    // Random array
    srand(time(0));
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 10000;
    }

    cout << "Sorting " << n << " elements:\n" << endl;

    testSort(bubbleSort, "Bubble Sort", arr, n);
    testSort(selectionSort, "Selection Sort", arr, n);
    testSort(insertionSort, "Insertion Sort", arr, n);
    testSort(mergeSort, "Merge Sort", arr, n);
    testSort(quickSort, "Quick Sort", arr, n);
    testSort(heapSort, "Heap Sort", arr, n);

    delete[] arr;
    return 0;
}
```

**Output (ước tính):**

```
Sorting 10000 elements:

Bubble Sort: 250000 µs (~250 ms)
Selection Sort: 150000 µs (~150 ms)
Insertion Sort: 100000 µs (~100 ms)
Merge Sort: 2000 µs (~2 ms)
Quick Sort: 1500 µs (~1.5 ms)
Heap Sort: 3000 µs (~3 ms)
```

## 11.8. Lỗi phổ biến

### 1. Bubble Sort - Quên optimization

```
// ❌ Chậm: Luôn chạy n² comparisons
for (int i = 0; i < n-1; i++) {
    for (int j = 0; j < n-1-i; j++) {
        if (arr[j] > arr[j+1]) swap(arr[j], arr[j+1]);
    }
}

// ✅ TỐT HƠN: Dừng sớm nếu đã sorted
for (int i = 0; i < n-1; i++) {
    bool swapped = false;
    for (int j = 0; j < n-1-i; j++) {
        if (arr[j] > arr[j+1]) {
            swap(arr[j], arr[j+1]);
            swapped = true;
        }
    }
    if (!swapped) break;  // Đã sorted!
}
```

### 2. Merge Sort - Tràn stack với n lớn

```
// ❌ VẤN ĐỀ: Đệ quy sâu với n rất lớn
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);        // Đệ quy
        mergeSort(arr, mid + 1, right);   // Đệ quy
        merge(arr, left, mid, right);
    }
}

// ✅ GIẢI PHÁP: Dùng iterative merge sort hoặc tail recursion
```

### 3. Quick Sort - Chọn pivot tệ

```
// ❌ TỆ: Luôn chọn first/last với sorted array
int pivot = arr[high];  // Nếu array đã sorted → O(n²)!

// ✅ TỐT HƠN: Median-of-three
```

```
int mid = low + (high - low) / 2;
int pivot = median(arr[low], arr[mid], arr[high]);
```

## 11.9. Khi nào dùng / không dùng

### 11.9.1. Chọn thuật toán theo tình huống

| Tình huống | Thuật toán |
|---|---|
| n < 50 (small) | Insertion Sort - Đơn giản, nhanh với small n |
| Nearly sorted | Insertion Sort - O(n) best case |
| Cần stable | Merge Sort |
| Cần in-place + O(n log n) | Heap Sort |
| General purpose | Quick Sort - Fastest in practice |
| Worst-case guarantee | Merge hoặc Heap - O(n log n) đảm bảo |
| Linked List | Merge Sort - Không cần random access |

### 11.9.2. Hybrid Approaches (Thực tế)

**Introsort (C++ `std::sort`):**

```
1. Bắt đầu với Quick Sort
2. Nếu recursion depth > 2 log n → Chuyển sang Heap Sort
3. Nếu subarray < 16 → Chuyển sang Insertion Sort

→ Best of all worlds!
```

**Timsort (Python):**

```
Merge Sort + Insertion Sort
→ O(n) best case, O(n log n) worst
→ Stable!
```

## 11.10. Ứng dụng thực tế

**1. E-commerce - Product Listing:**

```
vector<Product> products = getProducts();

// Sort by price
sort(products.begin(), products.end(),
     [](Product a, Product b) { return a.price < b.price; });

// Sort by rating (stable - giữ price order nếu rating bằng nhau)
stable_sort(products.begin(), products.end(),
            [](Product a, Product b) { return a.rating > b.rating; });
```

- Quick Sort cho general sorting
- Stable sort khi cần maintain order

**2. Database - ORDER BY:**

```
SELECT * FROM employees ORDER BY salary DESC;
```

Internal:

- Small result: Insertion Sort
- Large result: External Merge Sort (data > RAM)
- Index available: Index scan (already sorted)

**3. Operating System - Process Scheduling:**

```
// Sort processes by priority
vector<Process> processes;
sort(processes.begin(), processes.end(),
     [](Process a, Process b) { return a.priority > b.priority; });
```

- Quick Sort cho general
- Heap cho priority queue

**4. Search Engines - Ranking Results:**

```
vector<Page> searchResults = search("DSA tutorial");

// Sort by relevance score
sort(searchResults.begin(), searchResults.end(),
     [](Page a, Page b) { return a.relevance > b.relevance; });
```

Google PageRank kết hợp nhiều factors → Quick Sort

## 11.11. Tóm tắt

**Điểm quan trọng:**

1. **Simple Sorts O(n²):**
   - Bubble, Selection, Insertion
   - Dùng cho small arrays (n < 50)
2. **Advanced Sorts O(n log n):**
   - Merge: Stable, O(n) space
   - Quick: Fastest practice, O(n²) worst
   - Heap: O(n log n) guaranteed, in-place
3. **Best practices:**
   - n < 50: Insertion Sort
   - General: Quick Sort (C++ `std::sort`)
   - Need stable: Merge Sort
   - Need guarantee: Heap Sort
4. **Trade-offs:**
   - Time vs Space: Merge O(n) space
   - Stability vs Speed: Stable sorts usually slower
   - Best-case vs Worst-case
5. **Real-world:** Hybrid algorithms (Introsort, Timsort)
6. **Lỗi thường gặp:**
   - Bubble: Không optimize với swapped flag
   - Quick: Bad pivot choice
   - Merge: Extra space overhead
7. **Applications everywhere:**
   - Databases, Search engines, OS, E-commerce
8. **Remember:**
   - No "best" algorithm for all cases
   - Choose based on: n size, stability, space, data characteristics

# 12. Đồ thị (Graph)

## 12.1. Tổng quan chương

### 12.1.1. Nội dung chính

Graph là cấu trúc dữ liệu mạnh mẽ nhất, có thể biểu diễn hầu hết mọi quan hệ trong thế giới thực.

Nội dung:

1. Graph Definition (Vertices, Edges)
2. Directed vs Undirected, Weighted vs Unweighted
3. Graph Representations (Adjacency Matrix, Adjacency List)
4. Graph Traversals: DFS, BFS
5. Shortest Path: Dijkstra's Algorithm
6. Minimum Spanning Tree: Prim's, Kruskal's Algorithm
7. Applications

### 12.1.2. Kiến thức nền

- Trees (Graph là tổng quát hóa của Tree)
- Queue (cho BFS)
- Stack/Recursion (cho DFS)
- Priority Queue (cho Dijkstra, Prim)

## 12.2. Giải thích từng khái niệm

### 12.2.1. Graph (Đồ thị)

**WHAT - Graph**

**Graph G = (V, E)** bao gồm:

- **V:** Tập hợp các vertices (đỉnh)
- **E:** Tập hợp các edges (cạnh) nối các vertices

E ⊆ V × V (Tập con của tích Descartes)

**Ví dụ:**

```
Graph G:
V = {A, B, C, D}
E = {(A,B), (A,C), (B,D), (C,D)}

Visualization:
    A ─── B
    │     │
    C ─── D
```

**Các loại Graph:**

1. **Directed Graph (Digraph):**
   - Edges có hướng: A → B ≠ B → A

   ```
   A → B
   ↑   ↓
   C ← D
   ```

2. **Undirected Graph:**
   - Edges không hướng: A─B = B─A

   ```
   A — B
   |   |
   C — D
   ```

3. **Weighted Graph:**
   - Mỗi edge có weight (trọng số)

   ```
   A —5— B
   |3    |2
   C —4— D
   ```

4. **Unweighted Graph:**

- Edges không có weight (hoặc weight = 1)

**Thuật ngữ:**

- **Adjacent:** Hai vertices có edge nối
- **Degree:** Số edges của một vertex
- **Path:** Chuỗi vertices từ u đến v
- **Cycle:** Path bắt đầu và kết thúc ở cùng vertex
- **Connected:** Có path giữa mọi cặp vertices
- **Strongly Connected (Digraph):** Có directed path giữa mọi cặp

**Tại sao cần Graph?**

Graph là cấu trúc **universal** - Biểu diễn được hầu hết mọi thứ:

1. **Social Networks:**
   - Vertices = Users
   - Edges = Friendship
2. **Maps/GPS:**
   - Vertices = Cities/Intersections
   - Edges = Roads
   - Weights = Distance/Time
3. **Web:**
   - Vertices = Webpages
   - Edges = Hyperlinks
   - Google PageRank dùng graph!
4. **Computer Networks:**
   - Vertices = Routers/Computers
   - Edges = Connections
   - Routing algorithms
5. **Dependencies:**
   - Vertices = Tasks/Packages
   - Edges = Dependencies
   - Topological sort

### 12.2.2. Graph Representations

**WHAT - Cách lưu Graph**

2 cách chính lưu Graph trong máy tính:

1. **Adjacency Matrix**
2. **Adjacency List**

**1. Adjacency Matrix (Ma trận kề):**

Array 2D: `matrix[i][j] = 1` nếu có edge từ i → j

```
Graph:          Matrix:
  0→1             0  1  2  3
  ↓ ↓           ┌──────────
  3→2         0 │ 0  1  0  1
              1 │ 0  0  1  0
              2 │ 0  0  0  0
              3 │ 0  0  1  0

matrix[0][1] = 1 (edge 0→1)
matrix[0][3] = 1 (edge 0→3)
```

**Ưu/Nhược:**

- ✓ Check edge: O(1)
- ✓ Simple implementation
- ✗ Space: $O(V^2)$ - Lãng phí với sparse graph
- ✗ Get neighbors: O(V)

**2. Adjacency List (Danh sách kề):**

Mỗi vertex có list các neighbors:

```
Graph:          List:
  0→1           0: [1, 3]
  ↓ ↓           1: [2]
  3→2           2: []
                3: [2]

vector<vector<int>> adj(V);
adj[0] = {1, 3};
adj[1] = {2};
adj[3] = {2};
```

**Ưu/Nhược:**

- ✓ Space: O(V + E) - Efficient với sparse graph
- ✓ Get neighbors: O(degree(v))
- ✗ Check edge: O(degree(v))

### 12.2.3. Depth-First Search (DFS)

**WHAT - DFS**

**DFS** duyệt graph bằng cách đi sâu nhất có thể trước khi backtrack.

Dùng **Stack** (hoặc Recursion).

**Algorithm:**

```
Algorithm DFS(graph, start):
    visited[start] = true
    process(start)

    for each neighbor of start:
        if not visited[neighbor]:
            DFS(graph, neighbor)
```

**Ví dụ:**

```
Graph:
    A — B
    |   |
    C — D

DFS from A:
Visit A → Visit B → Visit D → Visit C

Order: A, B, D, C
```

**Complexity:**

- Time: O(V + E) - Visit mỗi vertex và edge một lần
- Space: O(V) - Stack/Visited array

**Ứng dụng DFS:**

- Tìm path giữa 2 vertices
- Detect cycle
- Topological sort
- Connected components

### 12.2.4. Breadth-First Search (BFS)

**WHAT - BFS**

**BFS** duyệt graph theo level: Visit tất cả neighbors trước khi đi sâu hơn.

Dùng **Queue**.

**Algorithm:**

```
Algorithm BFS(graph, start):
    queue = createQueue()
    visited[start] = true
    enqueue(queue, start)

    while not empty(queue):
        current = dequeue(queue)
        process(current)

        for each neighbor of current:
            if not visited[neighbor]:
                visited[neighbor] = true
                enqueue(queue, neighbor)
```

**Ví dụ:**

```
Graph:
    A — B
    |   |
    C — D

BFS from A:
Level 0: A
Level 1: B, C (neighbors of A)
Level 2: D (neighbor of B and C)

Order: A, B, C, D
```

**Complexity:**

- Time: O(V + E)
- Space: O(V) - Queue

**Ứng dụng BFS:**

- Shortest path (unweighted graph)
- Level-order traversal
- Web crawler
- Social network (friends of friends)

### 12.2.5. Shortest Path - Dijkstra's Algorithm

**WHAT - Dijkstra**

**Dijkstra's Algorithm** tìm shortest path từ một source vertex đến tất cả vertices khác trong **weighted graph với edges không âm**.

**Ý tưởng:**

- Dùng **Greedy approach**
- Luôn chọn vertex với distance nhỏ nhất chưa được visit
- Dùng Priority Queue (Min Heap)

**Algorithm:**

```
Algorithm Dijkstra(graph, source):
    for each vertex v:
        dist[v] = ∞
        prev[v] = null

    dist[source] = 0
    pq = MinPriorityQueue()
    pq.insert((0, source))

    while not empty(pq):
        (d, u) = pq.extractMin()

        if d > dist[u]: continue  // Already processed

        for each neighbor v of u with weight w:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                prev[v] = u
                pq.insert((dist[v], v))

    return dist
```

**Complexity:**

- Time: O((V + E) log V) với Binary Heap, O($V^2$ + E) với array
- Space: O(V) - dist, prev arrays

**Lưu ý:** Không hoạt động với negative weights! (Dùng Bellman-Ford thay thế)

## 12.3. Bản chất trong máy tính

### 12.3.1. Memory Layout

**Adjacency Matrix:**

```
int matrix[V][V];

Memory: Contiguous 2D array
Size: V × V × sizeof(int) = 100 × 100 × 4 = 40KB (V=100)
```

**Adjacency List:**

```
vector<vector<int>> adj(V);

Memory: Array of vectors (scattered in heap)
Size: V × pointer + E × int
    = 100 × 8 + 500 × 4 = 2.8KB (V=100, E=500)
```

→ Adjacency List tiết kiệm nhiều với sparse graph!

### 12.3.2. DFS vs BFS - Stack vs Queue

**DFS (Stack):**

```
Visit A
Push B, C
Pop C
Visit C
Push D
Pop D
Visit D
Pop B
Visit B
```

**BFS (Queue):**

```
Visit A
Enqueue B, C
Dequeue B
Visit B
Enqueue D
Dequeue C
Visit C
Dequeue D
Visit D
```

## 12.4. Lịch sử / Nguồn gốc

**1736: Seven Bridges of Königsberg**

- Leonhard Euler
- First graph theory problem!
- Bắt đầu của graph theory

**1956: Dijkstra's Algorithm**

- Edsger W. Dijkstra
- Shortest path algorithm

**1957: Prim's Algorithm**

- Robert Prim
- Minimum Spanning Tree

**1959: DFS Algorithm**

- Formalized for computer science

**Ngày nay:**

- Graph algorithms everywhere!
- Google Maps, Facebook, Netflix recommendations
- Network routing, Compilers, AI

## 12.5. Phân tích thuật toán

Đã phân tích ở phần 2. Tóm tắt:

| **Algorithm** | **Time** | **Space** |
|---|---|---|
| DFS | O(V+E) | O(V) |
| BFS | O(V+E) | O(V) |
| Dijkstra (Binary Heap) | O((V+E) log V) | O(V) |
| Dijkstra (Array) | $O(V^2)$ | O(V) |
| Prim (Binary Heap) | O((V+E) log V) | O(V) |
| Kruskal | O(E log E) | O(V) |

## 12.6. Minh họa từng bước

### 12.6.1. Dijkstra's Algorithm

```
Graph:
    A —5— B
    |3    |2
    C —4— D

Find shortest path from A to all:

Initial: dist = [0, ∞, ∞, ∞]  (A=0, B=1, C=2, D=3)
         prev = [-, -, -, -]

Step 1: Process A (dist=0)
  Update B: dist[B] = 0 + 5 = 5
  Update C: dist[C] = 0 + 3 = 3
  dist = [0, 5, 3, ∞]
  prev = [-, A, A, -]

Step 2: Process C (smallest dist=3)
  Update D: dist[D] = 3 + 4 = 7
  dist = [0, 5, 3, 7]
  prev = [-, A, A, C]

Step 3: Process B (dist=5)
  Update D: dist[D] = min(7, 5+2) = 7 (no change)

Step 4: Process D (dist=7)
  Done!

Results:
  A→A: 0
  A→B: 5 (path: A→B)
  A→C: 3 (path: A→C)
  A→D: 7 (path: A→C→D)
```

## 12.7. Code minh họa C++

### 12.7.1. Graph Representation

```
#include <iostream>
#include <vector>
#include <queue>
#include <stack>
#include <climits>
using namespace std;

// Adjacency List
class Graph {
private:
    int V;  // Number of vertices
    vector<vector<int>> adj;  // Adjacency list

public:
    Graph(int vertices) : V(vertices) {
        adj.resize(V);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
    }

    void addEdgeUndirected(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    // DFS Recursive
    void DFSUtil(int v, vector<bool>& visited) {
        visited[v] = true;
        cout << v << " ";

        for (int neighbor : adj[v]) {
            if (!visited[neighbor]) {
                DFSUtil(neighbor, visited);
            }
        }
    }
```

```
    void DFS(int start) {
        vector<bool> visited(V, false);
        DFSUtil(start, visited);
        cout << endl;
    }

    // DFS Iterative (with stack)
    void DFSIterative(int start) {
        vector<bool> visited(V, false);
        stack<int> s;

        s.push(start);

        while (!s.empty()) {
            int v = s.top();
            s.pop();

            if (!visited[v]) {
                visited[v] = true;
                cout << v << " ";

                // Push neighbors to stack
                for (auto it = adj[v].rbegin(); it != adj[v].rend(); ++it) {
                    if (!visited[*it]) {
                        s.push(*it);
                    }
                }
            }
        }
        cout << endl;
    }

    // BFS
    void BFS(int start) {
        vector<bool> visited(V, false);
        queue<int> q;

        visited[start] = true;
        q.push(start);

        while (!q.empty()) {
            int v = q.front();
```

```
            q.pop();
            cout << v << " ";

            for (int neighbor : adj[v]) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    q.push(neighbor);
                }
            }
        }
        cout << endl;
    }
};

int main() {
    Graph g(5);

    g.addEdgeUndirected(0, 1);
    g.addEdgeUndirected(0, 2);
    g.addEdgeUndirected(1, 3);
    g.addEdgeUndirected(2, 3);
    g.addEdgeUndirected(3, 4);

    cout << "DFS from vertex 0: ";
    g.DFS(0);

    cout << "BFS from vertex 0: ";
    g.BFS(0);

    return 0;
}
```

### 12.7.2. Weighted Graph với Dijkstra

```
class WeightedGraph {
private:
    int V;
    vector<vector<pair<int, int>>> adj;  // {neighbor, weight}

public:
    WeightedGraph(int vertices) : V(vertices) {
        adj.resize(V);
```

```
    }

    void addEdge(int u, int v, int weight) {
        adj[u].push_back({v, weight});
    }

    void addEdgeUndirected(int u, int v, int weight) {
        adj[u].push_back({v, weight});
        adj[v].push_back({u, weight});
    }

    vector<int> dijkstra(int source) {
        vector<int> dist(V, INT_MAX);
        vector<int> prev(V, -1);

        // Min heap: (distance, vertex)
        priority_queue<pair<int, int>,
                       vector<pair<int, int>>,
                       greater<pair<int, int>>> pq;

        dist[source] = 0;
        pq.push({0, source});

        while (!pq.empty()) {
            int d = pq.top().first;
            int u = pq.top().second;
            pq.pop();

            // Skip if already processed
            if (d > dist[u]) continue;

            // Relax edges
            for (auto& [v, weight] : adj[u]) {
                if (dist[u] + weight < dist[v]) {
                    dist[v] = dist[u] + weight;
                    prev[v] = u;
                    pq.push({dist[v], v});
                }
            }
        }

        return dist;
```

```
    }

    void printPath(int source, int dest, vector<int>& prev) {
        if (dest == source) {
            cout << source;
            return;
        }
        if (prev[dest] == -1) {
            cout << "No path";
            return;
        }

        printPath(source, prev[dest], prev);
        cout << " → " << dest;
    }
};

int main() {
    WeightedGraph g(4);

    g.addEdgeUndirected(0, 1, 5);
    g.addEdgeUndirected(0, 2, 3);
    g.addEdgeUndirected(1, 3, 2);
    g.addEdgeUndirected(2, 3, 4);

    cout << "Dijkstra from vertex 0:" << endl;
    vector<int> dist = g.dijkstra(0);

    for (int i = 0; i < 4; i++) {
        cout << "Distance to " << i << ": " << dist[i] << endl;
    }

    return 0;
}
```

**Output:**

```
Dijkstra from vertex 0:
Distance to 0: 0
Distance to 1: 5
Distance to 2: 3
Distance to 3: 7
```

## 12.8. Lỗi phổ biến

### 1. DFS - Quên đánh dấu visited

```
// ❌ SAI: Infinite loop nếu có cycle
void DFS(int v) {
    cout << v << " ";
    for (int neighbor : adj[v]) {
        DFS(neighbor);  // Có thể quay lại v → Vô tận!
    }
}

// ✅ ĐÚNG
void DFS(int v, vector<bool>& visited) {
    visited[v] = true;
    cout << v << " ";
    for (int neighbor : adj[v]) {
        if (!visited[neighbor]) {
            DFS(neighbor, visited);
        }
    }
}
```

### 2. Dijkstra - Dùng với negative weights

```
// ❌ SAI: Dijkstra KHÔNG hoạt động với negative weights!
// Cần dùng Bellman-Ford thay thế
```

### 3. BFS - Quên check visited trước enqueue

```
// ❌ SAI: Enqueue nhiều lần
for (int neighbor : adj[v]) {
    if (!visited[neighbor]) {
        q.push(neighbor);
        // Quên: visited[neighbor] = true;
    }
}

// ✅ ĐÚNG: Mark visited ngay khi enqueue
for (int neighbor : adj[v]) {
    if (!visited[neighbor]) {
        visited[neighbor] = true;  // Mark trước
        q.push(neighbor);
```

```
    }
}
```

## 12.9. Khi nào dùng / không dùng

### 12.9.1. Matrix vs List

| Tiêu chí | Adjacency Matrix | Adjacency List |
|---|---|---|
| **Space** | $O(V^2)$ - Dense/Sparse | O(V+E) - Better for sparse |
| **Check edge** | O(1) | O(degree) |
| **Get neighbors** | O(V) | O(degree) |
| **Add edge** | O(1) | O(1) |
| **Remove edge** | O(1) | O(degree) |
| **Best for** | Dense graph, $E \approx V^2$ | Sparse graph, $E << V^2$ |

**Khi nào dùng Matrix:**

- Dense graph (nhiều edges)
- Cần check edge frequently
- V nhỏ (< 1000)

**Khi nào dùng List:**

- Sparse graph (ít edges) - Phổ biến trong thực tế!
- V lớn
- Cần iterate neighbors

### 12.9.2. DFS vs BFS

| Tiêu chí | DFS | BFS |
|---|---|---|
| **Data structure** | Stack/Recursion | Queue |
| **Path found** | Any path | Shortest path (unweighted) |
| **Memory** | O(h) - h=depth | O(w) - w=width |
| **Use cases** | Topological sort, Cycle detection | Shortest path, Level-order |

## 12.10. Ứng dụng thực tế

### 1. Social Networks (Facebook):

```
// Friend suggestions: BFS
vector<int> suggestFriends(int userID, Graph& network) {
    // Friends of friends (level 2)
    vector<int> suggestions;
    vector<bool> visited(network.V, false);
    queue<pair<int, int>> q;  // (vertex, level)

    q.push({userID, 0});
    visited[userID] = true;

    while (!q.empty()) {
        auto [v, level] = q.front();
        q.pop();

        if (level == 2) {
            suggestions.push_back(v);
        }

        if (level < 2) {
            for (int friend : network.adj[v]) {
                if (!visited[friend]) {
                    visited[friend] = true;
                    q.push({friend, level + 1});
                }
            }
        }
    }

    return suggestions;
}
```

### 2. GPS Navigation (Google Maps):

```
// Shortest path from home to work
vector<int> path = dijkstra(cityGraph, home, work);

// Alternative routes
vector<vector<int>> alternatives = kShortestPaths(cityGraph, home, work, 3);
```

```
// Avoid tolls/highways
Graph filteredGraph = removeEdges(cityGraph, tollRoads);
```

### 3. Web Crawler:

```
void crawlWeb(string startURL) {
    Graph web;
    queue<string> q;
    set<string> visited;

    q.push(startURL);
    visited.insert(startURL);

    while (!q.empty()) {
        string url = q.front();
        q.pop();

        downloadPage(url);

        for (string link : extractLinks(url)) {
            if (visited.find(link) == visited.end()) {
                visited.insert(link);
                q.push(link);
                web.addEdge(url, link);
            }
        }
    }
}
```

### 4. Compiler - Dependency Resolution:

```
// Topological sort để compile files theo đúng thứ tự
vector<string> compileOrder = topologicalSort(dependencyGraph);

// file1.cpp depends on file2.h
// file2.h depends on file3.h
// → Compile: file3.h → file2.h → file1.cpp
```

### 5. Network Routing:

```
// Find best route for data packet
vector<int> route = dijkstra(networkGraph, sourceRouter, destRouter);
```

```
// Load balancing: Find K shortest paths
// Fault tolerance: Find alternative paths
```

## 12.11. Tóm tắt

**Điểm quan trọng:**

1. **Graph G = (V, E):**
   - Vertices (đỉnh) và Edges (cạnh)
   - Directed/Undirected, Weighted/Unweighted
2. **Representations:**
   - Matrix: $O(V^2)$ space, O(1) check edge
   - List: O(V+E) space, O(degree) check edge
   - List tốt hơn cho sparse graph (thực tế)
3. **Traversals:**
   - DFS: Stack/Recursion, O(V+E), deep first
   - BFS: Queue, O(V+E), level-by-level
4. **Shortest Path:**
   - Unweighted: BFS
   - Weighted (positive): Dijkstra O((V+E) log V)
   - Weighted (negative): Bellman-Ford O(VE)
5. **Minimum Spanning Tree:**
   - Prim: O((V+E) log V)
   - Kruskal: O(E log E)
6. **Applications:**
   - Social networks
   - Maps/GPS
   - Web (PageRank)
   - Networks
   - Compilers
   - Games (pathfinding)
7. **Lỗi thường gặp:**
   - Quên visited → Infinite loop
   - Dijkstra với negative weights
   - BFS không mark visited khi enqueue
8. **Graph = Most versatile structure:**
   - Tree là special case của Graph
   - Có thể model hầu hết mọi thứ
9. **Complexity:**

- Traversals: O(V+E)
- Shortest path: O((V+E) log V)
- MST: O(E log V)

10. **C++ Implementation:**
    - `vector<vector<int>>` cho adjacency list
    - `priority_queue` cho Dijkstra
    - `queue` cho BFS, `stack` cho DFS